Polymath GML-DA Meetings and Notes

Polymath 2025 Generative Machine Learning Models for Data Assimilation Group

September 15, 2025

[Wiki Link]

Contents

1 Meeting Notes

1.1 Meeting 1 (6/27)

- Introductions across group members
- Reviewed intro slides

1.2 Meeting 2 (7/2)

- A question was asked about why we need an apriori form for T_β. The answer is that the OT problem is generally difficult, and there are many possibilities for the map. We choose a "good" (optimal, in the OT sense, as stated in the slides) form of the map. A theorem states that for our purposes, it is the gradient of a convex function.

  Since T_β needs to be optimal, and so we minimize KL over the set of gradients of convex functions (someone check if my understanding is correct).

- Given a bunch of points $(x_i.y_i)$, with $x_i \in \mathbb{R}^2, y_i \in \mathbb{R}$, can this interpolate a convex function?

  A related discussion on MathOverflow (thanks Tony): Existence of a strictly convex function interpolating given gradients and values. This suggests that we need a set of points and the gradients at these points.

  Basically, instead of finding T we can check convex feasibility.

1.3 Meeting 3 (7/12)

- goal of LP: check if given βs give a final convex function.
  Questions: Given a bunch of points $(x_i.y_i)_{i=1,...,n}$, with $x_i \in \mathbb{R}^d, y_i \in \mathbb{R}$, can this interpolate a convex function?
  Idea: Points $(x_i.y_i)$ can interpolate a convex function means that there exist a convex function $f : \mathbb{R}^d \to \mathbb{R}$ such that $f(x_i) = y_i$. We have such convex function if and only if for all $i = 1, ..., n$, there exist subgradient vector $g_i \in \mathbb{R}^d$ such that for all $j = 1, ..., n$

  $$y_j \geq y_i + g_i^T(x_j - x_i)$$

  This means for every point (x_i, y_i), there is a supporting hyperplane that passes through (x_i, y_i) and keeps all other points on or above it. (Prove: the necesary condition already follows from definition of that convex function. For sufficient condition: we can build a convex function $f(x) = \max_{i=1,..,n}\{y_i + g_i^T(x - x_i)\}$, and $f(x_i) = y_i$ since we have for all $k \neq i$, $y_k + g_k^T(x_i - x_k) \leq y_i$.)
  Now using this idea, we can construct a linear program: For $i, j = 1, ..., n$,

  $$y_j \geq y_i + g_i^T(x_j - x_i)$$

  where $x_i, x_j \in \mathbb{R}^d, y_i, y_j \in \mathbb{R}$ are known vectors and values. And only g_i's are unknown vectors. If this linear program is feasible, then we know there exist such subgradient vectors, so points can interpolate a convex function. If the linear program is not feasible, then points can not interpolate convex functions.

In the primal, there are totally N points to check, and there are N inequlaities for each point. So the computational cost would be high if we have large amount of points. To deal with that, we can check the dual form of inequalities for each points. We can do that by using Farkas' lemma:
Either $Ax \leq b$ has a solution $x \in \mathbb{R}^n$, or $A^T y = 0$ has a solution $y \geq 0$ with $b^T y < 0$.
Apply Farkas Lemma to our original inequalities for each point i, we have $y_j \geq y_i + g_i^T(x_j - x_i)$ where $\forall i$ is not feasible if and only if $\sum_i \alpha_i(x_i - x_j) = 0$ has a solution $\alpha \geq 0$ with $\sum_i \alpha_i(y_i - y_j) < 0$. Since CVXPY does not take strict inequality, we instead make a LP formation: $\min_{\alpha \geq 0} \sum_i \alpha_i(y_i - y_j)$ such that $\sum_i \alpha_i(x_i - x_j) = 0$. We check the LP is unbounded or not. If it is unbounded, then it means that $\sum_i \alpha_i(x_i - x_j) = 0$ has a solution, and then our original inequality is infeasible. Vice versa.

- goal of NF: implement NF(?) using LP blackbox for convexity

1.4 Meeting 6 (8/1)

- We are evaluating two approaches to choosing a basic map to implement the Normalizing Flows on: ICNNs and convex radial basis functions.

- Question (Xuan Zhang): If we have ICNNs and basis functions giving different convex functions, how do we know if we are optimal in an Optimal Transport sense? (See Brenier's Theorem.)
Answer: We don't know if it's optimal in the intermediate steps, but our goal is only optimality at the end.

- To Do: Implement an ICNN to maintain convexity between compositions. Further feasibility of convex radial basis functions. Start preparing for the PolyMath Jr closing presentations next weekend.

2 Appendix: Recordings, Slides, Notes

- **Introductory Zoom meeting (6/20):**
  [Google Drive]
  [Intro Slides]
- **Meeting 1 (6/27):**
  [Zoom Recording]
  Passcode: `mz9m^u+a`
- **Meeting 2 (7/2):**
  [Zoom Recording]
  Passcode: `JCS5f@iZ`

3 Appendix: Exposition on OT

Ethan wrote this before the second meeting as a helpful resource but is not necessary - you can treat it as a wiki of definitions

3.1 Preliminaries from Set Theory and Analysis

3.1.1 Sets and Functions

Let's begin with the fundamental building blocks of mathematics: sets and functions.

Definition 3.1 (Set)**.** *A **set** is a collection of distinct objects, considered as an object in its own right. The objects are called **elements** of the set.*

We use notation like $A = \{a, b, c\}$. Standard sets include the natural numbers $\mathbb{N}$, integers $\mathbb{Z}$, rational numbers $\mathbb{Q}$, and real numbers $\mathbb{R}$.

Definition 3.2 (Function)**.** *Given two sets X and Y, a **function** (or **map**) f from X to Y, denoted $f : X \to Y$, is a rule that assigns to each element $x \in X$ a unique element $y \in Y$, denoted $f(x)$.*

- **Injective (one-to-one)**: $f(x_1) = f(x_2) \implies x_1 = x_2$.
- **Surjective (onto)**: For every $y \in Y$, there exists an $x \in X$ such that $f(x) = y$.
- **Bijective**: Both injective and surjective.

3.1.2 The Real Numbers

The real numbers $\mathbb{R}$ form a complete ordered field. The most crucial property for real analysis is completeness.

Definition 3.3 (Supremum and Infimum)**.** *Let $S \subset \mathbb{R}$ be a non-empty set.*

- *If S is bounded above, its **supremum** $\sup(S)$ is its least upper bound.*
- *If S is bounded below, its **infimum** $\inf(S)$ is its greatest lower bound.*

Theorem 3.4 (Completeness Axiom)**.** *Every non-empty subset of $\mathbb{R}$ that is bounded above has a least upper bound (a supremum) in $\mathbb{R}$.*

3.1.3 Sequences of Real Numbers

Definition 3.5 (Convergence)**.** *A sequence $(x_n)_{n=1}^{\infty}$ of real numbers converges to a limit $L \in \mathbb{R}$ if for every $\epsilon > 0$, there exists an integer N such that for all $n \geq N$, we have $|x_n - L| < \epsilon$. We write $\lim_{n\to\infty} x_n = L$.*

Definition 3.6 (Cauchy Sequence)**.** *A sequence (x_n) is a **Cauchy sequence** if for every $\epsilon > 0$, there exists an N such that for all $m, n \geq N$, we have $|x_m - x_n| < \epsilon$.*

Theorem 3.7. *A sequence of real numbers converges if and only if it is a Cauchy sequence. This property is what makes $\mathbb{R}$ a **complete** space.*

3.2 Metric Spaces

We generalize concepts from $\mathbb{R}$ to more abstract spaces.

3.2.1 Basic Definitions

Definition 3.8 (Metric Space)**.** *A **metric space** is a pair (X, d) where X is a set and $d : X \times X \to [0, \infty)$ is a function (the **metric** or **distance**) satisfying for all $x, y, z \in X$:*

1. *$d(x, y) = 0 \iff x = y$ (identity of indiscernibles)*
2. *$d(x, y) = d(y, x)$ (symmetry)*
3. *$d(x, z) \leq d(x, y) + d(y, z)$ (triangle inequality)*

Example 3.9. *$\mathbb{R}^n$ with the Euclidean distance $d(x, y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$ is a metric space.*

3.2.2 Convergence and Topology

Convergence of a sequence (x_n) to x in a metric space is defined similarly: $\lim_{n\to\infty} d(x_n, x) = 0$.

Definition 3.10 (Open and Closed Sets)**.** *Let (X, d) be a metric space.*

- *An **open ball** with center $x \in X$ and radius $r > 0$ is $B(x, r) = \{y \in X : d(x, y) < r\}$.*
- *A set $U \subset X$ is **open** if for every $x \in U$, there exists $r > 0$ such that $B(x, r) \subset U$.*
- *A set $C \subset X$ is **closed** if its complement $X \setminus C$ is open.*

3.2.3 Completeness and Compactness

Definition 3.11 (Complete Metric Space)**.** *A metric space (X, d) is **complete** if every Cauchy sequence in X converges to a point in X. A complete normed vector space is called a **Banach space**.*

Definition 3.12 (Compactness)**.** *A subset K of a metric space is **compact** if every open cover of K has a finite subcover.*

In $\mathbb{R}^n$, the Heine-Borel theorem states that a set is compact if and only if it is closed and bounded.

3.3 Measure Theory and Integration

This chapter introduces the machinery of measure theory, which allows us to define the size of sets in a very general way and to integrate a broad class of functions.

3.3.1 σ-Algebras and Measurable Spaces

Definition 3.13 (σ-Algebra)**.** *Let X be a set. A collection $\mathcal{F}$ of subsets of X is a* ***σ-algebra*** *if it satisfies:*

1. $X \in \mathcal{F}$.
2. *If $A \in \mathcal{F}$, then its complement $A^c = X \setminus A$ is also in $\mathcal{F}$.*
3. *If $(A_n)_{n=1}^{\infty}$ is a sequence of sets in $\mathcal{F}$, then their union $\bigcup_{n=1}^{\infty} A_n$ is in $\mathcal{F}$.*

The pair $(X, \mathcal{F})$ is called a ***measurable space****.*

The smallest σ-algebra containing a collection of subsets $\mathcal{C} \subset 2^X$ is called the σ-algebra **generated by** $\mathcal{C}$, denoted $\sigma(\mathcal{C})$. For a metric space, the **Borel σ-algebra** $\mathcal{B}(X)$ is the one generated by the open sets.

3.3.2 Measures

Definition 3.14 (Measure)**.** *Given a measurable space $(X, \mathcal{F})$, a* ***measure*** *is a function $\mu : \mathcal{F} \to [0, \infty]$ such that:*

1. $\mu(\emptyset) = 0$.
2. *(Countable Additivity) For any sequence of disjoint sets $(A_n)_{n=1}^{\infty}$ in $\mathcal{F}$,*

$$\mu\left(\bigcup_{n=1}^{\infty} A_n\right) = \sum_{n=1}^{\infty} \mu(A_n).$$

The triple $(X, \mathcal{F}, \mu)$ is a ***measure space****.*

A key result is the Carathéodory extension theorem, which guarantees the existence of the **Lebesgue measure** on $\mathbb{R}^n$, which assigns the natural notion of volume to subsets of $\mathbb{R}^n$.

3.3.3 Measurable Functions

Definition 3.15 (Measurable Function)**.** *Let $(X, \mathcal{F})$ and $(Y, \mathcal{G})$ be measurable spaces. A function $f : X \to Y$ is* ***measurable*** *if for every $B \in \mathcal{G}$, the preimage $f^{-1}(B) = \{x \in X : f(x) \in B\}$ is in $\mathcal{F}$.*

When the codomain is $\mathbb{R}$, we assume it is equipped with its Borel σ-algebra.

3.3.4 The Lebesgue Integral

The integral is built in three steps:

1. For a **simple function** $\phi = \sum_{i=1}^{n} a_i \mathbf{1}_{A_i}$ (where A_i are disjoint measurable sets), $\int \phi \, d\mu = \sum_{i=1}^{n} a_i \mu(A_i)$.
2. For a non-negative measurable function $f \geq 0$, $\int f \, d\mu = \sup\{\int \phi \, d\mu : 0 \leq \phi \leq f, \phi \text{ is simple}\}$.
3. For a general measurable function f, we write $f = f^+ - f^-$ where $f^+ = \max(f, 0)$ and $f^- = \max(-f, 0)$. If $\int f^+ \, d\mu$ and $\int f^- \, d\mu$ are finite, f is **integrable** and $\int f \, d\mu = \int f^+ \, d\mu - \int f^- \, d\mu$.

3.3.5 Convergence Theorems

These theorems are fundamental for swapping limits and integrals. Let (f_n) be a sequence of measurable functions.

Theorem 3.16 (Monotone Convergence Theorem (MCT))**.** *If $0 \leq f_n \leq f_{n+1}$ for all n and $f_n \to f$ almost everywhere, then $\lim_{n\to\infty} \int f_n \, d\mu = \int f \, d\mu$.*

Theorem 3.17 (Dominated Convergence Theorem (DCT))**.** *If $f_n \to f$ almost everywhere, and there is an integrable function g such that $|f_n| \leq g$ for all n, then $\lim_{n\to\infty} \int f_n \, d\mu = \int f \, d\mu$.*

3.3.6 L^p Spaces

For $p \in [1, \infty)$, the L^p space is the set of measurable functions f for which $|f|^p$ is integrable.

Definition 3.18 (L^p Space)**.** *For a measure space $(X, \mathcal{F}, \mu)$ and $p \in [1, \infty)$,*

$$L^p(X, \mu) = \left\{ f : X \to \mathbb{R} \text{ measurable} : \int_X |f|^p \, d\mu < \infty \right\}.$$

This is a Banach space with the norm $\|f\|_{L^p} = \left(\int_X |f|^p \, d\mu\right)^{1/p}$.

For $p = 2$, $L^2(X, \mu)$ is a Hilbert space. For $p = \infty$, $L^\infty(X, \mu)$ is the space of essentially bounded functions.

3.4 Probability Theory

Probability theory is measure theory in a special setting where the total measure of the space is one. This framework is essential for modern statistics, stochastic processes, and optimal transport.

3.4.1 Probability Spaces

Definition 3.19 (Probability Space)**.** *A **probability space** is a measure space $(\Omega, \mathcal{F}, \mathbb{P})$ where $\mathbb{P}(X) = 1$. The measure $\mathbb{P}$ is called a **probability measure**. Elements of Ω are outcomes, and sets in $\mathcal{F}$ are events.*

Let $\mathcal{P}(X)$ denote the set of all probability measures on a measurable space $(X, \mathcal{F})$.

3.4.2 Random Variables and Distributions

Definition 3.20 (Random Variable)**.** *A **random variable** is a measurable function $X : \Omega \to \mathbb{R}$ from a probability space to the real numbers (equipped with the Borel σ-algebra).*

Definition 3.21 (Distribution)**.** *The **distribution** (or **law**) of a random variable X is the probability measure μ_X on $\mathbb{R}$ defined by $\mu_X(B) = \mathbb{P}(X^{-1}(B))$ for any Borel set $B \in \mathcal{B}(\mathbb{R})$. This is the **pushforward measure** of $\mathbb{P}$ by X, often written $X_{\#}\mathbb{P}$.*

3.4.3 Expectation

Definition 3.22 (Expectation)**.** *The **expectation** (or expected value) of a random variable X is its Lebesgue integral with respect to the probability measure $\mathbb{P}$:*

$$\mathbb{E}[X] = \int_{\Omega} X \, d\mathbb{P}.$$

If X has distribution μ_X, its expectation can also be computed as $\mathbb{E}[X] = \int_{\mathbb{R}} x \, d\mu_X(x)$.

3.4.4 Weak Convergence of Measures

A key concept in optimal transport is the notion of distance between probability measures, which relies on weak convergence.

Definition 3.23 (Weak Convergence)**.** *Let $(\mu_n)_{n=1}^{\infty}$ be a sequence of probability measures on a metric space X. We say that μ_n **converges weakly** to a probability measure μ, denoted $\mu_n \rightharpoonup \mu$, if for every bounded, continuous function $f : X \to \mathbb{R}$,*

$$\int_X f \, d\mu_n \to \int_X f \, d\mu.$$

Theorem 3.24 (Portmanteau Theorem)**.** *Let (μ_n) and μ be probability measures on a metric space X. The following are equivalent to weak convergence:*

- $\limsup_{n\to\infty} \mu_n(C) \leq \mu(C)$ *for all closed sets $C \subset X$.*
- $\liminf_{n\to\infty} \mu_n(U) \geq \mu(U)$ *for all open sets $U \subset X$.*

- $\lim_{n\to\infty} \mu_n(A) = \mu(A)$ *for all sets* A *with boundary* ∂A *satisfying* $\mu(\partial A) = 0$.

Prokhorov's theorem provides a condition for the existence of a weakly convergent subsequence, linking weak convergence to the notion of **tightness** of a family of measures.

3.5 Functional Analysis

Functional analysis is the study of infinite-dimensional vector spaces equipped with a topology. Its tools are essential for understanding the dual formulation of optimal transport.

3.5.1 Banach and Hilbert Spaces

Recall that a **normed vector space** is a vector space V with a norm $\|\cdot\|$. A complete normed space is a **Banach space**. An inner product space whose induced norm makes it a complete space is a **Hilbert space**. We have already seen that $L^p(X,\mu)$ spaces are prime examples of Banach spaces (and Hilbert spaces for $p=2$).

3.5.2 Dual Spaces

The concept of duality is central to optimization and functional analysis.

Definition 3.25 (Continuous Dual Space)**.** *Let X be a normed vector space. The space of all continuous linear functionals $\phi : X \to \mathbb{R}$ is called the* ***continuous dual space*** *of X, denoted by X^*.*

For $f \in X^*$, its norm is defined as

$$\|f\|_{X^*} = \sup_{\|x\|_X \le 1} |f(x)|.$$

The dual space X^* is always a Banach space, even if X is not.

3.5.3 The Riesz Representation Theorem

This theorem (or rather, theorems) provides a concrete identification of the dual space for many important spaces.

Theorem 3.26 (Riesz Representation for L^p)**.** *Let $(X, \mathcal{F}, \mu)$ be a σ-finite measure space and let $1 \le p < \infty$. Let q be the conjugate exponent, i.e., $1/p + 1/q = 1$. Then the dual space $(L^p(\mu))^*$ is isometrically isomorphic to $L^q(\mu)$. This means that for every continuous linear functional $\phi \in (L^p(\mu))^*$, there exists a unique function $g \in L^q(\mu)$ such that*

$$\phi(f) = \int_X f g \, d\mu \quad \text{for all } f \in L^p(\mu).$$

A second version of this theorem is even more crucial for optimal transport, as it relates measures to functionals on the space of continuous functions.

Theorem 3.27 (Riesz-Markov-Kakutani Representation Theorem)**.** *Let X be a compact Hausdorff space. Let $C(X)$ be the Banach space of continuous functions on X with the supremum norm. Then the dual space $(C(X))^*$ is isometrically isomorphic to the space of signed regular Borel measures on X, denoted $\mathcal{M}(X)$. For every continuous linear functional $\phi \in (C(X))^*$, there exists a unique regular signed Borel measure μ on X such that*

$$\phi(f) = \int_X f \, d\mu \quad \text{for all } f \in C(X).$$

Furthermore, $\|\phi\|_{(C(X))^*} = |\mu|(X)$, the total variation of the measure.

.

3.6 Convex Analysis and Optimization

The modern formulation of optimal transport (due to Kantorovich) is an infinite-dimensional linear program, and its analysis relies on the theory of convex duality.

3.6.1 Convex Sets and Functions

Definition 3.28 (Convex Set)**.** *A set C in a vector space is **convex** if for any $x, y \in C$ and any $t \in [0,1]$, the point $tx + (1-t)y$ is also in C.*

Definition 3.29 (Convex Function)**.** *A function $f : C \to \mathbb{R}$ defined on a convex set C is **convex** if for any $x, y \in C$ and any $t \in [0,1]$,*

$$f(tx + (1-t)y) \leq tf(x) + (1-t)f(y).$$

*A function f is **concave** if $-f$ is convex.*

3.6.2 Convex Duality and Legendre Transform

A central concept in convex analysis is the dual representation of a function.

Definition 3.30 (Legendre-Fenchel Conjugate)**.** *Let $f : \mathbb{R}^n \to \mathbb{R} \cup \{\infty\}$. Its **Legendre-Fenchel conjugate** is the function $f^* : \mathbb{R}^n \to \mathbb{R} \cup \{\infty\}$ defined by*

$$f^*(p) = \sup_{x \in \mathbb{R}^n} \{\langle p, x \rangle - f(x)\}.$$

The conjugate f^* is always a convex function. For a convex, lower semi-continuous function f, we have $f^{**} = f$. This duality is key to Brenier's theorem in optimal transport.

3.6.3 Linear Programming Duality

A linear program (LP) is an optimization problem with a linear objective and linear constraints. Consider the primal LP:

$$\text{Primal (P):} \quad \min_x \quad c^T x \quad \text{subject to } Ax \geq b, x \geq 0.$$

The corresponding dual LP is:

$$\text{Dual (D):} \quad \max_y \quad b^T y \quad \text{subject to } A^T y \leq c, y \geq 0.$$

Theorem 3.31 (Weak Duality)**.** *For any feasible solution x for (P) and any feasible solution y for (D), we have $c^T x \geq b^T y$.*

Theorem 3.32 (Strong Duality)**.** *If a linear program has an optimal solution, then so does its dual, and the respective optimal objective values are equal.*

This principle of strong duality for linear programs, extended to infinite-dimensional spaces, is the foundation of Kantorovich's solution to the optimal transport problem.

3.7 The Optimal Transport Problem

We have finally arrived at the central topic. Optimal Transport (OT) is a problem about finding the most efficient way to move a distribution of mass from one configuration to another.

3.7.1 The Monge Problem

Let $\mu, \nu \in \mathcal{P}(\mathbb{R}^n)$ be two probability measures. Let $c(x, y) : \mathbb{R}^n \times \mathbb{R}^n \to [0, \infty)$ be a cost function, representing the cost of moving a unit of mass from location x to y. A common choice is $c(x, y) = \|x - y\|^p$ for $p \geq 1$.

Gaspard Monge (1781) posed the problem of finding a **transport map** $T : \mathbb{R}^n \to \mathbb{R}^n$ that pushes μ forward to ν (i.e., $T_{\#}\mu = \nu$) while minimizing the total transportation cost.

$$\inf_{T:T_{\#}\mu=\nu} \int_{\mathbb{R}^n} c(x, T(x))\, d\mu(x).$$

The condition $T_{\#}\mu = \nu$ means that for any Borel set B, $\mu(T^{-1}(B)) = \nu(B)$.

Monge's problem is often ill-posed: a solution may not exist (e.g., if μ is a Dirac mass and ν is a sum of two Dirac masses).

3.7.2 The Kantorovich Relaxation

Leonid Kantorovich (1940s) relaxed Monge's problem by optimizing over **transport plans** instead of transport maps. A transport plan is a joint probability measure $\pi \in \mathcal{P}(\mathbb{R}^n \times \mathbb{R}^n)$ whose marginals are μ and ν.

Let $\Pi(\mu, \nu)$ be the set of all such transport plans. The Kantorovich problem is:

$$\mathcal{C}(\mu, \nu) = \inf_{\pi \in \Pi(\mu,\nu)} \int_{\mathbb{R}^n \times \mathbb{R}^n} c(x, y)\, d\pi(x, y).$$

This is a linear programming problem. Its feasible set $\Pi(\mu, \nu)$ is always non-empty and it can be shown that an optimal plan π_* always exists under weak assumptions on the cost c.

3.7.3 Kantorovich Duality

The power of Kantorovich's formulation comes from its dual problem. Applying the duality theory for infinite-dimensional LPs, we get:

$$\sup_{(\phi,\psi)\in\Phi_c} \left\{ \int_{\mathbb{R}^n} \phi(x)\, d\mu(x) + \int_{\mathbb{R}^n} \psi(y)\, d\nu(y) \right\},$$

where Φ_c is the set of pairs of continuous bounded functions (ϕ, ψ) such that $\phi(x) + \psi(y) \leq c(x, y)$ for all x, y. Strong duality holds, meaning the value of the primal and dual problems are equal.

3.7.4 Brenier's Theorem

A fundamental result connects the Monge and Kantorovich problems for the squared Euclidean cost $c(x, y) = \|x - y\|^2$.

Theorem 3.33 (Brenier's Theorem, 1991)**.** *Let $\mu, \nu \in \mathcal{P}(\mathbb{R}^n)$, with μ being absolutely continuous with respect to the Lebesgue measure. For the cost $c(x, y) = \|x-y\|^2$, there exists a unique optimal transport plan π_*. This plan is of the Monge form, i.e., it is concentrated on the graph of a map T_*:*

$$\pi_* = (Id \times T_*)_{\#}\mu.$$

Furthermore, the optimal map T_* is the gradient of a convex function ϕ, i.e., $T_*(x) = \nabla\phi(x)$.

This theorem establishes a connection between optimal transport, convex analysis, and fluid mechanics (via the Monge-Ampère equation that ϕ must satisfy).

3.7.5 The Wasserstein Distance

The optimal transport cost can be used to define a distance on the space of probability measures.

Definition 3.34 (Wasserstein Distance)**.** *For $p \geq 1$, the p-**Wasserstein distance** between $\mu, \nu \in \mathcal{P}_p(\mathbb{R}^n)$ (the space of probability measures with finite p-th moment) is*

$$W_p(\mu, \nu) = \left(\inf_{\pi \in \Pi(\mu,\nu)} \int_{\mathbb{R}^n \times \mathbb{R}^n} \|x - y\|^p \, d\pi(x, y) \right)^{1/p}.$$

The space $(\mathcal{P}_p(\mathbb{R}^n), W_p)$ is a complete metric space. Convergence in the W_p metric implies weak convergence of measures, making it a powerful tool for analyzing probability distributions.

3.8 Partial Differential Equations and Optimal Transport

One of the most profound discoveries in the modern theory of optimal transport is its intimate connection to certain non-linear partial differential equations.

3.8.1 The Monge-Ampère Equation

Recall from Brenier's theorem that for the quadratic cost $c(x, y) = \|x - y\|^2$, the optimal map is the gradient of a convex function, $T(x) = \nabla\phi(x)$. What equation must this potential function ϕ satisfy?

The transport map T must satisfy the change of variables formula to ensure that the mass is correctly transported. If μ and ν have densities f and g with respect to the Lebesgue measure, this condition becomes:

$$\det(\nabla T(x))f(x) = g(T(x)).$$

Since $T(x) = \nabla\phi(x)$, the matrix $\nabla T(x)$ is the Hessian of ϕ, denoted $D^2\phi(x)$. Substituting this into the change of variables formula gives the celebrated **Monge-Ampère equation**:

$$\det(D^2\phi(x)) = \frac{g(\nabla\phi(x))}{f(x)}.$$

This is a highly non-linear, second-order PDE for the potential ϕ.

3.8.2 The Benamou-Brenier Formulation

The connection to PDEs can also be seen through a time-dependent formulation of optimal transport. Benamou and Brenier showed that the squared Wasserstein distance $W_2^2(\mu_0, \mu_1)$ can be found by solving a fluid dynamics problem.

Consider a distribution of mass $\rho(t, x)$ that evolves over time $t \in [0, 1]$ from an initial density ρ_0 to a final density ρ_1. If this evolution is driven by a velocity field $v(t, x)$, the density must satisfy the continuity equation:

$$\partial_t\rho + \nabla \cdot (\rho v) = 0.$$

The Benamou-Brenier formula states that

$$W_2^2(\mu_0, \mu_1) = \inf \int_0^1 \int_{\mathbb{R}^n} \rho(t, x)\|v(t, x)\|^2 \, dx \, dt,$$

where the infimum is taken over all pairs (ρ, v) that satisfy the continuity equation with boundary conditions $\rho(0, \cdot) = \rho_0$ and $\rho(1, \cdot) = \rho_1$. This recasts optimal transport as a problem of finding the path of minimum kinetic energy for a fluid to move from one state to another.

4 Appendix: PDFs

4.1 Intro Slides Notes (James)

On the next page!

Generative Modeling Slides

James Evans

07/02/2025

ML Project: Generative modeling

Ricardo Baptista, Suvedei Soyolerdene, Giulio Trigila, and Tanya Wang

Caltech, Baruch College CUNY, and NYU

PolyMathJr Summer Program

June 20, 2025

Probabilistic modeling

- Given data $\{\mathbf{z}^i\}$ sampled from an unknown probability density ρ, we would like to estimate $\rho(\mathbf{z})$
- Example: what is the distribution of the heights among the high school students in NYC?
- A more complicated example: given samples $\{(\mathbf{z}^i, \mathbf{y}^i)\}$ we would like to estimate the conditional probability density $\rho(\mathbf{z}|\mathbf{y})$
- Example: given that New York is located at 40.7128° N, 74.0060° W on the sea level and that tomorrow is June 22 (these are the factors $\mathbf{y}_i \in \mathbb{R}^m$) what is the probability that the temperature (i.e., the outcome $z \in \mathbb{R}$) is going to be 25 degrees Celsius?

We use probabilistic models to describe certain outputs (e.g. from a physical system) and make predictions about the future.

Large-scale probabilistic models

- **Applications**: Numerical weather prediction, GPS tracking, infectious disease spread, financial market analysis, etc.

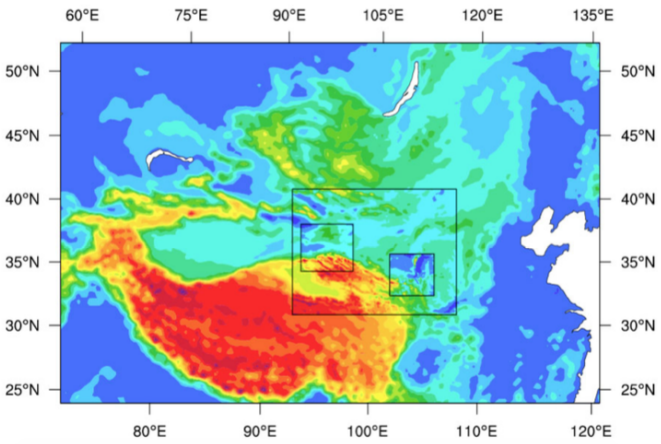


Figure: Source: NCAR ensemble wind forecast

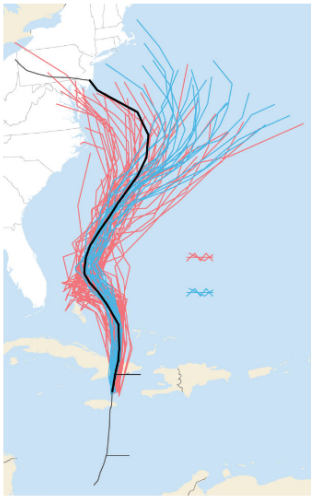


Figure: Source: Wall Street Journal

Main idea behind Mapping Methods

Most of the work related to Mapping methods is based on KL minimization

$$KL[\rho||\mu] = \int \log(\rho(x)/\mu(x))\rho(x)dx$$

- We want to find a map from ρ, known only through samples $\{x_i\}$, to a reference density $\mu = \mathcal{N}(0, I)$.
- Let $\mu = T_{\#}\tilde{\rho}$ and consider $KL[\rho||\tilde{\rho}] = KL[T] = \int \rho \log(\rho/\tilde{\rho})$
- Parametrize $T = T_\beta$, minimize $KL[T_\beta] = \text{const} - \int \rho \log(\tilde{\rho})$ over the parameters $\beta \Rightarrow \bar{\beta}$
- Use the change of variable formula to estimate ρ:

$$\rho(x) = J^G(x)\mu(G(x)) \text{ where } G = T_{\bar{\beta}}$$

*Tabak, Esteban G., and Eric Vanden-Eijnden. "Density estimation by dual ascent of the log-likelihood." Communications in Mathematical Sciences 8.1 (2010): 217-233.
**El Moselhy, Tarek A., and Youssef M. Marzouk. "Bayesian inference with optimal maps." Journal of Computational Physics 231.23 (2012): 7815-7850.

Detailed Explanation of Mapping Methods via KL Minimization

We are given samples $\{x_i\} \sim \rho$, where ρ is an unknown target density. The key idea of mapping methods is to estimate ρ by learning a transformation from a known reference density μ, typically $\mu = \mathcal{N}(0, I)$, using KL divergence minimization.

Starting Point: KL Divergence

The Kullback–Leibler divergence between two densities ρ and μ is defined as:

$$KL[\rho \,\|\, \mu] = \int \log\left(\frac{\rho(x)}{\mu(x)}\right) \rho(x)\, dx$$

This measures how different ρ is from μ. Since we only have access to samples from ρ, this formulation is not immediately useful.

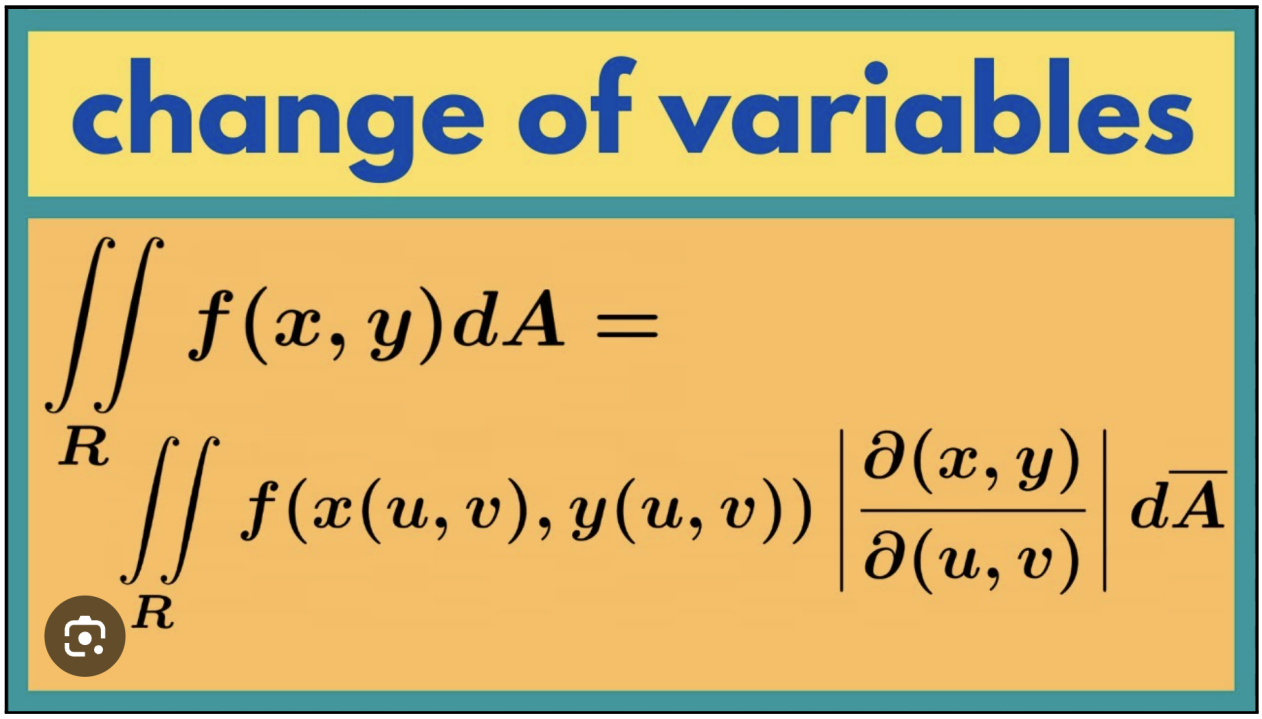
Introduce an Auxiliary Density and Pushforward Map

We define a new density $\tilde{\rho}$ on $\mathbb{R}^d$ and a smooth, invertible map $T : \mathbb{R}^d \to \mathbb{R}^d$ such that:

$$\mu = T_{\#}\tilde{\rho}$$

Here, $T_{\#}\tilde{\rho}$ denotes the *pushforward* of $\tilde{\rho}$ through T, i.e., the distribution of $T(x)$ when $x \sim \tilde{\rho}$. This construction allows us to represent the known reference density μ in terms of an unknown $\tilde{\rho}$ and a transformation T.

The change-of-variable formula tells us that:



Reformulating the KL Divergence

We now consider the Kullback–Leibler (KL) divergence between two densities:

$$\mathrm{KL}[\rho \,\|\, \mu] = \int \rho(x) \log\left(\frac{\rho(x)}{\mu(x)}\right) dx$$

Suppose $\mu = T_{\#}\tilde{\rho}$. Then, by the change of variables formula:

$$\mu(z) = \tilde{\rho}(T^{-1}(z)) \left|\det\left(\frac{\partial T^{-1}}{\partial z}(z)\right)\right|$$

To simplify notation, we now relabel variables and think of $x = z$ as the input to μ. So we define $\tilde{\rho}(x)$ as a proxy density that we want to match $\rho(x)$ against.

We now consider:

$$\mathrm{KL}[\rho \,\|\, \tilde{\rho}] = \int \rho(x) \log\left(\frac{\rho(x)}{\tilde{\rho}(x)}\right) dx$$

This KL divergence is still intractable because we don't know $\rho(x)$, only samples from it. However, we can rewrite $\tilde{\rho}(x)$ using the change-of-variable expression:

$$\tilde{\rho}(x) = \mu(T(x)) \cdot \left|\det\left(\frac{\partial T}{\partial x}(x)\right)\right|$$

This allows us to treat the KL divergence as a function of the transformation T. That is,

$$\mathrm{KL}[T] = \int \rho(x) \log\left(\frac{\rho(x)}{\mu(T(x)) \cdot \left|\det\left(\frac{\partial T}{\partial x}(x)\right)\right|}\right) dx$$

Parametric Families of Maps

In practice, we cannot optimize over all possible functions T, so we restrict ourselves to a parametric family of maps T_β, where $\beta \in \mathbb{R}^p$ is a vector of parameters (such as weights in a neural network or coefficients of a polynomial map).

Thus, we now write the divergence as:

$$KL[T_\beta] = \int \rho(x) \log \left(\frac{\rho(x)}{\mu(T_\beta(x)) \cdot \left| \det \left(\frac{\partial T_\beta}{\partial x}(x) \right) \right|} \right) dx$$

Since ρ is unknown but we have samples $\{x_i\} \sim \rho$, we approximate the expectation by empirical average:

Explaining the Matrix

The term $\frac{\partial T}{\partial x}(x)$ is the **Jacobian matrix** of the vector-valued function $T(x)$.

If:

$$T(x) = \begin{bmatrix} T_1(x) \\ T_2(x) \\ \vdots \\ T_d(x) \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

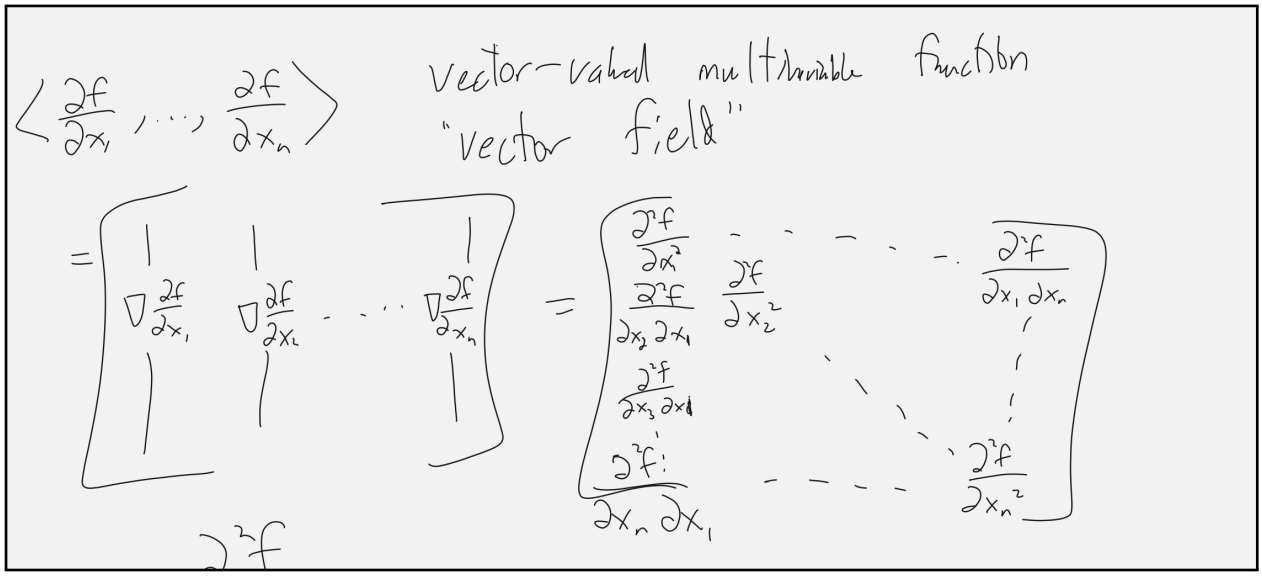
Then the Jacobian matrix is:

$$J_T(x) = \begin{bmatrix} \frac{\partial T_1}{\partial x_1} & \frac{\partial T_1}{\partial x_2} & \cdots & \frac{\partial T_1}{\partial x_d} \\ \frac{\partial T_2}{\partial x_1} & \frac{\partial T_2}{\partial x_2} & \cdots & \frac{\partial T_2}{\partial x_d} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial T_d}{\partial x_1} & \frac{\partial T_d}{\partial x_2} & \cdots & \frac{\partial T_d}{\partial x_d} \end{bmatrix} \in \mathbb{R}^{d \times d}$$

Note: If $T(x) = \nabla \phi(x)$, then the Jacobian matrix of T, denoted $J_T(x)$, is exactly the Hessian matrix of the scalar function ϕ:

$$\nabla \phi(\mathbf{z}) = \begin{bmatrix} \frac{\partial \phi}{\partial z_1} \\ \frac{\partial \phi}{\partial z_2} \\ \vdots \\ \frac{\partial \phi}{\partial z_d} \end{bmatrix} \in \mathbb{R}^d$$

$$J_T(x) = D^2 \phi(x)$$



Why We Drop $\rho(x)$ in KL Divergence Approximation

We begin with the KL divergence between a true (unknown) data distribution $\rho(x)$ and a model-induced density $\tilde{\rho}(x)$:

$$KL[\rho \| \tilde{\rho}] = \int \rho(x) \log \left(\frac{\rho(x)}{\tilde{\rho}(x)} \right) dx$$

In our setting, we define:

$$\tilde{\rho}(x) = \mu(T_\beta(x)) \cdot \left| \det \left(\frac{\partial T_\beta}{\partial x}(x) \right) \right|$$

so the divergence becomes:

$$KL[T_\beta] = \int \rho(x) \log \left(\frac{\rho(x)}{\mu(T_\beta(x)) \cdot \left| \det \left(\frac{\partial T_\beta}{\partial x}(x) \right) \right|} \right) dx$$

Using properties of logarithms, we rewrite:

$$KL[T_\beta] = \int \rho(x) \log \rho(x)\, dx - \int \rho(x) \log \mu(T_\beta(x))\, dx - \int \rho(x) \log \left| \det \left(\frac{\partial T_\beta}{\partial x}(x) \right) \right| dx$$

Now, suppose we have samples $\{x_i\}_{i=1}^n \sim \rho$. Then we approximate the remaining expectations using the Monte Carlo estimation:

$$\int \rho(x) f(x)\, dx \approx \frac{1}{n} \sum_{i=1}^{n} f(x_i)$$

Applying this, we get:

$$KL[T_\beta] \approx \text{const} - \frac{1}{n} \sum_{i=1}^{n} \log \mu(T_\beta(x_i)) - \frac{1}{n} \sum_{i=1}^{n} \log \left| \det \left(\frac{\partial T_\beta}{\partial x}(x_i) \right) \right|$$

Since the constant term $\int \rho(x) \log \rho(x)\, dx$ is independent of β, it can be omitted from the objective when optimizing. Therefore, we minimize:

$$KL[T_\beta] \approx \frac{1}{n}\sum_{i=1}^{n} \log\left(\frac{1}{\mu(T_\beta(x_i)) \cdot \left|\det\left(\frac{\partial T_\beta}{\partial x}(x_i)\right)\right|}\right)$$

Optimization

We minimize this approximate KL divergence with respect to β. Let $\bar{\beta}$ denote the optimal parameters:

$$\bar{\beta} = \arg\min_{\beta} KL[T_\beta]$$

Once we have the optimal map $G = T_{\bar{\beta}}$, we can estimate the original density ρ via:

$$\rho(x) = J^G(x) \cdot \mu(G(x)), \quad \text{where } J^G(x) = \left|\det\left(\frac{\partial G}{\partial x}(x)\right)\right|$$

This follows directly from the change-of-variable formula: we are pulling back the known density μ through the learned inverse map G to estimate ρ.

Mapping method

- So far we need an a-priori parametrization T_β rich enough to map ρ to a reference pdf (standard normal, say)
- Coming up with T_β is not an easy task $\rightarrow$ deep neural network
- The task becomes even harder if we want to impose specific characteristics on T_β like being optimal (theory of optimal transport) or triangular

$$\Downarrow$$

Normalizing Flows

Motivation for Normalizing Flows

Directly learning a single transformation $T_\beta : \mathbb{R}^d \to \mathbb{R}^d$ that maps a complex data distribution ρ to a simple reference distribution (such as a standard Gaussian) is a highly nontrivial task. Even with a deep neural

network, parameterizing such a function can be difficult due to issues with invertibility, stability, and efficient computation of the Jacobian determinant.

To address this, normalizing flows decompose the overall transformation into a sequence of simpler, invertible maps:

$$T = T_n \circ T_{n-1} \circ \cdots \circ T_1,$$

where each T_k is an easily parameterizable and tractable transformation (e.g., an affine coupling layer or autoregressive layer).
This layered construction offers several advantages:

- Each T_k can be made invertible by design, ensuring that the full map T is invertible.
- The Jacobian determinant of each T_k can be computed efficiently, allowing the use of the change-of-variables formula for exact density evaluation.
- The model becomes highly expressive while maintaining tractable training and sampling procedures.

Thus, rather than attempting to learn one large and complex transformation, normalizing flows build expressive models by composing multiple simple and structured transformations — making the learning process more stable, efficient, and interpretable.

Normalizing Flows (NF)

Main idea

Find $T = T_n \circ ... \circ T_1$ as a composition of elementary maps, easier to parametrize $T_k = T_{\beta^k}$

- Find T descending $KL[T(.,t)] = \int \rho \log(\rho/\tilde{\rho}_t)$:

$$\frac{dT(x,t)}{dt} = -\frac{\delta KL[\phi \circ T(.,t)]}{\delta \phi}\bigg|_{\phi=id} \tag{1}$$

where $T(x, t=0) = x$ and $\tilde{\rho}_t = J^{T(x,t)}\mu(T(x,t))$

In the end we have that $\tilde{\rho}_{t=0} = \mu$ and $\tilde{\rho}_\infty = \rho$

* Tabak, Esteban G., and Cristina V. Turner. "A family of nonparametric density estimation algorithms." Communications on Pure and Applied Mathematics 66.2 (2013): 145-164.
**Rezende, Danilo, and Shakir Mohamed. "Variational inference with normalizing flows." International conference on machine learning. PMLR, 2015.

Baptista, Trigila, Wang PolyMathJr 2025: ML project 6 / 13

Normalizing Flows

This slide explains how to transform one probability distribution into another using a series of simple, invertible functions. This is the key idea behind **Normalizing Flows (NF)**.

Main Idea

We want to build a complex transformation T by composing simpler ones:

$$T = T_n \circ T_{n-1} \circ \cdots \circ T_1$$

- It is difficult to directly design a function that maps a simple distribution (e.g., standard Gaussian) to a complex target distribution (e.g., real data).
- However, if we break the transformation into multiple steps, each step becomes easier to handle.
- Each map T_k can be parameterized by neural networks, denoted $T_k = T_{\beta_k}$, and trained using gradient-based optimization.

This is the fundamental strategy behind normalizing flows.

KL Divergence as the Objective

We want to learn a transformation T such that

$$T_{\#}\mu \approx \rho,$$

where:

- μ is the base (known) distribution, like a standard normal,
- ρ is the target distribution (e.g., empirical data),
- $T_{\#}\mu$ denotes the pushforward of μ by T, i.e., the distribution obtained by applying T to samples from μ.

To measure how close our current model $\tilde{\rho}_t$ is to the target ρ, we use the Kullback-Leibler divergence:

$$KL[\rho \| \tilde{\rho}_t] = \int \rho(x) \log \left(\frac{\rho(x)}{\tilde{\rho}_t(x)} \right) dx$$

Our goal is to find a transformation T that minimizes this KL divergence.

Key Equation: Functional Gradient Descent

To evolve T in a direction that reduces the KL divergence, we use:

$$\frac{dT(x,t)}{dt} = - \left. \frac{\delta KL[\phi \circ T(\cdot, t)]}{\delta \phi} \right|_{\phi = \text{id}} \tag{1}$$

Explanation:

- $T(x,t)$ is a time-dependent transformation.
- At $t = 0$, we initialize with $T(x, 0) = x$ (the identity map).
- The right-hand side is the functional derivative of KL divergence, which tells us how to adjust T to decrease KL most quickly.
- ϕ is a small perturbation applied to T, and we evaluate the derivative at $\phi = \text{id}$ (identity function).

This is analogous to gradient descent in parameter space, except we are descending in the space of functions. In normalizing flows, we don't typically work directly with this equation. Instead, it serves as a high-level description of how the transformation T should evolve over time to minimize KL divergence — guiding the flow in the infinite-dimensional space of functions.

Evolving the Density

At time t, the current density is given by:

$$\tilde{\rho}_t(x) = J^{T(x,t)}\mu(T(x,t)),$$

where $J^{T(x,t)}$ is the Jacobian determinant of the transformation T. This tells us how the probability density changes as we move through the transformation.

Initial and Final Conditions

We start with the base distribution:

$$\tilde{\rho}_{t=0} = \mu$$

and aim to reach the target:

$$\tilde{\rho}_{t\to\infty} = \rho$$

So as t increases, the transformation $T(x,t)$ warps the simple base distribution into the complex target distribution.

Example of Parameterizing the Flow

In the context of normalizing flows, we aim to construct a complex transformation

$$T = T_n \circ T_{n-1} \circ \cdots \circ T_1$$

that maps a simple base distribution μ, such as a standard Gaussian, to a complex target distribution ρ. Rather than defining each component map T_k by hand, we parameterize them using neural networks, enabling data-driven learning of the transformation.

Each map $T_k : \mathbb{R}^d \to \mathbb{R}^d$ is chosen to be invertible and differentiable. A widely used choice is the *affine coupling layer*, introduced in the RealNVP architecture. This map splits the input $x \in \mathbb{R}^d$ into two halves:

$$x = (x_a, x_b), \quad \text{where } x_a, x_b \in \mathbb{R}^{d/2}.$$

The transformation is then defined as:

$$\begin{aligned} y_a &= x_a \\ y_b &= x_b \odot \exp(s_\theta(x_a)) + t_\theta(x_a) \end{aligned}$$

Here:

- $s_\theta : \mathbb{R}^{d/2} \to \mathbb{R}^{d/2}$ is a neural network producing a log-scale vector,
- $t_\theta : \mathbb{R}^{d/2} \to \mathbb{R}^{d/2}$ is a neural network producing a shift vector,
- $\odot$ denotes elementwise multiplication,
- θ is the shared set of learnable parameters across both networks.

The exponential is applied elementwise to $s_\theta(x_a)$ to ensure positivity of the scaling factors, which guarantees invertibility of the transformation T_k. The overall flow step T_k is thus defined in terms of a pair of neural networks, both parameterized by θ_k. We write $T_k = T_{\theta_k}$, and collect all such parameters into $\beta = (\theta_1, \ldots, \theta_n)$.

Triangular Jacobian. The Jacobian matrix $J_{T_k}(x)$ of an affine coupling layer is lower triangular:

$$J_{T_k}(x) = \begin{bmatrix} I & 0 \\ * & \operatorname{diag}\left(\exp(s_\theta(x_a))\right) \end{bmatrix}$$

This triangular structure makes the determinant easy to compute:

$$|\det J_{T_k}(x)| = \prod_{j=1}^{d/2} \exp(s_\theta(x_a)_j) = \exp\left(\sum_{j=1}^{d/2} s_\theta(x_a)_j\right)$$

Training via Maximum Likelihood

Given a dataset $\{x^{(i)}\}_{i=1}^N$ sampled from the target distribution ρ, we aim to train the flow $T = T_n \circ \cdots \circ T_1$ such that the pushforward distribution $\tilde{\rho} := T_{\#}\mu$ closely approximates ρ.

This is done by maximizing the log-likelihood of the data under the model, or equivalently minimizing the negative log-likelihood:

$$\mathcal{L}(\beta) = -\sum_{i=1}^{N} \log \tilde{\rho}(x^{(i)}),$$

where β collects all parameters across all flow steps.

Change-of-Variables Formula

To compute $\tilde{\rho}(x)$, we apply the change-of-variables formula. Let $z = T^{-1}(x)$, then:

$$\tilde{\rho}(x) = \mu(z) \cdot \left| \det \left(\frac{\partial T^{-1}}{\partial x} \right) \right| = \mu(T^{-1}(x)) \cdot \left| \det J_{T^{-1}}(x) \right|.$$

Substituting this into the loss gives:

$$\mathcal{L}(\beta) = -\sum_{i=1}^{N} \left[\log \mu(T^{-1}(x^{(i)})) + \log \left| \det J_{T^{-1}}(x^{(i)}) \right| \right].$$

Intuition Behind the Loss

The loss function consists of two interpretable terms:

- $\log \mu(T^{-1}(x))$ encourages the model to learn transformations T that push observed data toward regions of high probability under the base distribution μ (typically a Gaussian).
- The Jacobian determinant $|\det J_{T^{-1}}(x)|$ corrects for how much the transformation stretches or shrinks volume locally, ensuring proper density mass is preserved.

Together, these terms ensure that the model learns a transformation that both matches the shape and volume distortion of the target distribution.

Optimization

The loss $\mathcal{L}(\beta)$ is differentiable with respect to the neural network parameters β, and can therefore be minimized using stochastic gradient descent or adaptive optimizers such as Adam. Gradients are propagated through each transformation T_k, and in turn through the neural networks s_θ and t_θ that parameterize them. During training, the entire sequence of flow steps learns to approximate the target distribution ρ via optimization of the composite map T.

Introduction: Normalizing Flows (NF)

In practice:

- The flow

$$\frac{dT(x,t)}{dt} = -\frac{\delta KL[\phi \circ T(.,t)]}{\delta \phi}\bigg|_{\phi=id} \tag{2}$$

  is time discretized: $y^{n+1} = y^n + \phi(y^n, \theta^n)$ where ϕ is a perturbation of the identity map
- The resulting map $T(\cdot, t^n) = \phi^n \circ \phi^{n-1} \circ ... \circ \phi^1$ is the composition of elementary maps $\phi^k = \phi(\cdot, \theta^k)$

Why it is useful?

- Normalization, positivity constraints, curse of dimensionality and over-resolution make the parametrization of ρ a hard task.
- We don't need to parametrize ρ, we rather parametrize elementary maps that, through composition, form a rich family of one-to-one functions we use to recover ρ.

Baptista, Trigila, Wang PolyMathJr 2025: ML project 7 / 13

Normalizing Flows Continued

In normalizing flows, we often encounter the expression:

$$\frac{dT(x,t)}{dt} = -\frac{\delta}{\delta\phi}\mathrm{KL}[\phi \circ T(\cdot,t)]\bigg|_{\phi=\mathrm{id}}.$$

At first glance, this may seem circular: T is built from compositions of ϕ, so how can we apply ϕ to T again?

The key idea is this: at each time t, we **freeze** the transformation $T(\cdot, t)$ and treat it as fixed. Then, we apply a small perturbation ϕ on the left, forming $\phi \circ T$. We examine how the KL divergence changes as we vary ϕ, and compute the functional derivative with respect to ϕ, evaluated at the identity:

$$\frac{\delta}{\delta\phi}\mathrm{KL}[\phi \circ T]\bigg|_{\phi=\mathrm{id}}.$$

This tells us the direction in which to perturb T (via composition with ϕ) to most reduce the KL divergence.

Why the Negative Gradient Direction?

The gradient of a function always points in the direction of steepest ascent. So if our goal is to minimize a function—like KL divergence—we must move in the *opposite* direction: the negative gradient.

To build intuition, think of the function as a surface in 3D space, where the height $z = f(x, y)$ represents the value of the function. At any point on this surface, the gradient $\nabla f(x, y)$ takes in your current x and y coordinates and gives you a direction in the xy-plane. That direction tells you how to walk in x and y to increase z as quickly as possible. In other words, the gradient is like a compass that points straight uphill.

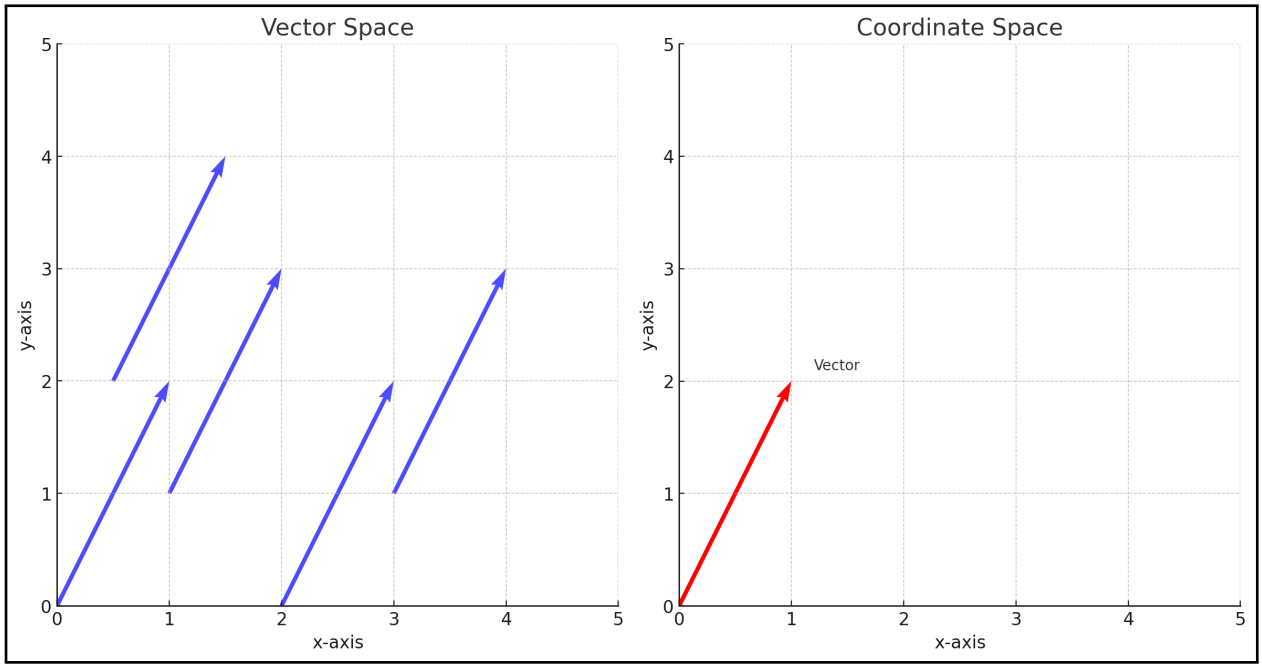
But if you want to *decrease* z, you walk in the opposite direction—downhill. That's why in gradient descent, we follow $-\nabla f$: it's the fastest way to reduce the function value.

In our case, we're minimizing not over scalars or vectors, but over functions. The KL divergence is a functional, and the derivative

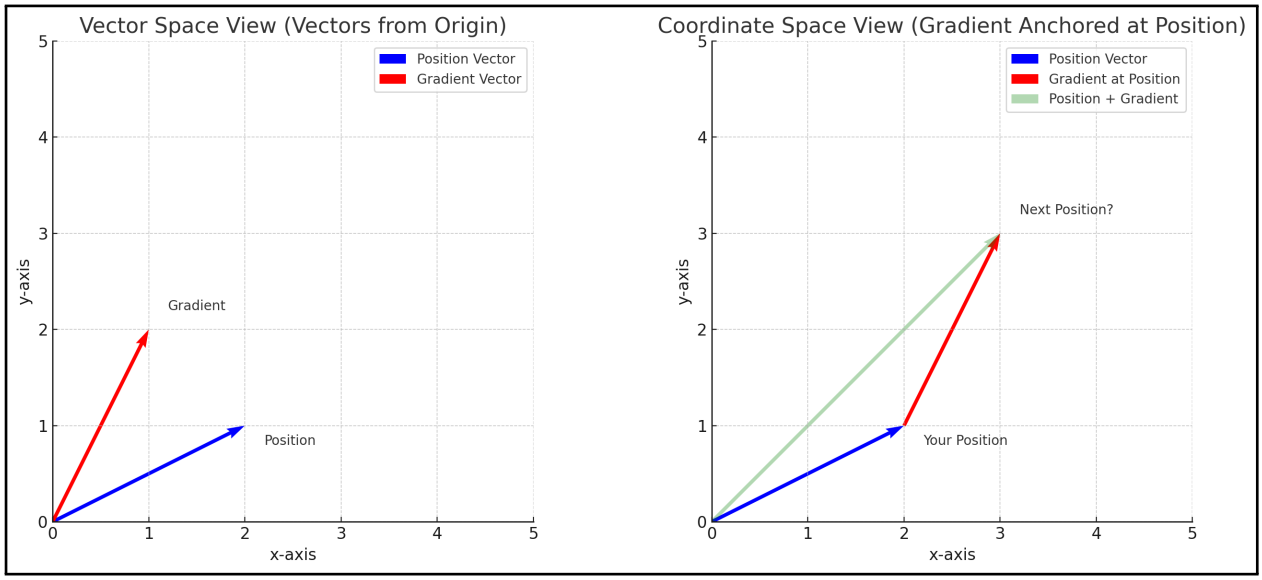
$$\frac{\delta}{\delta \phi} \mathrm{KL}[\phi \circ T]\Big|_{\phi=\mathrm{id}}$$

acts like a gradient in function space. It tells us: if we compose T with a small perturbation ϕ, how does KL change? This is our generalization of the compass. And just like before, we move in the *negative* direction to descend KL as efficiently as possible.

Note on Canonical Isomorphism and Vector Addition



Under the canonical isomorphism between a vector space and its coordinate representation, all vectors are drawn with their tails at the origin. This isomorphism allows us to represent vectors in a vector space as coordinate tuples in $\mathbb{R}^n$, anchored at the origin.



When we want to add vectors in coordinate space, we place them tip-to-tail. For example, to apply a gradient vector to a position vector, we interpret both as anchored at the origin, then translate the gradient vector to start at the tip of the position vector. The result of the addition is the vector from the origin to the tip of the second (translated) vector.

Optimal transport framework to map PDFs

Problem

How to move a pile of sand to fill a pit of the same volume minimizing a given cost?

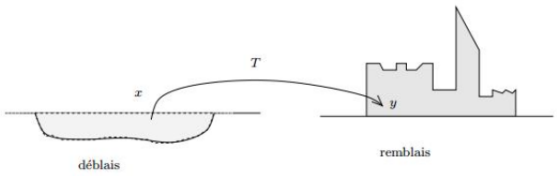


Fig. 3.1. Monge's problem of déblais and remblais

Math formulation

Given two probability densities, $\rho(\mathbf{z})$ and $\mu(\mathbf{z})$, $\mathbf{z} \in \mathbb{R}^d$, find a 1-to-1 map $T : \mathbb{R}^d \to \mathbb{R}^d$ such that $T_{\#\rho} = \mu$ and that minimizes the functional

$$M[T] = \int_{\mathbb{R}^d} c(\mathbf{z}, T(\mathbf{z}))\rho(\mathbf{z})d\mathbf{z}$$

The minimizer is called an *optimal transport map.*

Baptista, Trigila, Wang PolyMathJr 2025: ML project 8 / 13

The Optimal Transport Problem

The classical *Optimal Transport* (OT) problem asks:

> *Given two probability distributions, how can we transform one into the other in the most cost-efficient way?*

This is often described using the metaphor: "How do you move a pile of sand to fill a hole of the same volume, while minimizing the total effort required?"

Mathematical Setup

Let $\rho(z)$ and $\mu(z)$ be two probability density functions on $\mathbb{R}^d$. The goal is to find a measurable, invertible map $T : \mathbb{R}^d \to \mathbb{R}^d$ such that the *pushforward* of ρ by T equals μ:

$$T_{\#}\rho = \mu.$$

That is, when we apply T to samples drawn from ρ, the resulting distribution becomes μ.

Cost Functional

Each movement of a point $z \in \mathbb{R}^d$ to a new location $T(z)$ incurs a *cost*, denoted by $c(z, T(z))$, where $c : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}_{\geq 0}$ is a user-defined cost function (such as squared Euclidean distance).

The total transport cost across the entire distribution is defined by the functional:

$$M[T] = \int_{\mathbb{R}^d} c(z, T(z))\, \rho(z)\, dz.$$

This integral has the following interpretation:

- $\rho(z)$ is a probability density — it represents the instantaneous probability per unit volume at the point z.
- For each infinitesimal region around $z \in \mathbb{R}^d$, the product $\rho(z)\, dz$ gives the amount of "mass" located there (the probability mass in that tiny volume).
- The per unit cost to move that mass to its destination $T(z)$ is given by $c(z, T(z))$.
- Multiplying them gives the local transport cost: $c(z, T(z)) \cdot \rho(z)\, dz$.
- Integrating this quantity over all z accumulates the total cost of transporting the full distribution.

Such a map T^* is called an *optimal transport map.*

Summary Table: Definite vs. Indefinite Integrals

Type	Notation	Output	Meaning
Definite Integral	$\int_a^b f(x)\, dx$	Number	Net change a to b
Indefinite Integral	$\int f(x)\, dx$	Function $+ C$	Family of antiderivatives

Example: Interpreting Probability Densities in 2D

Let's say you have a 2D random variable $z = (z_1, z_2) \in \mathbb{R}^2$, and a probability density function $\rho(z)$.

- Then $\rho(z)$ has units of *probability per unit area.*
- If $\rho(z) = 0.25$ at some point, this means that in a small square of area 0.01 around that point, there is approximately $0.25 \times 0.01 = 0.0025$ units of probability mass.
- When we write:

  $$\int_{\mathbb{R}^2} \rho(z)\, dz = 1,$$

  we are saying that the *total probability mass* over the entire 2D space is 1.

How This Relates to Integration Types

In this example:

- The expression $\int_{\mathbb{R}^2} \rho(z)\, dz$ is a **definite integral**. It outputs a number (in this case, 1), representing the total probability mass over all of $\mathbb{R}^2$.
- If you were instead given the function $\rho(z)$, and wanted to find its antiderivative (which isn't typical for probability densities), that would be an **indefinite integral**. It would return a function $F(z) + C$, whose gradient (or divergence) gives you back $\rho(z)$.

Thus:

- **Definite integrals** are used to calculate *quantities*, like probability mass, area, or net change.
- **Indefinite integrals** are used to *recover functions* from their derivatives, up to an arbitrary constant.

Quadratic cost

Consider quadratic cost:

$$M[T] = \int_{\mathbb{R}^d} |\mathbf{z} - T(\mathbf{z})|^2 \rho(\mathbf{z}) d\mathbf{z}$$

If ρ and μ are smooth enough and have finite second moment, the optimal (Brenier) map exists, is unique and is given by the gradient of a convex function $\phi : \mathbb{R}^d \to \mathbb{R}$ satisfying the Monge-Ampere (MA) equation:

$$\rho(\mathbf{z}) = \mu(\nabla\phi(\mathbf{z})) \det(D^2\phi(\mathbf{z}))$$

One way of finding the optimal map from ρ to μ is to solve the MA equation enforcing convexity of the solution

Baptista, Trigila, Wang PolyMathJr 2025: ML project 9 / 13

We consider the problem of optimal transport between two probability densities ρ and μ defined over $\mathbb{R}^d$. A key ingredient in formulating the transport problem is the **cost functional**, which quantifies how "expensive" it is to move mass from one point to another.

Quadratic Cost

Definition (Quadratic Cost). The *quadratic cost* refers to the squared Euclidean distance between source and target locations in the transport map. That is, if $T : \mathbb{R}^d \to \mathbb{R}^d$ is a candidate transport map, the cost of transporting ρ to μ using T is defined as:

$$M[T] = \int_{\mathbb{R}^d} \|\mathbf{z} - T(\mathbf{z})\|^2 \, \rho(\mathbf{z}) \, d\mathbf{z}$$

Here:

- $\rho(\mathbf{z})$ is the density of the source measure,
- $T(\mathbf{z})$ is the image of $\mathbf{z}$ under the transport map,
- $\|\mathbf{z} - T(\mathbf{z})\|^2$ is the *pointwise cost*.

The goal of optimal transport is to find the map T that *pushes forward* ρ to μ (i.e., $T_{\#}\rho = \mu$) while minimizing $M[T]$. Remember, when we apply T to samples drawn from ρ, the resulting distribution becomes μ.

Brenier's Theorem and the Convex Potential

To understand the structure of optimal transport maps under quadratic cost, we begin by defining two key ideas: the *second moment* of a distribution, and the concept of a *convex potential function*.

Second Moment

Let $X \in \mathbb{R}^d$ be a random vector with probability density function ρ. The **second moment** of ρ is defined as:

$$\mathbb{E}[\|X\|^2] = \int_{\mathbb{R}^d} \|\mathbf{z}\|^2 \, \rho(\mathbf{z}) \, d\mathbf{z}$$

Here:

- $\mathbf{z} = (z_1, \ldots, z_d) \in \mathbb{R}^d$ is a point in space,
- $\|\mathbf{z}\|^2 = z_1^2 + z_2^2 + \cdots + z_d^2$ is the square of the Euclidean norm (i.e., squared distance from the origin),
- $\rho(\mathbf{z})$ is the value of the probability density at point $\mathbf{z}$.

This integral gives the expected squared distance of a sample drawn from ρ relative to the origin. A distribution is said to have a *finite second moment* if this integral is finite.

Convex Potential Function

A function $\phi : \mathbb{R}^d \to \mathbb{R}$ is called **convex** if it satisfies:

$$\phi(\lambda \mathbf{z}_1 + (1-\lambda)\mathbf{z}_2) \leq \lambda \phi(\mathbf{z}_1) + (1-\lambda)\phi(\mathbf{z}_2) \quad \text{for all } \lambda \in [0,1]$$

Intuitively, this means that *the graph of the function lies below the straight line connecting any two points on it.* There are no dips or valleys between any two points.

A **convex potential function** is just a convex function ϕ whose gradient $\nabla\phi$ defines a transport map.

Brenier's Theorem

Brenier's Theorem states the following:
If ρ and μ are two absolutely continuous probability densities on $\mathbb{R}^d$ with finite second moments, then there exists a unique optimal transport map T^* pushing ρ forward to μ, and it is given by:

$$T^*(\mathbf{z}) = \nabla\phi(\mathbf{z})$$

where:

- $\phi : \mathbb{R}^d \to \mathbb{R}$ is a convex potential function,
- $\nabla\phi(\mathbf{z})$ is the gradient vector, describing where to send mass originally located at $\mathbf{z}$.

So under quadratic cost, the optimal way to transport mass is to follow the gradient of a convex potential.

Note: Interpreting the Gradient in Optimal Transport

In standard calculus:

- $\nabla\phi(\mathbf{z})$ gives the direction and rate of steepest increase of ϕ at the point $\mathbf{z}$.

But in **optimal transport theory** — especially under **Brenier's Theorem** — we re-interpret the gradient $\nabla\phi$ as a function that maps each point $\mathbf{z}$ to a new point in space.

In this context, $\nabla\phi(\mathbf{z}) \in \mathbb{R}^d$ is not just a direction — it's interpreted as the **destination of mass located at $\mathbf{z}$**.

Absolutely Continuous Measures

A probability measure ρ on $\mathbb{R}^d$ is said to be **absolutely continuous with respect to Lebesgue measure** if it has a **density function** — that is, there exists a function $\rho(\mathbf{z})$ such that:

$$\mathbb{P}(A) = \int_A \rho(\mathbf{z})\, d\mathbf{z} \quad \text{for all measurable sets } A$$

In other words:

You can write the measure as an integral with respect to the standard volume in $\mathbb{R}^d$.
This means ρ "doesn't concentrate" on points, lines, or lower-dimensional shapes — it's spread out in space.

Note: Why Lower-Dimensional Sets Have Zero Lebesgue Measure

Because in $\mathbb{R}^d$, the Lebesgue measure is defined in terms of **volume** (i.e., d-dimensional volume), lower-dimensional subsets have zero measure.

- In $\mathbb{R}^1$: points have zero measure, intervals have positive measure.
- In $\mathbb{R}^2$: lines (1D objects) have zero area.
- In $\mathbb{R}^3$: surfaces (2D objects) have zero volume.
- More generally: any set of dimension $< d$ has zero d-dimensional Lebesgue measure.

Monge–Ampère Equation

To find the optimal convex function ϕ, one must solve a partial differential equation known as the **Monge–Ampère (MA) equation**:

$$\rho(\mathbf{z}) = \mu(\nabla\phi(\mathbf{z})) \cdot \det(D^2\phi(\mathbf{z}))$$

This equation expresses the condition that the gradient map $\nabla\phi$ preserves mass when transporting ρ to μ.
Term-by-term explanation:

- $\rho(\mathbf{z})$: Source density at $\mathbf{z}$,
- $\nabla\phi(\mathbf{z})$: Target point for mass at $\mathbf{z}$,
- $\mu(\nabla\phi(\mathbf{z}))$: Target density at the image point,
- $D^2\phi(\mathbf{z})$: Hessian matrix of second derivatives of ϕ,
- $\det(D^2\phi(\mathbf{z}))$: Jacobian determinant — volume distortion under $\nabla\phi$.

From Monge–Ampère to Optimal Transport Map

To get the **optimal transport map** T^* under **quadratic cost**, Brenier's Theorem tells us:

$$T^*(z) = \nabla\phi(z),$$

for some **convex potential function** ϕ.
But not just any convex function ϕ will do — it must be the one that **solves the Monge–Ampère equation**:

$$\rho(z) = \mu(\nabla\phi(z)) \cdot \det(D^2\phi(z)).$$

One idea on OT

We need to impose the convexity of the potential → we want the map to be the gradient of a convex function. If we build the flow

$$z^{k+1} = z^k + \beta^k \nabla_x F(z^k) = z^0 + \sum_{i=1}^{k} \nabla_x F(z^i)$$

with $z^0 = x$ the starting position, $\beta \in R$, and F convex, then the convexity of the potential depends on the sign of β.

- When β^i is positive there is no problem (sum of convex functions is still convex)
- If β is negative we need to pay attention
- We only have the value $\sum_{i=1}^{k} \nabla_x F(z^i)$ at the sample points z^i
- A condition that $\sum_{i=1}^{k} \nabla_x F(z^i)$ should satisfy is that, at least, should interpolate a convex function

Constructing Convex Transport Maps via Gradient Flow

In the context of optimal transport under quadratic cost, Brenier's Theorem guarantees that the optimal transport map T^* is the gradient of a convex potential function:

$$T^*(x) = \nabla \phi(x),$$

for some convex function $\phi : \mathbb{R}^d \to \mathbb{R}$. The goal of this construction is to build such a map iteratively using gradient flows, ensuring that the resulting map maintains the necessary convexity structure.

Discrete Flow Construction

We define a sequence of points $\{z^k\} \subset \mathbb{R}^d$ via the recurrence:

$$z^{k+1} = z^k + \beta^k \nabla F(z^k),$$

where:

- $z^0 = x \in \mathbb{R}^d$ is the initial point,
- $\beta^k \in \mathbb{R}$ is a step-size coefficient at iteration k,
- $F : \mathbb{R}^d \to \mathbb{R}$ is a known convex function.

Expanding this recursion yields:

$$z^{k+1}(x) = x + \sum_{i=1}^{k} \beta^i \nabla F(z^i),$$

which defines a transformation of the form:

$$T(x) := z^{k+1}(x).$$

Convexity Considerations

The transport map $T(x)$ is intended to approximate the gradient of a convex potential function. Our construction achieves this under the following conditions:

- **Positive weights:** Convex functions are closed under positive linear combinations. Since the gradient of a sum of convex functions (with positive weights) is equivalent to the sum of their gradients, the resulting transformation remains the gradient of a convex function.
- **Negative weights:** If any $\beta^i < 0$, convexity may be lost. In this case, the sum of gradients may no longer correspond to the gradient of a convex function. Careful control of step sizes is therefore necessary to preserve the desired structure.
- **Sampled gradients:** In practice, the gradients $\nabla F(z^i)$ are computed only at a finite collection of sample points z^i. The map $T(x)$ is therefore constructed by stitching together directional updates at these discrete locations. However, Brenier's theorem requires the overall transformation to behave like the gradient of a single globally defined convex function. As such, the sum

  $$\sum_{i=1}^{k} \beta^i \nabla F(z^i)$$

  must approximate—at least locally—the gradient of a convex potential. This imposes structural constraints on the sampling strategy, the choice of weights β^i, and the properties of the function F itself.

Interpretation and Goal

This iterative construction defines a flow of the point x under successive applications of directional updates determined by ∇F. The final position $z^{k+1}(x)$ defines a map that can be interpreted as a transport map:

$$T(x) = z^{k+1}(x),$$

provided it satisfies the convexity condition.

Research opportunity

Main goal of this project

Given samples $(\mathbf{z}^i)$ drawn from the unknown $\rho(\mathbf{z})$ we want to build a new generative model that map samples from $\rho_0(\mathbf{z})$ to $\rho(\mathbf{z})$.

Avenues for numerical exploration:

- Implement the data-driven OT flow algorithm with adaptive bandwidth and kernel
- Evaluate convergence when using an improved back-and-forth procedure

Avenues for theoretical exploration:

- Show the equivalence between the data-driven and maximum mean discrepancy flow
- Mathematical understanding of back-and-forth procedure

Delve into applications:

- Evaluation on 2D benchmark problems (banana, checkerboard)
- Image generation and solving Bayesian inference problems

Thank you!
Questions?

References

- Marzouk et. al, An introduction to sampling via measure transport, Handbook of UQ, 2016
- Papamakarios et. al, Normalizing Flows for Probabilistic Modeling and Inference, JMLR, 2021
- Santambrogio, Optimal Transport for Applied Mathematicians, Springer, 2015
- Tabak E.G., Trigila G. Data-driven optimal transport, Communications on Pure and Applied Math, 2016
- Galashov, de Bortoli, Gretton, Deep MMD Gradient Flow without adversarial training, 2024

4.2 Minimum β for Convexity

On the next page!

Minimum Beta Notes

James Evans

07/24/2025

Minimum β for Convexity of $h_\beta(x) = f(x) + \beta g(x)$

Juno Fang

July 24, 2025

1 Problem Statement

Let $f, g : A \subseteq \mathbb{R}^d \to \mathbb{R}$ be twice differentiable convex functions. We aim to determine the minimum $\beta \in \mathbb{R}$ such that the function

$$h_\beta(x) := f(x) + \beta g(x)$$

remains convex on the domain A.

2 Theoretical Background

A twice differentiable function $h : A \to \mathbb{R}$ is convex on an open convex set $A \subset \mathbb{R}^d$ if and only if its Hessian $\nabla^2 h(x)$ is positive semidefinite (PSD) for all $x \in A$.

Since both f and g are convex and smooth, $\nabla^2 f(x) \succeq 0$ and $\nabla^2 g(x) \succeq 0$ for all $x \in A$. We seek the smallest scalar $\beta \geq 0$ such that:

$$\nabla^2 h_\beta(x) = \nabla^2 f(x) + \beta \nabla^2 g(x) \succeq 0 \quad \forall x \in A.$$

Assumptions and Smoothness

Suppose $f : \mathbb{R}^n \to \mathbb{R}$ is a twice continuously differentiable function. This means:

- $f \in C^2$ (all partial derivatives up to second order exist and are continuous).
- The gradient is defined as:
$$\nabla f = \left\langle \frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_n} \right\rangle$$
- The Hessian matrix is:
$$\nabla^2 f = \left[\frac{\partial^2 f}{\partial x_i \partial x_j} \right]_{i,j}$$

which is symmetric because mixed partials are equal due to Clairaut's theorem.

Clairaut's Theorem (Equality of Mixed Partials)

For $f : \mathbb{R}^n \to \mathbb{R}$, if all second partial derivatives exist and are continuous near a point $\vec{x}$, then the mixed partials satisfy:
$$\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i} \quad \text{at that point.}$$

Critical Points

To find critical points, solve:
$$\nabla f(\vec{p}) = \vec{0}$$
This means all partial derivatives vanish at $\vec{p}$. These are the candidate points for local minima, maxima, or saddle points.

Let $S := \nabla^2 f(\vec{p})$ be the Hessian evaluated at a critical point $\vec{p}$.

We analyze the matrix S by looking at its eigenvalues.

Positive Semidefinite Condition

$$S \succeq 0 \iff x^\top S x \geq 0 \quad \forall x \in \mathbb{R}^n \iff \lambda_i \geq 0 \quad \text{for all eigenvalues } \lambda_i \text{ of } S$$

Spectral Theorem

Since S is symmetric, it can be diagonalized:
$$S = QDQ^\top$$
where:

- $Q \in \mathbb{R}^{n \times n}$ is an orthogonal matrix ($Q^\top = Q^{-1}$),
- $D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ is a diagonal matrix of eigenvalues.

Signature and Classification

Let the signature of S refer to the sign pattern of the eigenvalues:

- **All** $\lambda_i > 0$: S is positive definite $\rightarrow$ strict local **minimum**
- **All** $\lambda_i < 0$: S is negative definite $\rightarrow$ strict local **maximum**
- **Mixed signs**: S is indefinite $\rightarrow$ **saddle point**
- **All** $\lambda_i \geq 0$: S is positive semidefinite $\rightarrow$ test is **inconclusive**
- **All** $\lambda_i \leq 0$: S is negative semidefinite $\rightarrow$ test is **inconclusive**

Convexity Remark

Note: The second derivative test is **local**, and if the Hessian is only semidefinite at a critical point, the test is inconclusive.

However, if f is globally convex — that is, $\nabla^2 f(x) \succeq 0$ for all $x \in \mathbb{R}^n$ — then **any** critical point is guaranteed to be a global minimum.

Similarly, if f is concave (i.e., $\nabla^2 f(x) \preceq 0$ everywhere), then any critical point is a global maximum.

3 Reformulated Optimization Problem

We solve:

$$\min \beta \quad \text{such that} \quad \nabla^2 f(x) + \beta \nabla^2 g(x) \succeq 0 \quad \forall x \in A.$$

This is a semi-infinite optimization problem with infinitely many PSD constraints.

4 Numerical Strategy

Discretize A with points $x_1, \ldots, x_n \in A$. For each x_i, compute the smallest β such that:

$$\nabla^2 f(x_i) + \beta \nabla^2 g(x_i) \succeq 0.$$

Let $\lambda_{\min}^{(i)}(\beta)$ be the smallest eigenvalue of $\nabla^2 f(x_i) + \beta \nabla^2 g(x_i)$. The condition becomes:

$$\lambda_{\min}^{(i)}(\beta) \geq 0.$$

If $\nabla^2 g(x_i) \succ 0$, $\lambda_{\min}^{(i)}(\beta)$ is increasing in β. Solve for minimal β_i where $\lambda_{\min}^{(i)}(\beta_i) = 0$. The overall minimum β is:

$$\beta^* = \max_{1 \leq i \leq n} \beta_i.$$

5 Final Answer

The minimal β is:

$$\beta^* = \sup_{x \in A} \inf \{\beta \in \mathbb{R} : \nabla^2 f(x) + \beta \nabla^2 g(x) \succeq 0\}$$

Approximate via dense sampling or convex programming with matrix inequalities.

4.3 Convexity Interpolation

On the next page!

Convexity Interpolation Notes

James Evans

07/14/2025

Recalling the Construciton of a Convex Interpolant

If the inequality

$$y_j \geq y_i + g_i^\top (x_j - x_i) \quad \text{for all } i, j$$

is satisfied, then one can explicitly construct a convex function $f(x)$ that interpolates the data.

Each pair (x_i, y_i) along with its subgradient g_i defines an affine function:

$$f_i(x) = y_i + g_i^\top (x - x_i).$$

If the inequality condition holds, we can take the maximum over these affine functions to define a convex interpolant:

$$f(x) = \max_{i=1,\ldots,n} \left\{ y_i + g_i^\top (x - x_i) \right\}.$$

This $f(x)$ is convex because the pointwise maximum of affine functions is convex. Moreover, it satisfies $f(x_i) = y_i$, hence it interpolates the given data.

Convexity via First-Order Inequality: Two Applications

The first-order condition for convexity is given by:

$$f(x_j) \geq f(x_i) + \langle g_i, x_j - x_i \rangle$$

This inequality ensures that the affine approximation $f(x_i) + \langle g_i, x - x_i \rangle$ lies below the function $f(x)$, i.e., that f is convex. It plays a central role in two distinct applications.

Case 1: Constructing a Convex Function from Data

Suppose we are given points (x_i, y_i). To construct a convex function f that interpolates these points, the standard method is:

$$f(x) = \max_{i=1,\ldots,n} \left\{ y_i + g_i^\top (x - x_i) \right\}$$

Each term $y_i + g_i^\top (x - x_i)$ is an affine function. Since the pointwise maximum of affine functions is convex, $f(x)$ will be convex if the subgradients g_i satisfy the subgradient condition:

$$y_j \geq y_i + g_i^\top (x_j - x_i) \quad \text{for all } i, j$$

Case 2: Verifying Convexity of a Function Sum

Now consider two fixed convex functions f and f'. The goal is to find the smallest value of β such that the function

$$f(x) + \beta f'(x)$$

is still convex. For this, we assume the subgradients g_i come from f, and define:

$$y_i = f(x_i), \quad y_i' = f'(x_i)$$

The convexity condition becomes:

$$f(x_j) + \beta f'(x_j) \geq f(x_i) + \beta f'(x_i) + g_i^\top (x_j - x_i)$$

This is rearranged to isolate β, and a linear program is solved to find the largest such value for each i. We define:

$$\beta_i = \max \left\{ \beta : y_j - y_i \geq g_i^\top (x_j - x_i) + \beta(y_j' - y_i') \right\}$$

Then the overall threshold is given by:

$$\beta^* = \min_i \beta_i.$$

Though the purposes differ, both applications are grounded in the same first-order condition for convexity.

Convexity Interpolation

Tony Deng

The question we try to solve is: given two convex functions $f, f' : \mathbb{R}^d \to \mathbb{R}$, we want to find the minimum β such that $f + \beta f'$ is still convex.
A discrete version of this problem based on point interpolation can be formulated as follows: Given a list of triples (x_i, y_i, y_i') where $x_i \in \mathbb{R}^d, y_i, y_i' \in \mathbb{R}$, we want to find the minimum β such that $y_i + \beta y_i'$ interpolate a convex function. This problem can be further reformulated as the following linear program

$$\begin{cases} \min \beta \\ y_j - y_i + \beta(y_j' - y_i') \geq (x_j - x_i)^T g_i \end{cases}$$

Why We Check Convexity of $f + \beta f'$

Slide 10 states that we need to impose the convexity of the potential. This is because we want the resulting flow map to be the gradient of a convex function — a key structural requirement in many settings, including optimal transport.

To enforce this, we study whether the function $f + \beta f'$ remains convex for a given scalar β. This expression represents a convex base potential f perturbed in the direction of another convex function f', scaled by β. We are trying to find the smallest such $\beta \geq 0$ that preserves convexity.

In the flow defined by:

$$z^{k+1} = z^k + \beta^k \nabla_x F(z^k) = z^0 + \sum_{i=1}^{k} \nabla_x F(z^i),$$

the total displacement is a sum of gradients. If each $\beta^k > 0$, the potential remains convex since sums of convex functions are convex. However, if any $\beta^k < 0$, we must verify that the resulting potential is still

convex.

Therefore, checking the convexity of $f+\beta f'$ ensures that the accumulated flow remains valid — i.e., generated by a convex potential — even when perturbations are introduced.

Convexity via First-Order Interpolation Inequality

The key convexity condition (in subgradient form) as described in the interpolation problem is:

$$\boxed{f(x_j) \geq f(x_i) + \langle g_i,\ x_j - x_i \rangle}$$

where g_i is a subgradient of f at the point x_i. The notation $\langle \cdot,\ \cdot \rangle$ denotes the dot product (inner product) between vectors in $\mathbb{R}^d$.

In our setting, we are considering values of the function $f + \beta f'$ evaluated at points x_i, so we want the following inequality to hold:

$$y_j + \beta y'_j \geq y_i + \beta y'_i + \langle g_i,\ x_j - x_i \rangle$$

Rewriting this by moving terms to one side:

$$y_j - y_i + \beta(y'_j - y'_i) \geq \langle x_j - x_i,\ g_i \rangle$$

This is the constraint used in the linear program.

Final Linear Program

We minimize β, subject to the constraint:

$$y_j - y_i + \beta(y'_j - y'_i) \geq (x_j - x_i)^\top g_i \quad \text{for all } i, j$$

This corresponds to finding the smallest β such that adding $\beta f'$ to f still results in a convex function over the observed sample points.

One idea on OT

We need to impose the convexity of the potential → we want the map to be the gradient of a convex function. If we build the flow

$$z^{k+1} = z^k + \beta^k \nabla_x F(z^k) = z^0 + \sum_{i=1}^{k} \nabla_x F(z^i)$$

Missing a beta_i here

with $z^0 = x$ the starting position, $\beta \in R$, and F convex, then the convexity of the potential depends on the sign of β.

- When β^i is positive there is no problem (sum of convex functions is still convex)
- If β is negative we need to pay attention
- We only have the value $\sum_{i=1}^{k} \nabla_x F(z^i)$ at the sample points z^i
- A condition that $\sum_{i=1}^{k} \nabla_x F(z^i)$ should satisfy is that, at least, should interpolate a convex function

Note: If all $\beta^k = 0$, then the flow is frozen — that is, the update rule

$$z^{k+1} = z^k + \beta^k \nabla_x F(z^k)$$

reduces to $z^{k+1} = z^k$ at every step. This means the iterates do not change and the map constructed is simply the identity. The potential function F is still present, but its gradient does not influence the flow.

Since by assumption, this minimum occurs when $\beta < 0$, we can also formulate the problem by replacing β by $-\beta$,

$$\begin{cases} \max \beta \\ y_j - y_i \geq (x_j - x_i)^T g_i + \beta(y'_j - y'_i) \end{cases}$$

Reformulating the Problem When $\beta < 0$

Since, by assumption, the optimal value of β occurs when $\beta < 0$, we can simplify the optimization by substituting $\beta \leftarrow -\beta$. This turns the minimization of a negative quantity into a maximization of a positive one.

Starting from the original constraint:

$$y_j - y_i + \beta(y'_j - y'_i) \geq (x_j - x_i)^T g_i$$

We substitute $\beta \leftarrow -\beta$, which gives:

$$y_j - y_i - \beta(y'_j - y'_i) \geq (x_j - x_i)^T g_i$$

Rewriting this:

$$y_j - y_i \geq (x_j - x_i)^T g_i + \beta(y'_j - y'_i)$$

This leads to the following reformulated linear program:

$$\begin{cases} \max \beta \\ \text{subject to} \quad y_j - y_i \geq (x_j - x_i)^T g_i + \beta(y'_j - y'_i) \quad \text{for all } i, j \end{cases}$$

This is equivalent to the original problem, but posed as a maximization of a positive β, which can simplify interpretation and computation.

By using matrix notation, we construct matrices

$$A_i = \begin{bmatrix} (x_1 - x_i)^T & y'_1 - y'_i \\ \vdots & \\ (x_n - x_i)^T & y'_n - y'_i \end{bmatrix}, b_i = \begin{bmatrix} y_1 - y_i \\ \vdots \\ y_n - y_i \end{bmatrix}$$

Matrix Formulation of the Constraints

To simplify the convexity constraints across all points j, we express them using matrix notation.

Recall the original pointwise convexity constraint:

$$y_j - y_i + \beta(y'_j - y'_i) \geq (x_j - x_i)^T g_i \quad \text{for all } j$$

We rewrite this in matrix form as:

$$A_i \begin{bmatrix} g_i \\ \beta \end{bmatrix} \leq b_i$$

where the matrices are defined as follows:

$$A_i = \begin{bmatrix} (x_1 - x_i)^T & y'_1 - y'_i \\ \vdots & \vdots \\ (x_n - x_i)^T & y'_n - y'_i \end{bmatrix}, \qquad b_i = \begin{bmatrix} y_1 - y_i \\ \vdots \\ y_n - y_i \end{bmatrix}$$

Each row of A_i corresponds to a constraint for a specific index j. The first component $(x_j - x_i)^T$ multiplies the subgradient g_i, and the second component $y'_j - y'_i$ multiplies β.

The vector b_i captures the right-hand sides $y_j - y_i$ of all constraints for fixed i. Together, this matrix formulation compactly encodes all the linear inequalities needed to ensure that the values $y_i + \beta y'_i$ interpolate a convex function.

Let $c = (0, 0, ..., 0, 1)$ be the $d+1$ dimensional vector with 1 in its last coordinate and 0 elsewhere. Then the linear problem becomes

$$\begin{cases} \max c^T x \\ A_i x \geq b_i \end{cases}$$

Since the threshold for β must satisfies the criterion that the linear program must be feasible for all i, we define

$$\beta_i = \begin{cases} \max c^T x \\ A_i x \geq b_i \end{cases}$$

And the threshold is

$$-\beta = -\min_i \beta_i$$

Dimension Check for the Constraint $A_i \begin{bmatrix} g_i \\ \beta \end{bmatrix} \leq b_i$

We verify that the matrix multiplication in the constraint

$$A_i \begin{bmatrix} g_i \\ \beta \end{bmatrix} \leq b_i$$

is dimensionally consistent.

- The vector $g_i \in \mathbb{R}^d$, and $\beta \in \mathbb{R}$. Therefore, the stacked vector is

$$\begin{bmatrix} g_i \\ \beta \end{bmatrix} \in \mathbb{R}^{d+1}.$$

- The matrix A_i is defined as

$$A_i = \begin{bmatrix} (x_1 - x_i)^\top & y_1' - y_i' \\ \vdots & \vdots \\ (x_n - x_i)^\top & y_n' - y_i' \end{bmatrix} \in \mathbb{R}^{n \times (d+1)}.$$

  Each row consists of a d-dimensional row vector plus one scalar, making $d+1$ columns total, and there are n rows (one for each j).

- The right-hand side vector b_i is

$$b_i = \begin{bmatrix} y_1 - y_i \\ \vdots \\ y_n - y_i \end{bmatrix} \in \mathbb{R}^n.$$

Therefore, the matrix-vector product is

$$A_i \begin{bmatrix} g_i \\ \beta \end{bmatrix} \in \mathbb{R}^n,$$

which is compatible with the inequality $A_i[g_i; \beta] \leq b_i$, since both sides are in $\mathbb{R}^n$.

Step-by-Step Explanation of the Threshold Computation for β

Understanding the Vector c and Its Role in the Linear Program

We are solving a linear program where the decision variable is a vector

$$x = \begin{bmatrix} g_i \\ \beta \end{bmatrix} \in \mathbb{R}^{d+1}.$$

- The first d entries of x represent the subgradient $g_i \in \mathbb{R}^d$.
- The last entry of x is the scalar $\beta \in \mathbb{R}$.

So the full decision variable x bundles both g_i and β into a single $(d+1)$-dimensional vector.

Extracting and Optimizing β

We want to optimize the scalar β, which is the last coordinate of the vector x. To do this, we define a vector

$$c = (0, 0, \ldots, 0, 1) \in \mathbb{R}^{d+1},$$

which is a row vector with:

- Zeros in the first d coordinates,
- A 1 in the last coordinate (which corresponds to β).

Why This Works

When we compute the dot product $c^\top x$, we get:

$$c^\top x = \begin{bmatrix} 0 & 0 & \ldots & 0 & 1 \end{bmatrix} \begin{bmatrix} g_i \\ \beta \end{bmatrix} = \beta.$$

Therefore, maximizing $c^\top x$ is equivalent to maximizing β.

The Linear Program for each i

Each subproblem (for fixed i) becomes:

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{subject to} \quad & A_i x \leq b_i \end{aligned}$$

This finds the maximum allowable value of β that satisfies the convexity condition for fixed point x_i.

Call this optimal value β_i.

Combine all constraints

The convexity condition must hold for all i, so we take:

$$\beta^* = \min_i \beta_i$$

4.4 Function Convexity Interpolation

On the next page!

Function Convexity Interpolation Notes

James Evans

07/23/2025

Function Convexity Interpolation

Tony Deng

1 Mathematical Background for convexity

We are trying to solve the following question: Given two smooth convex functions $f, g : A \to \mathbb{R}$ where $A \subset \mathbb{R}^d$ and their gradient, we want to find the minimum β such that $f + \beta g$ is still convex. Equivalently, we want to find the minimum β such that for all $x, z \in A$,

$$(f + \beta g)(z) - (f + \beta g)(x) \geq \nabla(f + \beta g)\big|_x \cdot (z - x)$$

In other words, β must satisfy

$$f(z) - f(x) - \nabla f\big|_x \cdot (z - x) \geq -\beta \left(g(z) - g(x) - \nabla g\big|_x \cdot (z - x)\right)$$

Since f, g are convex, we know that this implies that

$$\beta \geq -\frac{f(z) - f(x) - \nabla f\big|_x \cdot (z - x)}{\left(g(z) - g(x) - \nabla g\big|_x \cdot (z - x)\right)}$$

If the denominator is zero, we define the fraction to be $-\infty$. Then the threshold β^* can be computed as

$$\sup_{z \in A, x \in A} -\frac{f(z) - f(x) - \nabla f\big|_x \cdot (z - x)}{\left(g(z) - g(x) - \nabla g\big|_x \cdot (z - x)\right)}$$

If the inequality

$$y_j \geq y_i + g_i^\top (x_j - x_i) \quad \text{for all } i, j$$

is satisfied, then one can explicitly construct a convex function $f(x)$ that interpolates the data.

Each pair (x_i, y_i) along with its subgradient g_i defines an affine function:

$$f_i(x) = y_i + g_i^\top (x - x_i).$$

If the inequality condition holds, we can take the maximum over these affine functions to define a convex interpolant:

$$f(x) = \max_{i=1,\ldots,n} \left\{ y_i + g_i^\top (x - x_i) \right\}.$$

This $f(x)$ is convex because the pointwise maximum of affine functions is convex. Moreover, it satisfies $f(x_i) = y_i$, hence it interpolates the given data.

2 Local Convexity

Let $f : A \to \mathbb{R}$, $A \subset \mathbb{R}^d$. We say that f is locally convex at x if there is a neighborhood U of x such that for all $z \in U$,

$$f(z) - f(x) \geq \nabla f|_x \cdot (z - x)$$

Let $f, g : A \to \mathbb{R}$ be convex. We want to find the minimum β such that $f + \beta g$ is convex at A. For each x, we can find β_x such that $f + \beta g$ is locally convex at x for all $\beta \geq \beta_x$. Then, we can set the threshold $\beta^* = \sup_x \beta_x$. β_x can be computed as follows:

$$\beta_x = \sup_{z \in U} -\frac{f(z) - f(x) - \nabla f\big|_x \cdot (z - x)}{\big(g(z) - g(x) - \nabla g\big|_x \cdot (z - x)\big)}$$

where U is a neighborhood around x.

- For one pair, $\beta \geq 1$
- For another, maybe $\beta \geq -2$
- For another, $\beta \geq 5$

If you want a single β that works **for all pairs**, you need to pick the **largest** of these lower bounds.

That's why:

$$\beta^* := \sup_{x,z \in A} \left(\frac{-\text{numerator}}{\text{denominator}} \right)$$

This gives the **tightest possible threshold** — the smallest β that satisfies *all* those inequalities.

3 Numerical Simulation

This algorithm has the following parameters: r, N, M and the set A. First, we sample $x_1, ..., x_N$ uniformly random from A. This can be easily achieved if A is a rectangular box. Then, we sample M points from the $d-1$ sphere of radius r. This can be done by sampling $X \sim N(0,1)^d$. Then, $X/\|X\|$ will be uniformly distributed in the sphere (This idea is suggested by Sahith). For each $i \in [N], j \in [M]$, we define $z_{i,j} = x_i + X_j/\|X_j\|$. Then, we can compute β as

$$\beta = \max_{i,j} -\frac{f(z_{i,j}) - f(x_i) - \nabla f\big|_{x_i} \cdot (z_{i,j} - x_i)}{\Big(g(z_{i,j}) - g(x_i) - \nabla g\big|_{x_i} \cdot (z_{i,j} - x_i)\Big)}$$

Parameters:

- r: radius of a sphere used to perturb each point.
- N: number of base points $x_1, \ldots, x_N$ sampled from the domain A.
- M: number of perturbation directions per point.
- A: domain (e.g., a box in $\mathbb{R}^d$).

Sampling Procedure:

Base Points:

- Sample $x_1, \ldots, x_N \in A$ uniformly at random.
- Each x_i is a point in the domain.

Perturbation Directions:

- Sample M random vectors $X_j \sim \mathcal{N}(0, I_d)$, i.e., from a standard normal in $\mathbb{R}^d$.
- Normalize them to unit length: $\frac{X_j}{\|X_j\|}$ lies on the *unit sphere* S^{d-1}.
- Scale to get points on the **radius** r sphere: $x_i + r \cdot \frac{X_j}{\|X_j\|}$.
- Denote this perturbed point as:

$$z_{i,j} := x_i + \frac{X_j}{\|X_j\|} \cdot r.$$

Objective:

For each pair (i, j), compute the expression:

$$\frac{f(z_{i,j}) - f(x_i) - \nabla f|_{x_i} \cdot (z_{i,j} - x_i)}{g(z_{i,j}) - g(x_i) - \nabla g|_{x_i} \cdot (z_{i,j} - x_i)}$$

Clarifying the Dimensions and Geometry

Sampling from $\mathbb{R}^d$:

Each random vector $X_j \sim \mathcal{N}(0, I_d)$ is a point in:

$$X_j \in \mathbb{R}^d$$

So the set of all such vectors spans $\mathbb{R}^d$, a d-dimensional Euclidean space.

Unit Sphere $S^{d-1} \subset \mathbb{R}^d$:

The **unit sphere** in $\mathbb{R}^d$ is defined as:

$$S^{d-1} = \left\{ v \in \mathbb{R}^d \,\middle|\, \|v\| = 1 \right\}$$

- Although it's embedded in $\mathbb{R}^d$, it is a $(d-1)$-dimensional **manifold**.
- So when we normalize X_j to:

$$s_j := \frac{X_j}{\|X_j\|} \in S^{d-1}$$

we are taking a vector that *lives* in $\mathbb{R}^d$, but now lies *on* the surface of the sphere — which has only $d-1$ degrees of freedom (since one degree is used up by the constraint $\|s_j\| = 1$).

Scaled Sphere of Radius r:

Now scale this unit vector to lie on the sphere of radius r:

$$r \cdot s_j = r \cdot \frac{X_j}{\|X_j\|} \in \mathbb{R}^d$$

This is still a vector in $\mathbb{R}^d$, but it lies on the surface of a sphere of radius r.

Perturbation Point:

To perturb a base point $x_i \in \mathbb{R}^d$, we define:

$$z_{i,j} := x_i + r \cdot \frac{X_j}{\|X_j\|} \in \mathbb{R}^d$$

So:

- $x_i \in \mathbb{R}^d$
- $z_{i,j} \in \mathbb{R}^d$
- $z_{i,j}$ lies on the sphere of radius r centered at x_i

4 Potential Improvement

I think if we let the variable x distribute regularly evenly (equidistribution) as opposed to uniformly random, we might get a more accurate result or need less datapoint. We can achieve this if we let x distributed on the lattice on d-dimensional space. However, I do not know how to make X distributed regularly even on the $n-1$-dimensional sphere. There is an article for equidistribution on S^1 (Sphere on $\mathbb{R}^2$). Also, I have implemented code on python, but I think the code runs very slowly. We might need to see how to improve the speed of the program. By testing, increasing the dimension (until 10-dim) will not increase the running time of the program for too much.

4.5 Interpolation Problem

On the next page!

Interpolation Problem

Polymath Jr

07/11/2025

One idea on OT

We need to impose the convexity of the potential → we want the map to be the gradient of a convex function. If we build the flow

$$z^{k+1} = z^k + \beta^k \nabla_x F(z^k) = z^0 + \sum_{i=1}^{k} \nabla_x F(z^i)$$

with $z^0 = x$ the starting position, $\beta \in R$, and F convex, then the convexity of the potential depends on the sign of β.

- When β^i is positive there is no problem (sum of convex functions is still convex)
- If β is negative we need to pay attention
- We only have the value $\sum_{i=1}^{k} \nabla_x F(z^i)$ at the sample points z^i
- A condition that $\sum_{i=1}^{k} \nabla_x F(z^i)$ should satisfy is that, at least, should interpolate a convex function

Baptista, Trigila, Wang PolyMathJr 2025: ML project 10 / 13

In the slide, when β^i is positive, the sum $\sum \nabla_x F(z^i)$ remains convex — no issue arises. But if any $\beta^i < 0$, convexity is no longer guaranteed, so we need to check whether the resulting sum still corresponds to a convex potential. That's exactly where Xuan's proposed solution to the interpolation problem comes in: it provides a way to verify whether a given sum of gradients (even with negative coefficients) still defines a convex function.

Xuan's Proposed Solution to the Interpolation Problem:

Questions: Given a bunch of points $(x_i.y_i)_{i=1,...,n}$, with $x_i \in \mathbb{R}^d, y_i \in \mathbb{R}$, can this interpolate a convex function?

Idea: Points $(x_i.y_i)$ can interpolate a convex function means that there exist a convex function $f : \mathbb{R}^d \to \mathbb{R}$ such that $f(x_i) = y_i$. We have such convex function if and only if for all $i = 1, ..., n$, there exist subgradient vector $g_i \in \mathbb{R}^d$ such that for all $j = 1, ..., n$

$$y_j \geqslant y_i + g_i^T(x_j - x_i)$$

Defining the Subgradient

If f is **differentiable** at x_0, then the gradient $\nabla f(x_0)$ is the **unique** subgradient.

But if f is **not differentiable**, then there may be **many** valid subgradients — forming a **subdifferential set**, denoted:

$$\partial f(x_0) = \left\{ g \in \mathbb{R}^d \,\middle|\, f(x) \geq f(x_0) + g^\top (x - x_0) \text{ for all } x \right\}$$

Example

Let $f(x, y) = x^2 + 2y$, which is a convex function.

Let $x_i = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, so

$$f(x_i) = 1^2 + 2 \cdot 2 = 5$$

Since f is differentiable, the subgradient at x_i is the gradient:

$$\nabla f(x, y) = \begin{bmatrix} 2x \\ 2 \end{bmatrix} \Rightarrow \nabla f(1, 2) = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

So $g_i = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$.

Now let $x_j = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$, so

$$f(x_j) = 2^2 + 2 \cdot 4 = 4 + 8 = 12$$

Compute the subgradient inequality:

$$x_j - x_i = \begin{bmatrix} 2 \\ 4 \end{bmatrix} - \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$g_i^\top (x_j - x_i) = \begin{bmatrix} 2 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} = 2 \cdot 1 + 2 \cdot 2 = 6$$

$$f(x_i) + g_i^\top (x_j - x_i) = 5 + 6 = 11$$

Compare:

$$f(x_j) = 12 \geq 11 = f(x_i) + g_i^\top (x_j - x_i)$$

Therefore, the inequality holds and g_i is a valid subgradient.

This means for every point (x_i, y_i), there is a supporting hyperplane that passes through (x_i, y_i) and keeps all other points on or above it. (Prove: the necesary condition already follows from definition of that convex function. For sufficient condition: we can build a convex function $f(x) = \max_{i=1,..,n}\{y_i + g_i^T(x - x_i)\}$, and $f(x_i) = y_i$ since we have for all $k \neq i$, $y_k + g_k^T(x_i - x_k) \leqslant y_i$.)

Understanding the Convex Interpolation Construction

Given subgradients $g_i \in \mathbb{R}^d$ and values $y_i \in \mathbb{R}$ at points $x_i \in \mathbb{R}^d$, the goal is to construct a convex function that interpolates these values.

A candidate convex function is given by the expression:

$$f(x) = \max_{i=1,\ldots,n} \left\{ y_i + g_i^\top (x - x_i) \right\}.$$

This formula defines a function as the pointwise maximum of a finite collection of affine functions. Now, we will explain why this construction is valid.

Interpretation of Each Term

Each inner term $y_i + g_i^\top (x - x_i)$ defines a hyperplane that passes through the point (x_i, y_i) with slope g_i. For every data point (x_i, y_i), the corresponding affine function represents a local linear lower approximation of the convex function.

Role of the Maximum

Taking the maximum across these affine functions ensures that the resulting function lies above all the supporting hyperplanes. Geometrically, this creates a surface resembling a "tent roof" formed by the upper envelope of the individual planes. The surface bends upward wherever one affine piece overtakes another, preserving convexity.

Why the Resulting Function is Convex

A standard result in convex analysis states that the pointwise maximum of convex functions is itself convex. Since each affine function $y_i + g_i^\top (x - x_i)$ is convex, the function $f(x)$ constructed in this pointwise way is convex by use of this standard result.

Conclusion

This construction demonstrates the sufficiency of the subgradient condition: if the inequality

$$y_j \geq y_i + g_i^\top (x_j - x_i) \quad \text{for all } i, j$$

is satisfied, then one can explicitly construct a convex function $f(x)$ that interpolates all the points (x_i, y_i) by taking the pointwise maximum of the associated affine functions.

Now using this idea, we can construct a linear program: For $i, j = 1, ..., n$,

$$y_j \geqslant y_i + g_i^T (x_j - x_i)$$

where $x_i, x_j \in \mathbb{R}^d, y_i, y_j \in \mathbb{R}$ are known vectors and values. And only g_i's are unknown vectors. If this linear program is feasible, then we know there exist such subgradient vectors, so points can interpolate a convex function. If the linear program is not feasible, then points can not interpolate convex functions.

4.6 interpolation_check.py

On the next page!

interpolation_check.py

James Evans

07/19/2025

def check_prime()

```python
# Input: points are array of points, values are array of values where each point evaluates, j is index of points
# Output: true if point j has the tangent plane lays below all the points.
def check_prime(points, values, j):
    N, d = points.shape
    g = cp.Variable(d)   # the subgradient varaible at point j

    constraints = []
    for i in range(N):
        if i == j:
            continue
        lhs = values[i] - values[j]
        rhs = (points[i] - points[j]) @ g
        constraints.append(lhs >= rhs)

    # Use dummy objective since we only care about feasibility
    prob = cp.Problem(cp.Minimize(0), constraints)
    prob.solve()

    return prob.status in [cp.OPTIMAL, cp.OPTIMAL_INACCURATE]
```

We examine the following two lines of code from the program:

```
N, d = points.shape
g = cp.Variable(d)   # the subgradient variable at point j
```

Lines 5-8

Extracting Dimensions

$$N, d = \texttt{points.shape}$$

Here, `points` is a NumPy array of shape (N, d), representing a collection of N points in $\mathbb{R}^d$. That is:

- N is the number of data points.
- d is the dimension of each point.

For example, if we have:

$$\texttt{points} = \begin{bmatrix} 1.0 & 2.0 \\ 3.0 & 4.0 \\ 5.0 & 6.0 \end{bmatrix},$$

then $N = 3$ and $d = 2$, meaning there are 3 points in $\mathbb{R}^2$.

Defining the Subgradient Variable

```
g = cp.Variable(d)
```

This line defines a decision variable $g \in \mathbb{R}^d$ using the `cvxpy` optimization library. The variable g represents a *subgradient* of a convex function at the point $x_j \in \mathbb{R}^d$. We are not taking the gradient of the function itself. Instead, we are checking whether a function *could* be convex by testing if there exists a subgradient at each point consistent with convexity.

Mathematical Context

In convex analysis, a function $f : \mathbb{R}^d \to \mathbb{R}$ is convex if for any point x_j, there exists a subgradient $g_j \in \mathbb{R}^d$ such that the following subgradient inequality holds for all $i \neq j$:

$$f(x_i) \geq f(x_j) + \langle g_j, x_i - x_j \rangle.$$

The program seeks to verify whether such a subgradient g_j exists for each point x_j. The variable `g` is introduced to model this subgradient and is used to formulate the linear constraints required for the feasibility check.

Lines 18-22

We examine the following lines of code from the function `check_prime`:

```
# Use dummy objective since we only care about feasibility
prob = cp.Problem(cp.Minimize(0), constraints)
prob.solve()

return prob.status in [cp.OPTIMAL, cp.OPTIMAL_INACCURATE]
```

This block constructs and solves a convex optimization problem to verify whether the subgradient inequalities at a given point x_j are feasible. The check is based entirely on the existence of a solution, not the optimization of any meaningful objective.

First, the line `cp.Minimize(0)` defines a constant objective function equal to zero. Since we are only interested in whether the constraints can be satisfied, we use a dummy objective to turn the problem into a pure feasibility check.

Next, `prob = cp.Problem(cp.Minimize(0), constraints)` constructs the convex program. The constraints encode the subgradient condition that must hold for a function to be convex at point x_j:

$$f(x_i) \geq f(x_j) + \langle g, x_i - x_j \rangle \quad \text{for all } i \neq j.$$

The command `prob.solve()` invokes the solver. If it finds a feasible vector $g \in \mathbb{R}^d$ satisfying all the inequalities, then the function values are consistent with convexity at x_j.

Finally, the line `return prob.status in [cp.OPTIMAL, cp.OPTIMAL_INACCURATE]` checks whether the solver successfully found such a subgradient. If the solver returns `OPTIMAL`, then the constraints were satisfied exactly. If it returns `OPTIMAL_INACCURATE`, then the constraints were satisfied approximately, which is still acceptable in practice. The function returns `True` in either case, indicating that convexity is locally satisfied at x_j.

def prime_interpolation_check()

```
25  # check primal LP each point j. Use this version of check if we have relative small number of points.
26  # Input: points are array of points, values are array of values where each point evaluates.
27  # Output: true if points can interpolate the convex function.
28  def prime_interpolation_check(points, values):
29      return all(check_prime(points, values, j) for j in range(len(points)))
30
```

Explanation of `prime_interpolation_check`

The function `prime_interpolation_check(points, values)` verifies whether a set of given points in $\mathbb{R}^d$, along with associated scalar values, can be interpolated by a convex function. It does so by invoking a pointwise convexity check based on the subgradient condition.

Inputs

- `points`: an $N \times d$ array representing N points $x_1, \ldots, x_N \in \mathbb{R}^d$.
- `values`: a length-N array of real numbers $f_1, \ldots, f_N$, where $f_i = f(x_i)$ for some unknown function f.

Method

The function checks each index $j = 0, \ldots, N-1$ by calling the helper function `check_prime(points, values, j)`, which attempts to find a subgradient $g_j \in \mathbb{R}^d$ at point x_j such that the subgradient inequality holds for all $i \neq j$:

$$f(x_i) \geq f(x_j) + \langle g_j, x_i - x_j \rangle.$$

In other words, it tests whether the tangent plane at point x_j lies below all other evaluated points. If such a subgradient exists for each j, then the data are consistent with the values being generated by a convex function.

Implementation

The logic is implemented as follows:

```
def prime_interpolation_check(points, values):
    return all(check_prime(points, values, j) for j in range(len(points)))
```

This line iterates through all indices j and ensures that the subgradient test passes at each point. The function returns `True` if and only if the condition holds for every $j \in \{0, \ldots, N-1\}$.

Use Case

This version of the interpolation check is appropriate when the number of sample points N is relatively small, since it solves one linear program per point in the primal space of dimension d.

check_dual()

```python
# Input: points are array of points, values are array of values where each point evaluates, j is index of points
# Output: true if point j has the tangent plane lays below all the points.
def check_dual(points, values, j):
    N = len(points)
    alpha = cp.Variable(N)

    constraints = [
        alpha >= 0,
        cp.sum(cp.multiply(alpha[:, None], points - points[j]), axis=0) == 0
    ]

    objective = cp.Minimize(cp.sum(cp.multiply(alpha, values - values[j])))

    prob = cp.Problem(objective, constraints)
    prob.solve()

    # If the optimal value is -\infty, dual is feasible, then the primal is infeasible
    return prob.status != cp.UNBOUNDED  # return True means primal is feasible
```

Dual Linear Program for Convexity Interpolation

We consider the function `check_dual(points, values, j)`, which tests whether a given set of function values is consistent with convexity at point x_j using a dual linear program (LP). This method provides a certificate for the existence (or failure) of a valid subgradient at x_j.

Problem Setting

Given:

- A set of points $x_1, x_2, \ldots, x_N \in \mathbb{R}^d$
- Corresponding scalar values $f_1, f_2, \ldots, f_N \in \mathbb{R}$

Our goal is to determine whether these values can be interpolated by a convex function. That is, we ask whether there exists a convex function f such that $f(x_i) = f_i$ for all i.

Subgradient Inequality (Primal Condition)

A necessary condition for convexity at point x_j is the existence of a subgradient $g_j \in \mathbb{R}^d$ such that:

$$f(x_i) \geq f(x_j) + \langle g_j, x_i - x_j \rangle \quad \text{for all } i \neq j.$$

This condition forms the basis of the *primal* LP. If such a subgradient exists, then a tangent plane at x_j lies below all other points.

Dual Linear Program

Rather than directly solving for g_j, the dual LP attempts to show whether the primal is infeasible. It introduces dual variables $\alpha \in \mathbb{R}^N$, with the following formulation:

Variables

$$\alpha_1, \ldots, \alpha_N \in \mathbb{R}$$

Constraints

$$\alpha_i \geq 0 \quad \text{for all } i = 1, \ldots, N$$

$$\sum_{i=1}^{N} \alpha_i (x_i - x_j) = 0$$

The first condition ensures non-negativity of the dual weights. The second condition enforces that the weighted displacement from point x_j has zero net direction, i.e., the convex combination of vectors $x_i - x_j$ is centered.

Objective

$$\min_{\alpha} \sum_{i=1}^{N} \alpha_i (f_i - f_j)$$

This can be rewritten as:

$$\sum_{i=1}^{N} \alpha_i f_i - f_j \sum_{i=1}^{N} \alpha_i$$

The objective seeks to minimize the gap between the convex combination of function values f_i and the anchor value f_j.

Interpretation

If the dual LP is *unbounded below*, then the objective can be made arbitrarily negative under the given constraints. In this case, no subgradient g_j can satisfy the subgradient inequality, meaning the primal LP is infeasible and convexity fails at x_j.

On the other hand, if the dual LP has a bounded solution (i.e., the solver status is not `UNBOUNDED`), then the subgradient inequality may hold, and convexity at x_j cannot be ruled out.

Return Value

The code returns `True` if:

$$\texttt{prob.status} \neq \texttt{cp.UNBOUNDED}$$

This indicates that the primal LP is feasible — a subgradient satisfying the convexity condition exists at x_j.

Note: Xuan's implementation swaps the roles of i and j, so the notation in his code does not match the indexing used in my other Polymath Jr write ups.

```
# check dual LP each point j. Use this version of check if we have large number of points.
# Input: points are array of points, values are array of values where each point evaluates.
# Output: true if points can interpolate the convex function.
def dual_interpolation_check(points, values):
    return all(check_dual(points, values, j) for j in range(len(points)))

```

4.7 convex_function_interpolation.py

On the next page!

Convex function interpolation.py

James Evans

07/22/2025

Note: `compute_beta_j_primal` and `compute_beta_star_primal` are alternative but mathematically equivalent implementations of Tony's `linear_programming.py` program located in the Interpolation Problem subfolder.

```
35
36  def compute_beta_j_dual(points, values1, values2, j):
37      N, d = points.shape
38
39      delta_x = points - points[j]         # shape (N, d)
40      delta_y = values1 - values1[j]       # shape (N,)
41      delta_yprime = values2 - values2[j]  # shape (N,)
42
43      lam = cp.Variable(N, nonneg=True)
44
45      # Constraints:
46      constraints = []
47
48      # sum_i λ_i * (x_i - x_j) = 0
49      for k in range(d):
50          constraints.append(cp.sum(cp.multiply(lam, delta_x[:, k])) == 0)
51
52      # sum_i λ_i * (y'_i - y'_j) = 1
53      constraints.append(cp.sum(cp.multiply(lam, delta_yprime)) == 1)
54
55      # Objective: minimize sum_i λ_i * (y_i - y_j)
56      objective = cp.Minimize(cp.sum(cp.multiply(lam, delta_y)))
57
58      prob = cp.Problem(objective, constraints)
59      prob.solve()
60
61      return prob.value
```

def compute_beta_j_dual()

Given a collection of points $x_1, \ldots, x_N \in \mathbb{R}^d$, associated scalar function values $y_i = f(x_i)$, and *auxiliary function values* $y_i' = g(x_i)$ (where g is a second scalar function evaluated at the same samples), the goal is to assess whether these samples are consistent with a convex function. Specifically, for a fixed reference point

x_j, we seek to construct a dual certificate that verifies whether the observed triplet (x_j, y_j, y'_j) is compatible with the existence of a convex interpolant of f (with y'_i used only for normalization).

Mathematical Setup

Define the differences:

$$\begin{aligned}\delta x_i &:= x_i - x_j \in \mathbb{R}^d,\\ \delta y_i &:= y_i - y_j \in \mathbb{R},\\ \delta y'_i &:= y'_i - y'_j \in \mathbb{R}.\end{aligned}$$

Let $\lambda \in \mathbb{R}^N_{\geq 0}$ be a nonnegative weight vector. The optimization problem considered is:

$$\begin{aligned}\beta^*_j := \min_{\lambda \in \mathbb{R}^N_{\geq 0}} \quad & \sum_{i=1}^N \lambda_i \, \delta y_i \\ \text{subject to} \quad & \sum_{i=1}^N \lambda_i \, \delta x_i = 0 \quad \text{(centering constraint)},\\ & \sum_{i=1}^N \lambda_i \, \delta y'_i = 1 \quad \text{(normalization via auxiliary values)}.\end{aligned}$$

Centering Constraint

The constraint

$$\sum_{i=1}^N \lambda_i (x_i - x_j) = 0$$

says that the weighted sum of the vectors from x_j to each x_i must cancel out. In other words, the optimization is only allowed to combine the x_i in such a way that their net effect, relative to x_j, is zero.

Why is this important? Because x_j is the reference point: it's the location where we are testing whether the function value y_j can be explained by nearby data under convexity. If this constraint were not present, the optimization could shift attention to a different location — not x_j — and give a result that has nothing to do with what's happening at x_j.

This constraint prevents that. It ensures that the λ_i-weighted combination of points does not move the center of analysis away from x_j. That way, the test remains anchored at x_j, and we are truly checking whether the function is behaving convexly around that specific point.

Normalization via y'

The constraint

$$\sum_{i=1}^N \lambda_i (y'_i - y'_j) = 1$$

uses the auxiliary values $y'_i = g(x_i)$ purely to *fix the scale* of the weights. Each term $\delta y'_i = y'_i - y'_j$ is a scalar difference of the auxiliary function at x_i relative to x_j. The weights λ_i determine how these differences combine.

By requiring the total weighted auxiliary difference to equal 1, the optimizer cannot choose the trivial $\lambda = 0$ vector. Any other nonzero normalization (e.g. $\sum_i \lambda_i = 1$) would serve a similar scaling role; here we normalize with respect to y' as written in the code.

Objective

The objective

$$\sum_{i=1}^{N} \lambda_i (y_i - y_j)$$

computes a weighted sum of differences between each function value y_i and the reference value y_j. It quantifies how much higher the surrounding values are compared to y_j, according to the weights λ_i. The minimum of this expression gives the smallest such weighted discrepancy consistent with the constraints. A negative optimum indicates inconsistency with convexity at x_j.

```
62
63  def compute_beta_star_dual(points, values1, values2):
64      N = points.shape[0]
65      beta_array = np.zeros(N)
66
67      for j in range(N):
68          beta_array[j] = compute_beta_j_dual(points, values1, values2, j)
69
70      beta_star = np.min(beta_array)
71      return beta_star
72
```

def compute_beta_star_dual()

What the Code Computes

This function evaluates the dual convexity test at every sample point x_j, using the surrounding function values y_i (with y_i' only for normalization as above).

For each $j = 1, \ldots, N$, it computes a quantity β_j^* by solving a linear program centered at x_j. This program checks whether the data near x_j is locally consistent with convexity.

Once all β_j^* values are computed, the function returns

$$\beta^* = \min_j \beta_j^*.$$

This value β^* represents the smallest (most critical) result across all test points. The code is explicitly looping through every point in the dataset, running the dual test at each one individually, collecting the results, and returning the worst-case value.

Why Take the Minimum over β_j^*?

Each value β_j^* is defined as the optimal value of a minimization problem centered at a single point x_j:

$$\beta_j^* := \min_{\lambda \in \mathbb{R}^N_{\geq 0}} \sum_{i=1}^{N} \lambda_i (y_i - y_j) \quad \text{subject to constraints.}$$

This quantity measures how well the surrounding data supports convexity at x_j. If $\beta_j^* < 0$, it indicates a violation of convexity at that point. If $\beta_j^* \geq 0$, the local data is consistent with a convex function.

To evaluate whether the dataset as a whole is convex-consistent, we define

$$\beta^* := \min_j \beta_j^*.$$

This takes the worst-case value over all sample points. If $\beta^* < 0$, then convexity fails at at least one point. If $\beta^* \geq 0$, then all local tests pass.

Taking the minimum over j is therefore necessary: even a single failure invalidates convexity. The final value β^* acts as a global certificate—convexity is confirmed if and only if $\beta^* \geq 0$.

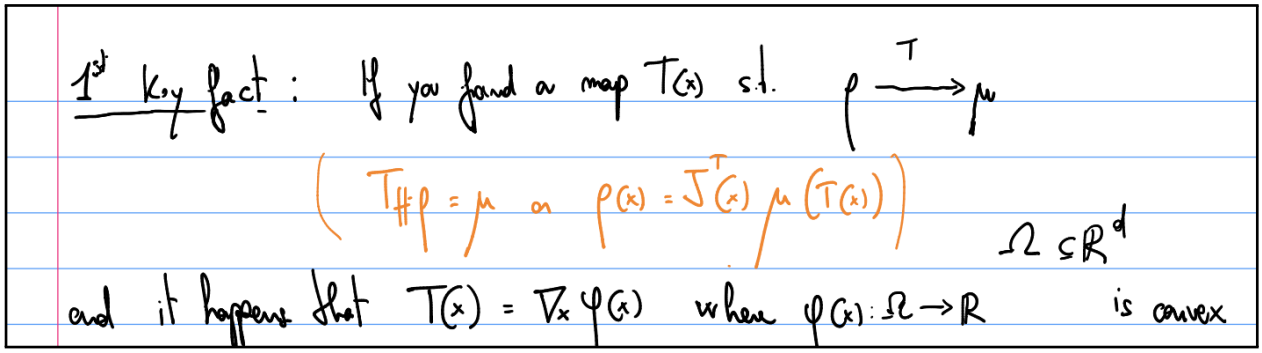
4.8 Giulio's Notes from 07-25-2025

On the next page!

Giulio

Polymath Jr

07/25/2025



Brenier's Theorem

Let ρ and μ be two probability measures on $\mathbb{R}^d$. Assume:

- ρ is absolutely continuous with respect to Lebesgue measure (i.e., it has a density $\rho(x)$),
- $\Omega \subset \mathbb{R}^d$ is convex,
- The cost function is the squared Euclidean distance:

$$c(x, y) = \frac{1}{2}\|x - y\|^2$$

Then there exists a unique optimal transport map T pushing ρ to μ which minimizes the transport cost, and this optimal map is of the form:

$$T(x) = \nabla\varphi(x)$$

for some convex scalar function φ.

Geometric Intuition

If you think of ρ as mass (such as sand) on Ω, and you're trying to move it to match a target shape μ (such as a hole), the most efficient way to do it under squared distance cost is to push it along the gradient of a convex potential.

Mathematical Consequences

When $T = \nabla\varphi$, the Jacobian determinant change-of-variable formula becomes:

$$\rho(x) = \det(D^2\varphi(x)) \cdot \mu(\nabla\varphi(x))$$

This relates the second derivative (Hessian) of the convex potential φ to the densities ρ and μ, and is used in Monge–Ampère equations in optimal transport.

Why Must $\Omega \subset \mathbb{R}^d$ Be Convex?

The domain $\Omega \subset \mathbb{R}^d$ being convex means for any two points $x_1, x_2 \in \Omega$, the line segment connecting them is also entirely contained in Ω. In symbols:

$$\forall x_1, x_2 \in \Omega,\ \forall \lambda \in [0,1], \quad \lambda x_1 + (1-\lambda)x_2 \in \Omega$$

To clarify, the expression

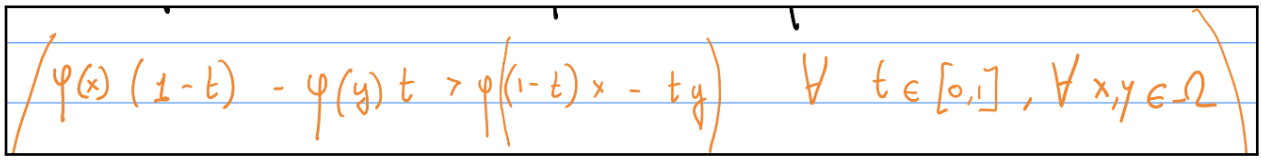
$$x(\lambda) = \lambda x_1 + (1-\lambda)x_2$$

produces a point between x_1 and x_2:

- If $\lambda = 0$, you're at x_2
- If $\lambda = 1$, you're at x_1
- If $\lambda = 0.5$, you're at the midpoint
- If $\lambda = 0.25$, you're 25% of the way from x_1 to x_2
- If $\lambda = 0.75$, you're 75% of the way from x_1 to x_2

This is why $\lambda x_1 + (1-\lambda)x_2$ is called a **convex combination**, and as $\lambda \in [0,1]$, it traces the full **closed line segment** between the two points.

Many results in convex analysis (for example the existence and uniqueness of solutions to Monge–Ampère equations) require convexity of the domain.



Understanding Convex Combinations and Convexity

Disclaimer: The equation written in the notes, namely

$$\varphi(x)(1-t) - \varphi(y)t > \varphi((1-t)x - ty),$$

is **incorrect**. It does not reflect the definition of convexity. The correct form involves convex combinations and is stated below in step 2.

Step 1: Convex Combination Definition

The expression

$$x(\lambda) = \lambda x_1 + (1-\lambda)x_2$$

parametrizes the **line segment** between x_2 and x_1. This is a **function that outputs a point** between two fixed vectors.

Note: this version of the convex combination puts x_1 first. That's totally valid — both:

$$\lambda x_1 + (1-\lambda)x_2 \quad \text{and} \quad (1-t)x + ty$$

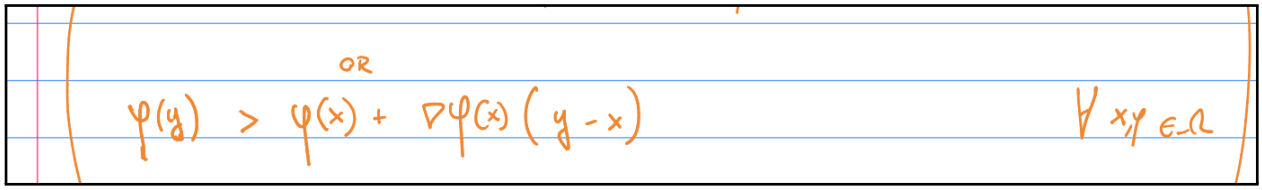
are convex combinations. They're just written with different symbols and ordering.

Step 2: Use a General Convex Combination for Convexity

To define convexity of a function φ, we use:

$$\varphi\big((1-t)x+ty\big) \le (1-t)\varphi(x)+t\varphi(y) \quad \forall t\in[0,1],\ \forall x,y\in\Omega.$$

In other words the function evaluated at any point on the **line segment between** x **and** y is less than or equal to the straight-line interpolation between $\varphi(x)$ and $\varphi(y)$.



This equation says the function φ lies above its tangent hyperplane at every point x. **Note:** The expression

$$\varphi(y) \ge \varphi(x) + \nabla\varphi(x)(y-x)$$

does imply a dot product between the gradient vector $\nabla\varphi(x)$ and the displacement vector $y-x$. More formally, it should be written as:

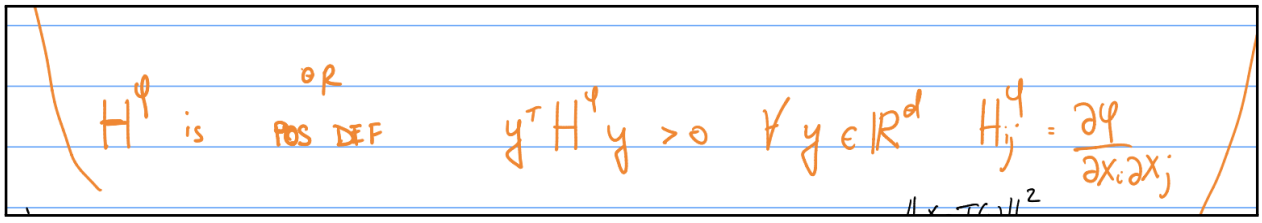
$$\varphi(y) \ge \varphi(x) + \nabla\varphi(x)^\top (y-x) \quad \text{or} \quad \varphi(y) \ge \varphi(x) + \langle \nabla\varphi(x), y-x\rangle$$

Tangent Plane Formula

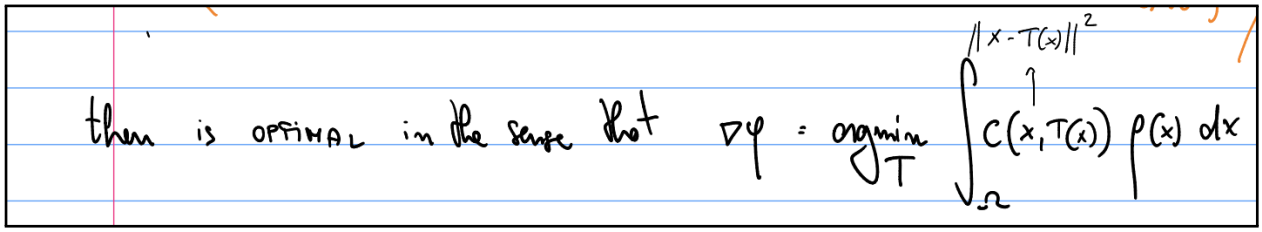
The equation of a tangent plane to a surface given by $z=f(x,y)$ at a point $(a,b,f(a,b))$ is:

$$z = f(a,b) + f_x(a,b)(x-a) + f_y(a,b)(y-b)$$

where f_x and f_y are the partial derivatives of f with respect to x and y, respectively.



$$S \succeq 0 \iff x^\top S x \ge 0 \quad \forall x\in\mathbb{R}^n \iff \lambda_i \ge 0 \quad \text{for all eigenvalues } \lambda_i \text{ of } S$$



Intuition Behind the Optimal Transport Cost

The map $T = \nabla\varphi$ is optimal in the sense that it minimizes the total transport cost:

$$\min_T \int_\Omega c(x, T(x))\, \rho(x)\, dx,$$

where $\rho(x)$ is the probability density function on the domain $\Omega \subset \mathbb{R}^d$, and $c(x, T(x))$ is the cost of transporting mass from x to $T(x)$.

How to interpret $\rho(x)$:

- A single point x has zero measure, so the value $\rho(x)$ itself does *not* represent probability or mass.
- Instead, to estimate the probability of mass near x, we consider a small neighborhood of width δx and write:
$$\mathbb{P}(x \le X \le x + \delta x) \approx \rho(x)$$

Putting this into the transport cost integral:

- The expression $c(x, T(x))\, \rho(x)\, dx$ gives the cost of moving the infinitesimal amount of mass $\rho(x)\, dx$ from x to $T(x)$.
- Integrating over the entire domain Ω sums up the cost of moving *all mass*, weighted according to the density at each location.

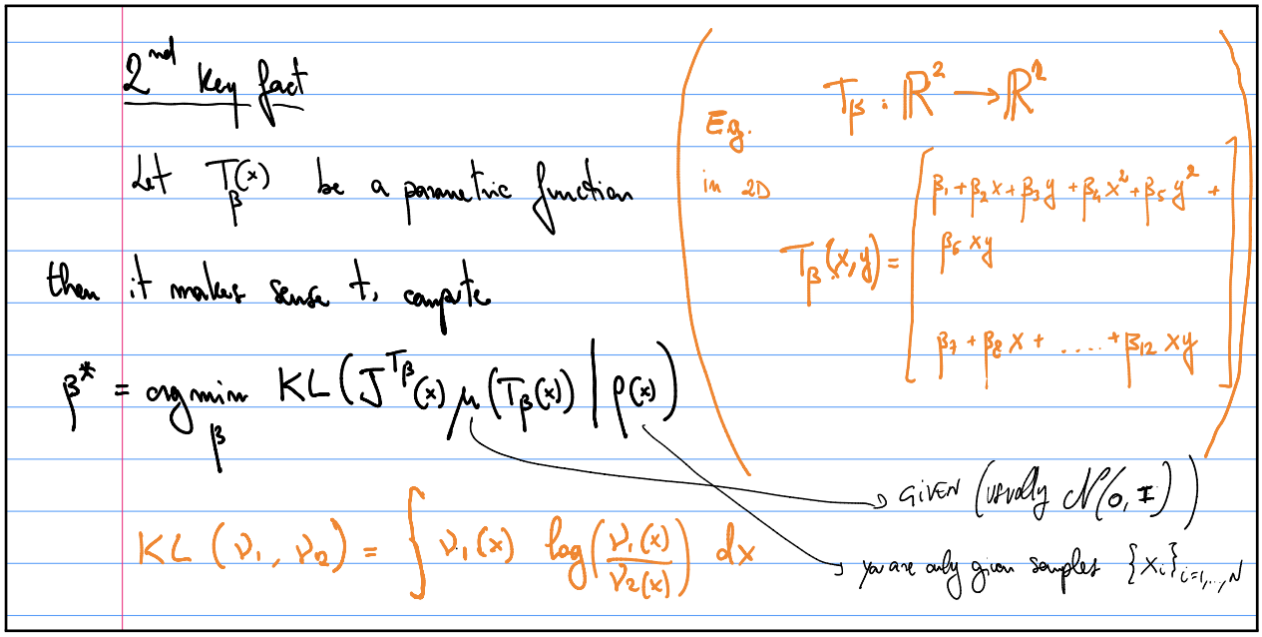
In the case of quadratic cost:

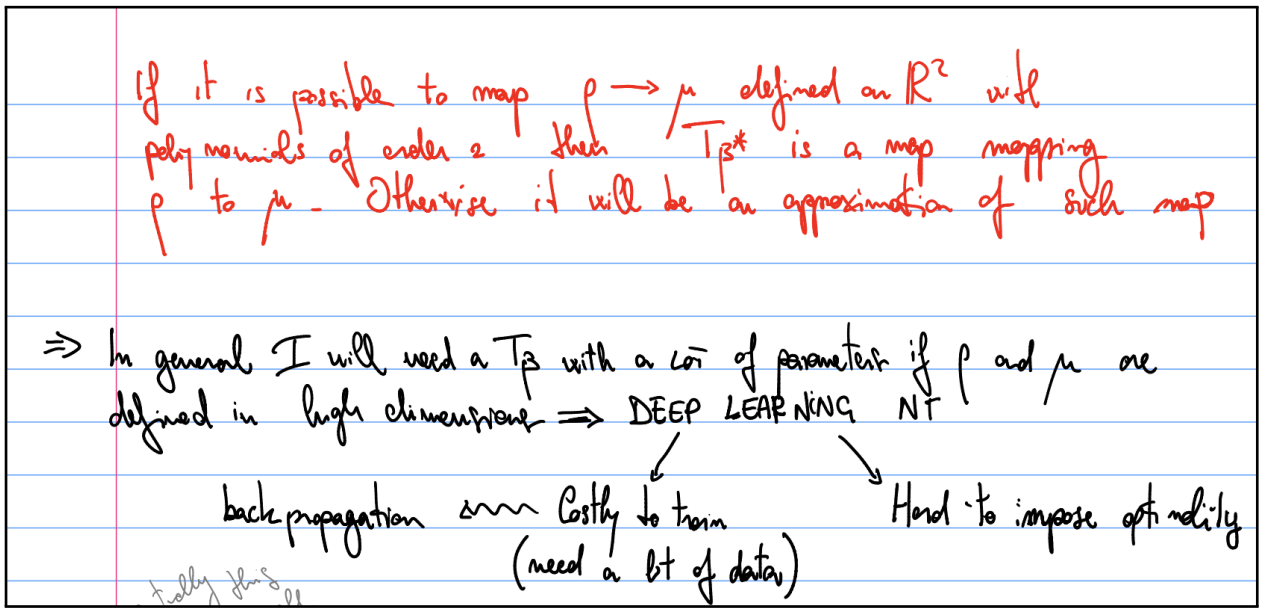
$$c(x, T(x)) = \|x - T(x)\|^2,$$

the total cost becomes:

$$\int_\Omega \|x - T(x)\|^2\, \rho(x)\, dx.$$

This measures how far the mass is being moved, squared and weighted by how much mass is located at each $x \in \Omega$.





Why Use Neural Networks to Represent the Transport Map T^β

When you're working with **optimal transport** between two probability distributions ρ and μ, the goal is to learn a map

$$T^\beta : \mathbb{R}^d \to \mathbb{R}^d$$

that pushes ρ forward to μ, while ideally minimizing a transport cost (e.g., squared Euclidean distance). Now if ρ and μ are defined in **high-dimensional space** (e.g., $d = 100$ or more), then:

- T^β is a function from high-dimensional input to high-dimensional output.
- In general, such functions are **complex** and can exhibit intricate structures (e.g., nonlinearities, interactions between coordinates, etc.).
- Traditional function approximators (e.g., linear models, polynomials) often **fail** in these settings because they are too rigid.

Why Neural Networks (Deep Learning)?

Neural networks are **universal function approximators**—they can approximate a wide variety of functions using relatively compact representations, especially when:

- The data lives in high dimensions.
- The function to learn (here, the transport map T^β) has nontrivial structure.

By stacking multiple nonlinear layers, **deep nets** can:

- Capture complex relationships across many variables.
- Reuse patterns across dimensions.
- Efficiently model functions in high-dimensional spaces **without explicitly enumerating all variables**.

Thus, when you're trying to learn a map T^β between high-dimensional distributions, deep learning gives you a **flexible and powerful framework** to represent T^β, which you can train using data sampled from ρ and μ.

Neural Networks: Setup and Benefits

- You define $T^{\beta}(x) = \mathrm{NN}_{\beta}(x)$.
- You choose the **architecture** (layers, activations, etc.), and let β represent all the trainable weights.
- Neural networks are **expressive enough** to approximate many kinds of functions.

Downsides of Deep Learning

Despite its flexibility, deep learning comes with some notable downsides:

- You must use **backpropagation** to train the model.
- It can be **costly to train**, especially in high dimensions, since you typically need a **large amount of data**.
- It is **hard to impose optimality constraints**—such as convexity or theoretical structure from optimal transport—on the learned neural network function.

Training neural networks: Setup

- Training data

$$S_n = \{(\bar{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$$

$$\bar{x} \in \mathbb{R}^d \qquad y \in \{-1, +1\}$$

- Neural network

$$\bar{\theta} = [w_{10}^{(1)}, w_{11}^{(1)}, \ldots, w_{JK}^{(L)}]$$

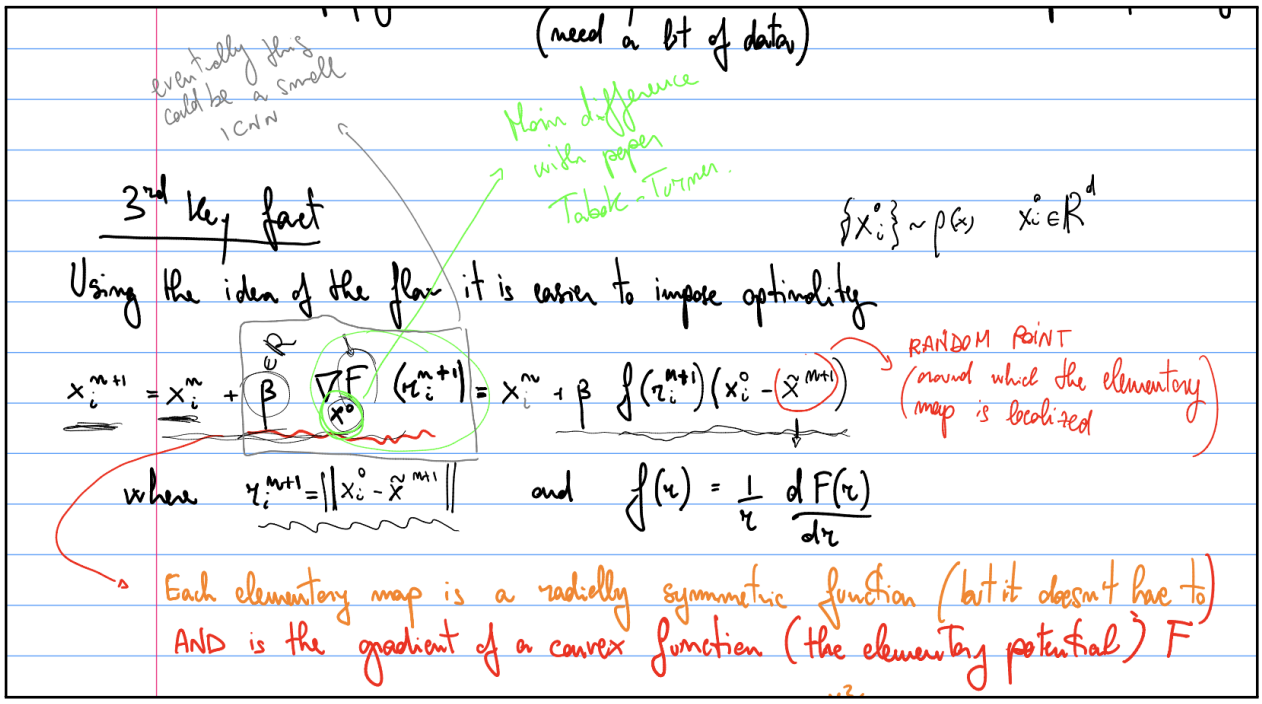
$$h(\bar{x}; \bar{\theta}) = z^{(L+1)} = z_1^{(L+1)}$$

your output layer

h represents your whole network

- Loss function

$$J(\bar{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \mathrm{Loss}(y^{(i)}, h(\bar{x}^{(i)}; \bar{\theta}))$$



Update Rule and Notation

We define an incremental transport map by updating each point $x_i \in \mathbb{R}^d$ via a sequence of elementary maps. The update rule at step n is:

$$x_i^{n+1} = x_i^n + \beta\, f(z_i^{n+1})\,(x_i^0 - \bar{x}^{n+1})$$

Main idea

Find $T = T_n \circ \ldots \circ T_1$ as a composition of elementary maps, easier to parametrize $T_k = T_{\beta^k}$

Explanation of Terms

- x_i^0: the original sample from the source distribution ρ
- x_i^n: the position of point i after n transformation steps
- $\bar{x}^{n+1}$: a randomly sampled center point (from ρ) used to localize the next elementary map
- $z_i^{n+1} = \|x_i^0 - \bar{x}^{n+1}\|$: the Euclidean distance from the original point x_i^0 to the center
- $f(z)$: a scalar radial function controlling the magnitude of movement, defined below
- β: a scalar step size (learned or fixed)

🔁 **Sampling Center Points** $\bar{x}^{n+1}$

At each step $n+1$, you randomly pick a new center point $\bar{x}^{n+1} \sim \rho$.

- If, by chance, the same **original point** is selected as $\bar{x}^{n+1}$ again at some later step $m+1$, it **might have moved** since the first time it was selected.
- That is:

If $\bar{x}^{n+1} = x_j^0$ and later $\bar{x}^{m+1} = x_j^0$ again,

then the **value of that point has changed** because it has undergone several transformations:

$x_j^0 \to x_j^1 \to x_j^2 \to \cdots \to x_j^m$

Radial Function Definition

We consider scalar potentials $F(x)$ of the form:

$$F(x) = \Phi(\|x - \bar{x}\|),$$

where:

- $\bar{x} \in \mathbb{R}^d$ is a fixed center point (e.g., sampled from the source distribution ρ),
- $r = \|x - \bar{x}\|$ is the radial distance from x to $\bar{x}$,
- $\Phi : \mathbb{R}_{\geq 0} \to \mathbb{R}$ is a smooth convex scalar function.
- $F(x)$ is a radially symmetric and convex function.

Gradient of the Potential

To compute $\nabla F(x)$, we apply the chain rule:

$$\begin{aligned} \nabla F(x) &= \frac{d}{dr}\Phi(r) \cdot \nabla r \\ &= \Phi'(r) \cdot \frac{x - \bar{x}}{\|x - \bar{x}\|} \\ &= \Phi'(r) \cdot \frac{x - \bar{x}}{r}. \end{aligned}$$

We define the radial function:

$$f(r) := \frac{1}{r}\frac{d\Phi(r)}{dr},$$

so that the gradient becomes:

$$\nabla F(x) = f(r)(x - \bar{x}).$$

Step-by-step breakdown

Write the norm as a square root of an inner product:

$$r(x) = \|x - \bar{x}\| = \sqrt{(x - \bar{x})^\top (x - \bar{x})}$$

Differentiate using the chain rule:

Note: Since $r(x)$ is a scalar-valued function of a single vector variable x, computing the gradient of r is equivalent to differentiating with respect to x.

$$\frac{d}{dx}\sqrt{(x-\bar{x})^\top(x-\bar{x})}$$

Let's set $u(x) = (x-\bar{x})^\top(x-\bar{x})$. Then $r(x) = \sqrt{u(x)}$, and we use the chain rule:

$$\frac{d}{dx}r(x) = \frac{1}{2\sqrt{u(x)}} \cdot \frac{d}{dx}u(x)$$

Differentiate the inner function:

$$\frac{d}{dx}\left((x-\bar{x})^\top(x-\bar{x})\right) = 2(x-\bar{x})$$

Put it all together:

$$\frac{d}{dx}\|x-\bar{x}\| = \frac{1}{2\|x-\bar{x}\|} \cdot 2(x-\bar{x}) = \frac{x-\bar{x}}{\|x-\bar{x}\|}$$

Clarifying the Derivative Step

Let's clarify where the 2 in the denominator came from in this step:

We are differentiating:

$$r(x) = \|x-\bar{x}\| = \sqrt{(x-\bar{x})^\top(x-\bar{x})}$$

Let's denote:

$$u(x) = (x-\bar{x})^\top(x-\bar{x}) \quad \text{so} \quad r(x) = \sqrt{u(x)}$$

$$\frac{d}{dx}\sqrt{u(x)} = \frac{1}{2\sqrt{u(x)}} \cdot \frac{d}{dx}u(x)$$

That 2 in the denominator comes from the derivative of the square root:

$$\frac{d}{dx}\sqrt{u} = \frac{1}{2\sqrt{u}} \cdot \frac{du}{dx}$$

Interpretation

This shows that the flow field defined by $f(r)(x-\bar{x})$ is the gradient of the convex function $F(x) = \Phi(\|x-\bar{x}\|)$. Since Φ is convex and composition with the norm preserves convexity, F is convex as well. Therefore, each such flow is guaranteed to be the gradient of a convex function.

Connection to Brenier's Theorem

Recall from Brenier's Theorem: Let ρ and μ be two probability measures on $\mathbb{R}^d$, and assume:

- ρ is absolutely continuous with respect to Lebesgue measure (i.e., has a density $\rho(x)$),
- the domain $\Omega \subset \mathbb{R}^d$ is convex,
- and the cost function is the squared Euclidean distance:

$$c(x,y) = \frac{1}{2}\|x-y\|^2,$$

Then there exists a unique optimal transport map T pushing ρ to μ which minimizes the transport cost, and this optimal map is of the form:

$$T(x) = \nabla\varphi(x)$$

for some convex function φ.

Therefore, the elementary maps used in our construction — which are gradients of convex scalar potentials — are consistent with the structural form of optimal transport maps prescribed by Brenier's Theorem.

Interpretation

Each elementary map is defined by a localized radial transformation centered at $\bar{x}^{n+1}$. The idea is to push the point x_i^n slightly in the direction from $\bar{x}^{n+1}$ to x_i^0, scaled by a function of how far x_i^0 is from the center.

- The vector $x_i^0 - \bar{x}^{n+1}$ gives the direction of flow.
- The scalar function $f(z_i^{n+1})$ gives the magnitude of the flow.
- This flow is applied additively to the current position x_i^n, yielding x_i^{n+1}.

Note: The formulas

$$T(x) = \nabla\varphi(x) \quad \text{and} \quad x_i^{m+1} = x_i^m + \beta\nabla F(x_i^m)$$

represent the same underlying idea: both describe a transport map defined by the gradient of a convex function. The first is a continuous-time formulation from optimal transport, while the second is a discrete-time update used in iterative methods. They are two versions of the same gradient flow under different time settings.

One idea on OT

We need to impose the convexity of the potential → we want the map to be the gradient of a convex function. If we build the flow

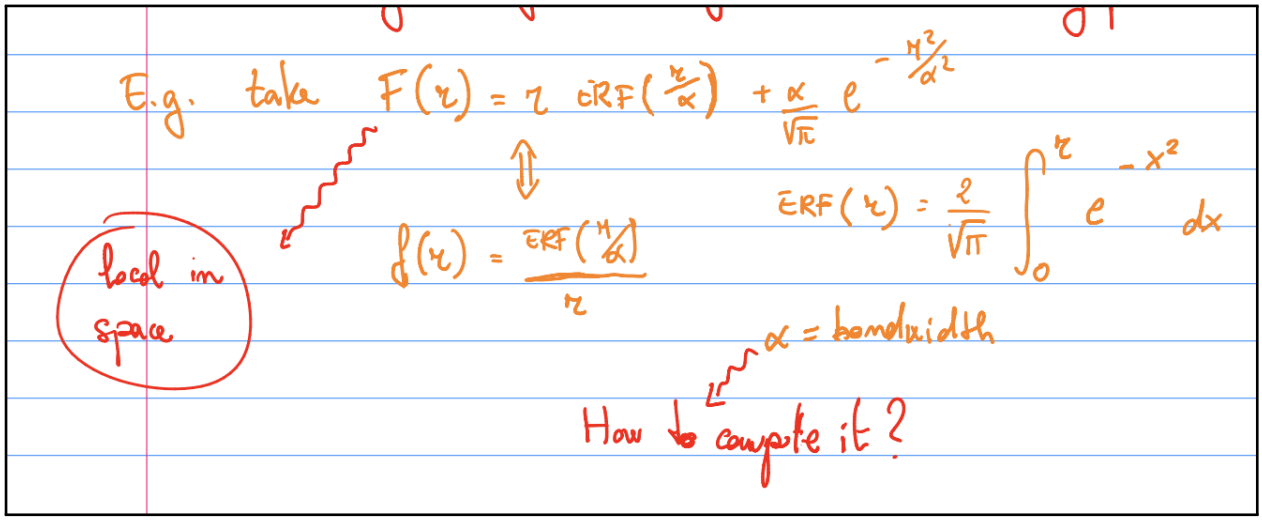
$$z^{k+1} = z^k + \beta^k \nabla_x F(z^k) = z^0 + \sum_{i=1}^{k} \nabla_x F(z^i)$$

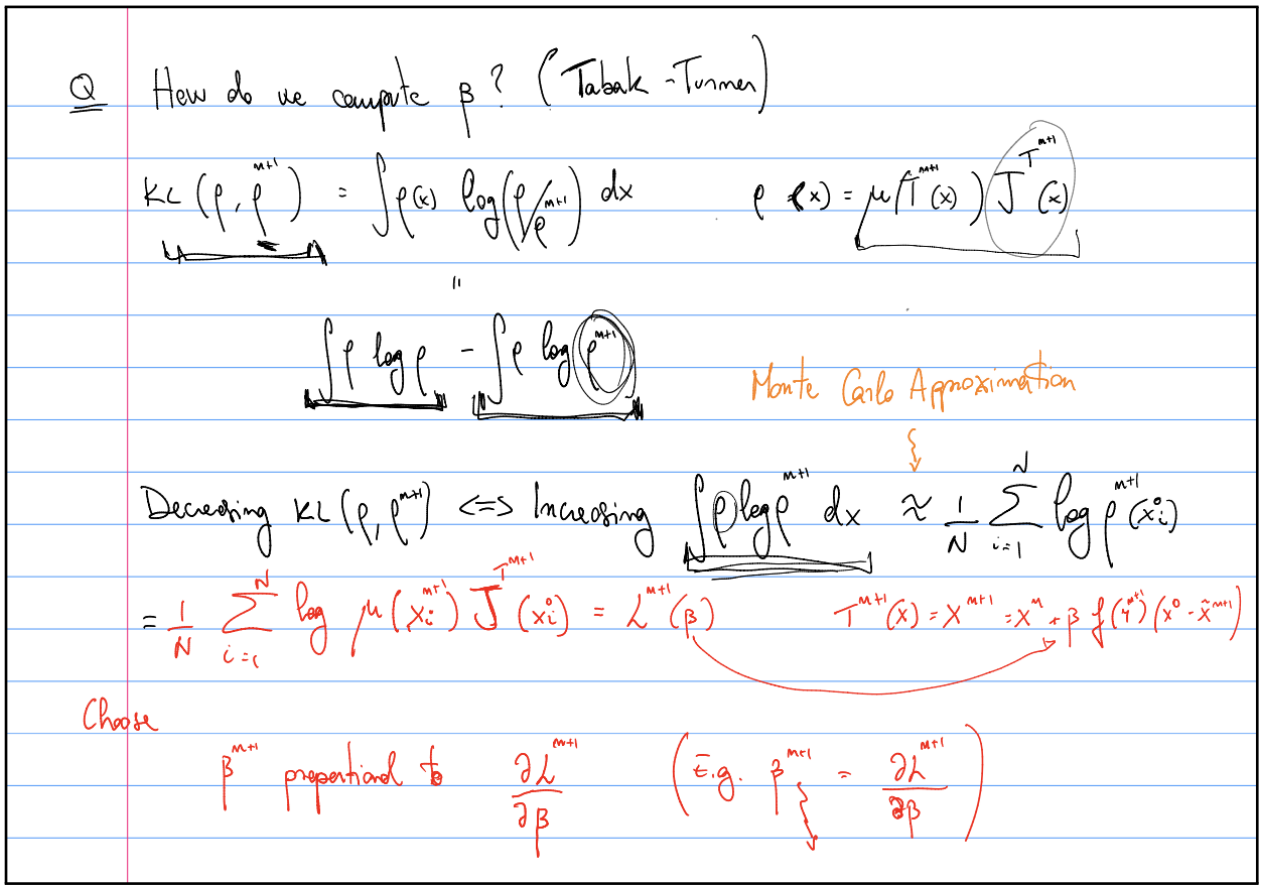
with $z^0 = x$ the starting position, $\beta \in R$, and F convex, then the convexity of the potential depends on the sign of β.

- When β^i is positive there is no problem (sum of convex functions is still convex)
- If β is negative we need to pay attention
- We only have the value $\sum_{i=1}^{k} \nabla_x F(z^i)$ at the sample points z^i
- A condition that $\sum_{i=1}^{k} \nabla_x F(z^i)$ should satisfy is that, at least, should interpolate a convex function

Recall that the gradient is a linear operator, so the gradient of a sum is equal to the sum of the gradients.

Baptista, Trigila, Wang PolyMathJr 2025: ML project 10 / 13





Computing the Step Size β^{m+1} in Particle Flow (Tabak–Turner)

We aim to compute the step size β^{m+1} that moves a particle cloud closer to a target distribution ρ using the iterative flow-based method proposed by Tabak and Turner. This is done by minimizing the Kullback–Leibler (KL) divergence between the target ρ and the current particle approximation ρ^{m+1}.

KL Divergence Objective

We aim to minimize:

$$\mathrm{KL}(\rho, \rho^{m+1}) = \int \rho(x) \log \left(\frac{\rho(x)}{\rho^{m+1}(x)} \right) dx$$

which quantifies the discrepancy between the target density ρ and the current approximation ρ^{m+1}.

Expressing ρ^{m+1} via Pushforward

Let μ be a known base distribution, and let T^{m+1} be the transport map at iteration $m+1$. Then:

$$\rho^{m+1}(x) = \mu\left(T^{m+1}(x)\right) \cdot \left|J^{m+1}(x)\right|$$

where $J^{m+1}(x)$ denotes the Jacobian determinant of the transformation T^{m+1}.

Simplifying the KL Expression

Decomposing the KL divergence:

$$\mathrm{KL}(\rho, \rho^{m+1}) = \int \rho(x) \log \rho(x) dx - \int \rho(x) \log \rho^{m+1}(x) dx$$

Since the first term is constant with respect to ρ^{m+1}, minimizing KL is equivalent to maximizing:

$$\int \rho(x) \log \rho^{m+1}(x) dx$$

Monte Carlo Approximation

We estimate this expectation using Monte Carlo samples $x_i^0 \sim \rho$, yielding:

$$\int \rho(x) \log \rho^{m+1}(x) dx \approx \frac{1}{N} \sum_{i=1}^{N} \log \rho^{m+1}(x_i^0)$$

Substituting the expression for ρ^{m+1}, we define the log-likelihood:

$$\mathcal{L}^{m+1}(\beta) := \frac{1}{N} \sum_{i=1}^{N} \log \mu(x_i^{m+1}) + \log \left|J^{m+1}(x_i^0)\right|$$

Transport Map

The particles are updated via the map:

$$T^{m+1}(x^m) = x^m + \beta^{m+1} \cdot f^{(m+1)}(x^0 - x^m)$$

where:

- x^m is the particle position at iteration m,
- x^0 is the original sample,
- $f^{(m+1)}$ is a flow direction field.

Updating β

To improve the fit between ρ^{m+1} and ρ, we maximize $\mathcal{L}^{m+1}(\beta)$ with respect to β. This yields the update:

$$\beta^{m+1} = \frac{\partial \mathcal{L}^{m+1}}{\partial \beta}$$

This tells us **how fast** $\mathcal{L}^{m+1}$ changes **if we slightly increase or decrease** β.

- If this derivative is **positive**, then increasing β makes $\mathcal{L}^{m+1}$ larger (good).
- If it is **negative**, then decreasing β makes $\mathcal{L}^{m+1}$ larger (also good).
- The size of the derivative tells us **how strongly** β should be adjusted.

So the update:

$$\beta^{m+1} = \frac{\partial \mathcal{L}^{m+1}}{\partial \beta}$$

means: "Update β by the amount that improves the log-likelihood the most **in this step**."

4.9 Code Clarification for Sahith's Tabak-Turner Implementation

On the next page!

Code Clarification for Sahith's Tabak-Turner Implementation

James Evans

08/08/2025

```python
def estimate_forward_KL(X, flow, log_rho_exact):
    # compute log-density under the exact model
    log_ex = log_rho_exact(X)

    # compute log-density under the flow estimate
    log_est = flow.log_prob(X)

    # Monte Carlo KL
    kl_mc = np.mean(log_ex - log_est)
    return kl_mc
```

Forward KL Divergence Estimation

We consider the forward Kullback–Leibler divergence from the true distribution ρ to the model $\hat{\rho}$:

$$\mathrm{KL}(\rho \,\|\, \hat{\rho}) \;=\; \int_{\mathbb{R}^n} \rho(x)\, \log \frac{\rho(x)}{\hat{\rho}(x)}\, dx.$$

Expanding the logarithm, we can rewrite this as:

$$\mathrm{KL}(\rho \,\|\, \hat{\rho}) = \int_{\mathbb{R}^n} \rho(x) \big[\log \rho(x) - \log \hat{\rho}(x) \big]\, dx.$$

Monte Carlo Approximation

If $X_1, \ldots, X_m \overset{\text{i.i.d.}}{\sim} \rho$, then by the law of large numbers we have:

$$\frac{1}{m} \sum_{i=1}^{m} \big[\log \rho(X_i) - \log \hat{\rho}(X_i) \big] \;\xrightarrow[m\to\infty]{}\; \mathrm{KL}(\rho \,\|\, \hat{\rho}).$$

Therefore, a Monte Carlo estimator of the forward KL divergence is given by:

$$\widehat{\mathrm{KL}}(\rho \,\|\, \hat{\rho}) \;=\; \frac{1}{m} \sum_{i=1}^{m} \big[\log \rho(X_i) - \log \hat{\rho}(X_i) \big].$$

Interpretation

- The term $\log \rho(X_i)$ is the exact log-density of the true distribution evaluated at the sample X_i.
- The term $\log \hat{\rho}(X_i)$ is the log-density predicted by the model at the same sample.
- The difference $\log \rho(X_i) - \log \hat{\rho}(X_i)$ measures, for that sample, how much more likely it is under the true distribution than under the model.

- Averaging over all samples approximates the expectation $\mathbb{E}_\rho[\,\cdot\,]$ and yields the KL estimate.

```python
def _volume_n_ball(self, n):
    """Calculates the volume of an n-dimensional unit ball."""
    return np.pi**(n/2) / gamma(n/2 + 1)

def _calculate_alpha(self, x0, n, m):
    """
    Calculates the length-scale 'alpha' for a map centered at x0,
    based on equation (13) from the paper. The goal is to have the
    map's influence cover roughly n_p points.

    Args:
        x0 (np.ndarray): The center of the elementary map.
        n (int): The dimensionality of the data space.
        m (int): The total number of data points.

    Returns:
        float: The calculated alpha.
    """
    omega_n = self._volume_n_ball(n)
    # The formula in the paper is derived for a target normal distribution.
    # It adapts the map's radius to be larger in low-density regions.
    term = (omega_n**-1 * self.n_p / m)**(1/n)
    alpha = (2 * np.pi)**0.5 * term * np.exp(np.linalg.norm(x0)**2 / (2*n))
    return alpha
```

Role of ρ and μ in the Algorithm

In the training phase, every point provided to `fit(..)` is assumed to be sampled from the *source density* ρ:

$$x_i \sim \rho.$$

The algorithm learns a transformation that maps ρ to the *target density* μ.

Pipeline Overview

Input: Start with $x_i \sim \rho$.

Preconditioning: We transform each x_i to

$$z_i = \frac{x_i - \bar{x}}{\text{std}},$$

where $\bar{x}$ is the empirical mean and std is a global scaling factor chosen so that the cloud is roughly isotropic. These z_i are still samples from a distribution linearly related to ρ.

Forward Maps $\rho \to \mu$: The algorithm builds a composition of small, localized *radial maps* that progressively transform z_i to resemble the target Gaussian $\mu = \mathcal{N}(0, I_n)$.

Final Map: The learned transformation T satisfies

$$y_i = T(x_i) \quad \text{with} \quad y_i \sim \mu,$$

and, by the change-of-variables formula,

$$\hat{\rho}(x) = \mu\big(T(x)\big)\,\big|\det \nabla T(x)\big|.$$

Mathematical Role of `volume_n_ball` and `calculate_alpha`

The algorithm builds the transport map as a composition of *elementary radial maps* centered at points x_0. The *length scale* α for a given map determines its **region of influence**. We choose α so that the ball $B(x_0, \alpha)$ contains, on average, n_p samples from the *source* distribution ρ (in preconditioned coordinates).

`volume_n_ball` This helper function returns the volume of the n-dimensional unit ball:

$$\omega_n = \frac{\pi^{n/2}}{\Gamma\left(\frac{n}{2}+1\right)}.$$

For any radius $r > 0$, the n-dimensional volume is:

$$\mathrm{Vol}\big(B(0,r)\big) = \omega_n r^n.$$

Small-ball mass requirement

Let $\phi_n(z)$ be the n-dimensional standard Gaussian density:

$$\phi_n(z) = (2\pi)^{-n/2} \exp\left(-\frac{\|z\|^2}{2}\right).$$

In preconditioned space, the *target* μ is $\mathcal{N}(0, I_n)$, so the algorithm estimates α assuming the point cloud is drawn from this μ.

The condition "$B(x_0, \alpha)$ contains n_p points on average" is:

$$m \int_{B(x_0,\alpha)} \phi_n(z)\, dz = n_p,$$

where m is the total number of points.

Local density approximation

If α is small compared to the scale over which ϕ_n changes, we approximate:

$$\phi_n(z) \approx \phi_n(x_0) \quad \text{for } z \in B(x_0, \alpha).$$

Thus:

$$\int_{B(x_0,\alpha)} \phi_n(z)\, dz \;\approx\; \phi_n(x_0) \cdot \mathrm{Vol}\big(B(x_0,\alpha)\big) \;=\; \phi_n(x_0) \cdot \omega_n \alpha^n.$$

Substitution into the mass equation

The mass equation becomes:

$$m \cdot \phi_n(x_0) \cdot \omega_n \alpha^n = n_p.$$

Using the Gaussian density at x_0:

$$\phi_n(x_0) = (2\pi)^{-n/2} \exp\left(-\frac{\|x_0\|^2}{2}\right),$$

we have:

$$m \cdot (2\pi)^{-n/2} \exp\left(-\frac{\|x_0\|^2}{2}\right) \cdot \omega_n \alpha^n = n_p.$$

Solve for α

Rearranging:

$$\alpha^n = \frac{n_p}{m\,\omega_n} \cdot (2\pi)^{n/2} \cdot \exp\left(\frac{\|x_0\|^2}{2}\right).$$

Taking the n-th root:

$$\alpha = \left(\frac{n_p}{m\,\omega_n}\right)^{1/n} \cdot (2\pi)^{1/2} \cdot \exp\left(\frac{\|x_0\|^2}{2n}\right).$$

Why the Small-Ball Mass Equation Represents an Expectation

We have m sample points $z^{(1)}, z^{(2)}, \ldots, z^{(m)}$ drawn i.i.d. from the *target* density $\phi_n(z)$. Define the indicator random variable:

$$X_i = \begin{cases} 1 & \text{if } z^{(i)} \in B(x_0, \alpha), \\ 0 & \text{otherwise.} \end{cases}$$

By the definition of expectation for an indicator variable:

$$\mathbb{E}[X_i] = \mathbb{P}\Big(z^{(i)} \in B(x_0, \alpha)\Big) = \int_{B(x_0,\alpha)} \phi_n(z)\, dz.$$

Now let N_{in} be the total number of points in the ball:

$$N_{\text{in}} = \sum_{i=1}^{m} X_i.$$

By linearity of expectation:

$$\mathbb{E}[N_{\text{in}}] = \sum_{i=1}^{m} \mathbb{E}[X_i] = m \int_{B(x_0,\alpha)} \phi_n(z)\, dz.$$

Thus, the quantity

$$m \int_{B(x_0,\alpha)} \phi_n(z)\, dz$$

is exactly the *expected number of sample points* inside the ball $B(x_0, \alpha)$.

```python
def _radial_f(self, r, alpha):
    """
    The radial localization function f(r), based on equation (24).
    Handles the r=0 case to avoid division by zero.

    Args:
        r (np.ndarray): Array of radial distances ||x - x0||.
        alpha (float): The length-scale of the map.

    Returns:
        np.ndarray: The value of the localization function.
    """
    # Handle r=0 separately to avoid division by zero.
    # The limit of erf(x)/x as x->0 is 2/sqrt(pi).
    f_vals = np.zeros_like(r)
    nonzero_r = r != 0
    zero_r = ~nonzero_r

    r_scaled = r[nonzero_r] / alpha
    f_vals[nonzero_r] = erf(r_scaled) / r[nonzero_r]
    f_vals[zero_r] = 2.0 / (alpha * np.sqrt(np.pi))
    return f_vals
```

Mathematical Role of `_radial_f`

Let $x_0 \in \mathbb{R}^n$ be the center of an elementary radial map, and let

$$r = \|x - x_0\|$$

denote the Euclidean distance from x to x_0. Suppose the potential F is purely radial, i.e. $F(x) = \Phi(r)$ for some scalar function Φ. Then by the chain rule,

$$\nabla F(x) = \frac{d\Phi}{dr} \cdot \frac{x - x_0}{r}.$$

The scalar function

$$f(r) := \frac{d\Phi}{dr}$$

is called the *radial localization function*. In the algorithm, $f(r)$ is defined (based on equation (24) in the paper) as

$$f(r;\alpha) = \begin{cases} \dfrac{\operatorname{erf}\left(\frac{r}{\alpha}\right)}{r}, & r > 0, \\ \dfrac{2}{\alpha\sqrt{\pi}}, & r = 0, \end{cases}$$

where $\alpha > 0$ is the length scale of the map.

Special case $r = 0$. Directly evaluating $\operatorname{erf}(r/\alpha)/r$ at $r = 0$ causes a division-by-zero. Instead, we take the limit:

$$\lim_{r \to 0} \frac{\operatorname{erf}(r/\alpha)}{r}.$$

Using the Taylor expansion $\operatorname{erf}(t) \approx \frac{2}{\sqrt{\pi}}\, t$ for small t, we find:

$$\lim_{r \to 0} \frac{\operatorname{erf}(r/\alpha)}{r} = \lim_{r \to 0} \frac{\frac{2}{\sqrt{\pi}}(r/\alpha)}{r} = \frac{2}{\alpha\sqrt{\pi}},$$

which matches the $r = 0$ branch above.

Interpretation

- For $r > 0$, $f(r;\alpha)$ decays roughly like $1/r$ for large r, localizing the influence of the map.
- For $r = 0$, the finite value $\frac{2}{\alpha\sqrt{\pi}}$ ensures smooth behavior at the center.
- The gradient of the potential is then given by $\nabla F(x) = f(r;\alpha)\,(x - x_0)/r$.

Differentiating the Radially Symmetric Potential

We are given the function:

$$F_\ell(r) = r \cdot \operatorname{erf}\left(\frac{r}{\alpha}\right) + \frac{\alpha}{\sqrt{\pi}} e^{-(r/\alpha)^2}$$

We aim to compute the derivative with respect to r, that is,

$$\frac{dF_\ell}{dr}$$

Differentiate the first term

$$\frac{d}{dr}\left(r \cdot \operatorname{erf}\left(\frac{r}{\alpha}\right)\right)$$

Using the product rule:

$$= \operatorname{erf}\left(\frac{r}{\alpha}\right) + r \cdot \frac{d}{dr}\left[\operatorname{erf}\left(\frac{r}{\alpha}\right)\right]$$

Recall the derivative of the error function:

$$\frac{d}{dx}\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}}e^{-x^2} \quad \Rightarrow \quad \frac{d}{dr}\operatorname{erf}\left(\frac{r}{\alpha}\right) = \frac{2}{\alpha\sqrt{\pi}}e^{-(r/\alpha)^2}$$

So the full derivative of the first term is:

$$\operatorname{erf}\left(\frac{r}{\alpha}\right) + \frac{2r}{\alpha\sqrt{\pi}}e^{-(r/\alpha)^2}$$

Differentiate the second term

$$\frac{d}{dr}\left(\frac{\alpha}{\sqrt{\pi}}e^{-(r/\alpha)^2}\right) = \frac{\alpha}{\sqrt{\pi}} \cdot \left(-\frac{2r}{\alpha^2}\right)e^{-(r/\alpha)^2} = -\frac{2r}{\alpha\sqrt{\pi}}e^{-(r/\alpha)^2}$$

Combine the results

The exponential terms from both derivatives cancel:

$$\frac{2r}{\alpha\sqrt{\pi}}e^{-(r/\alpha)^2} - \frac{2r}{\alpha\sqrt{\pi}}e^{-(r/\alpha)^2} = 0$$

So we are left with:

$$\frac{dF_\ell}{dr} = \operatorname{erf}\left(\frac{r}{\alpha}\right)$$

Radial Function Definition

We consider scalar potentials $F(x)$ of the form:

$$F(x) = \Phi(\|x - \bar{x}\|),$$

where:

- $\bar{x} \in \mathbb{R}^d$ is a fixed center point (e.g., sampled from the source distribution ρ),
- $r = \|x - \bar{x}\|$ is the radial distance from x to $\bar{x}$,
- $\Phi : \mathbb{R}_{\geq 0} \to \mathbb{R}$ is a smooth convex scalar function,
- $F(x)$ is a radially symmetric and convex function.

Gradient of the Potential

To compute $\nabla F(x)$, we apply the chain rule:

$$\begin{aligned}\nabla F(x) &= \frac{d}{dr}\Phi(r) \cdot \nabla r \\ &= \Phi'(r) \cdot \frac{x - \bar{x}}{\|x - \bar{x}\|} \\ &= \Phi'(r) \cdot \frac{x - \bar{x}}{r}.\end{aligned}$$

We define the radial function:

$$f(r) := \frac{1}{r}\frac{d\Phi(r)}{dr},$$

so that the gradient becomes:

$$\nabla F(x) = f(r)(x - \bar{x}).$$

Step-by-step breakdown

Write the norm as a square root of an inner product:

$$r(x) = \|x - \bar{x}\| = \sqrt{(x - \bar{x})^\top (x - \bar{x})}$$

Differentiate using the chain rule:

Note: Since $r(x)$ is a scalar-valued function of a single vector variable x, computing the gradient of r is equivalent to differentiating with respect to x.

$$\frac{d}{dx}\sqrt{(x - \bar{x})^\top (x - \bar{x})}$$

Let's set $u(x) = (x - \bar{x})^\top (x - \bar{x})$. Then $r(x) = \sqrt{u(x)}$, and we use the chain rule:

$$\frac{d}{dx} r(x) = \frac{1}{2\sqrt{u(x)}} \cdot \frac{d}{dx} u(x)$$

Differentiate the inner function:

$$\frac{d}{dx}\left((x - \bar{x})^\top (x - \bar{x})\right) = 2(x - \bar{x})$$

$$\frac{d}{dx}\|x - \bar{x}\| = \frac{1}{2\|x - \bar{x}\|} \cdot 2(x - \bar{x}) = \frac{x - \bar{x}}{\|x - \bar{x}\|}$$

Clarifying the Derivative Step

We are differentiating:

$$r(x) = \|x - \bar{x}\| = \sqrt{(x - \bar{x})^\top (x - \bar{x})}$$

Let's denote:

$$u(x) = (x - \bar{x})^\top (x - \bar{x}) \quad \text{so} \quad r(x) = \sqrt{u(x)}$$

$$\frac{d}{dx}\sqrt{u(x)} = \frac{1}{2\sqrt{u(x)}} \cdot \frac{du(x)}{dx}$$

```python
def _radial_f_prime(self, r, alpha):
    """
    The derivative of the radial localization function, f'(r).
    Handles the r=0 case, where the derivative is 0.

    Args:
        r (np.ndarray): Array of radial distances ||x - x0||.
        alpha (float): The length-scale of the map.

    Returns:
        np.ndarray: The value of the derivative.
    """
    f_prime_vals = np.zeros_like(r)
    nonzero_r = r != 0

    r_nz = r[nonzero_r]
    r_scaled = r_nz / alpha

    term1 = (2.0 / (alpha * np.sqrt(np.pi))) * np.exp(-r_scaled**2)
    term2 = erf(r_scaled) / r_nz
    f_prime_vals[nonzero_r] = (term1 - term2) / r_nz

    return f_prime_vals
```

Mathematical Derivation of _radial_f_prime

We start with the *radial localization function*

$$f(r) = \frac{\operatorname{erf}\left(\frac{r}{\alpha}\right)}{r}, \quad r > 0,$$

where:

- $r = \|x - x_0\|$ is the Euclidean distance from x to the center x_0,
- $\alpha > 0$ is the length–scale of the map,
- erf is the Gauss error function:

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt.$$

Differentiate $f(r)$ for $r > 0$

Let

$$g(r) = \operatorname{erf}\left(\frac{r}{\alpha}\right),$$

so that

$$g'(r) = \frac{2}{\alpha\sqrt{\pi}} e^{-\left(\frac{r}{\alpha}\right)^2}.$$

Using the quotient rule:

$$f'(r) = \frac{g'(r) \cdot r - g(r)}{r^2}.$$

Substitute g and g'

We obtain:

$$f'(r) = \frac{\frac{2}{\alpha\sqrt{\pi}} e^{-\left(\frac{r}{\alpha}\right)^2} - \frac{\operatorname{erf}\left(\frac{r}{\alpha}\right)}{r}}{r}.$$

Handle $r = 0$.

From the Taylor expansion of $\operatorname{erf}(x)$ around $x = 0$, we have

$$f(r) \approx \frac{2}{\alpha\sqrt{\pi}} - \frac{2}{3\alpha^3\sqrt{\pi}} r^2 + O(r^4),$$

so

$$f'(0) = 0.$$

The code explicitly sets $f'(0) = 0$ to avoid division by zero.

Final formula implemented in code

For $r > 0$:

$$f'(r) = \frac{\frac{2}{\alpha\sqrt{\pi}} e^{-\left(\frac{r}{\alpha}\right)^2} - \frac{\operatorname{erf}\left(\frac{r}{\alpha}\right)}{r}}{r}.$$

For $r = 0$:

$$f'(0) = 0.$$

```
def fit(self, x, log_rho_exact=None, n_steps=1000):
    """
    Fits the density model to the data by building a sequence of maps.
    """
    m, n = x.shape
    tol = 1e-9 # Tolerance for Hessian

    # Starting map
    mean = np.mean(x, axis=0)
    z = x - mean
    std = np.sqrt(np.mean(np.sum(z**2, axis=1)) / n)
    z /= std

    self.preconditioning = {'mean': mean, 'std': std}
    self.maps = []

    # Iterative map building
    for i in range(n_steps):
        # 1. Select a center x0
        if np.random.rand() > 0.5:
            # Pick from the actual observations at their current normalized state
            idx = np.random.randint(m)
            x0 = z[idx]
        else:
            # Sample the target normal distribution to keep x0 from becoming too large
            x0 = np.random.randn(n)

        # 2. Calculate length-scale alpha
        alpha = self._calculate_alpha(x0, n, m)

        # 3. Calculate Gradient (G) and Hessian (H) of log-likelihood
        r = np.linalg.norm(z - x0, axis=1)
        f = self._radial_f(r, alpha)
        f_prime = self._radial_f_prime(r, alpha)

        term_G = n + np.dot(z, x0) - np.sum(z**2, axis=1)
        G = np.sum(term_G * f + r * f_prime)

        term_H1 = (n + r**2) * f**2
        term_H2 = 2 * r * f * f_prime
        term_H3 = (r * f_prime)**2
        H = -np.sum(term_H1 + term_H2 + term_H3)

        if np.abs(H) < tol:
            continue

        # 4. Calculate optimal step beta and constrain it
        beta = -G / H
        beta = np.clip(beta, -self.epsilon, self.epsilon)

        # 5. Apply the map to transform the data
        z = z + beta * f[:, np.newaxis] * (z - x0)

        # 6. Store the map's parameters
        self.maps.append({'x0': x0, 'alpha': alpha, 'beta': beta})

        if log_rho_exact is not None:
            # Estimate the KL divergence if the exact log density is provided
```

```
            kl = estimate_forward_KL(x, self, log_rho_exact)
            self.kl_history.append(kl)

        if (i+1) % (n_steps//10) == 0:
            if log_rho_exact is None:
                print(f"Step {i+1}/{n_steps} completed")
            else:
                kl = estimate_forward_KL(x, self, log_rho_exact)
                print(f"Step {i+1}/{n_steps} completed, KL estimate: {kl:.4f}")
```

Mathematical Explanation of `fit`

Setup (preconditioning)

Given samples $x_1, \dots, x_m \in \mathbb{R}^n$ from ρ, define

$$\bar{x} = \frac{1}{m}\sum_{i=1}^{m} x_i, \qquad z_i^{(0)} = \frac{x_i - \bar{x}}{\mathrm{std}}, \quad \mathrm{std} = \sqrt{\frac{1}{mn}\sum_{i=1}^{m} \|x_i - \bar{x}\|^2}.$$

We now work with $z \equiv z^{(0)}$ (mean 0, isotropic scale).

One Elementary Map

Pick a center $x_0 \in \mathbb{R}^n$ (either a data point z_j or a standard normal draw), choose a length scale $\alpha > 0$ (by the small-ball rule), and define for a scalar parameter $\beta \in \mathbb{R}$ the *radial* update

$$T_\beta(z) = z + \beta\, f(r)\,(z - x_0), \qquad r = \|z - x_0\|,$$

with

$$f(r) = \frac{\mathrm{erf}(r/\alpha)}{r}, \qquad f'(r) = \frac{\frac{2}{\alpha\sqrt{\pi}} e^{-(r/\alpha)^2} - \frac{\mathrm{erf}(r/\alpha)}{r}}{r}.$$

Log–likelihood for a Standard Gaussian Target

For the standard normal base $\mu(y) = (2\pi)^{-n/2}\exp(-\|y\|^2/2)$, the log–likelihood of the transformed cloud $\{y_i = T_\beta(z_i)\}$ is

$$\mathcal{L}(\beta) = \sum_{i=1}^{m} \Big(\log \mu(T_\beta(z_i)) + \log |\det \nabla T_\beta(z_i)| \Big).$$

Jacobian Eigenvalues of a Radial Map

This equation can be found in the Tabak–Turner paper (page 11):

$$\log |\det \nabla T_\beta| = (n-1)\log\big(1 + \beta f\big) + \log\big(1 + \beta(f + r f')\big).$$

Quadratic Expansion in β

Expand $\mathcal{L}(\beta)$ at $\beta = 0$ to second order:

$$\mathcal{L}(\beta) \approx \mathcal{L}(0) + G\,\beta - \tfrac{1}{2} H\,\beta^2,$$

where G and H are given in the Tabak–Turner paper, page 12:

$$G = \sum_{i=1}^{m} \Big[(n + z_i \cdot x_0 - \|z_i\|^2) f(r_i) \; + \; r_i f'(r_i) \Big],$$

$$H = \sum_{i=1}^{m} \Big[(n + r_i^2) f(r_i)^2 \; + \; 2 r_i f(r_i) f'(r_i) \; + \; (r_i f'(r_i))^2 \Big].$$

Newton Step for the Optimal Strength

The quadratic approximation

$$\mathcal{L}(\beta) \approx \mathcal{L}(0) + G\,\beta - \tfrac{1}{2} H\,\beta^2$$

is a concave parabola in β (when $H > 0$). Maximizing this requires setting its derivative with respect to β to zero:

$$\frac{d}{d\beta}\mathcal{L}(\beta) \approx G - H\,\beta = 0.$$

Solving for β gives

$$\beta^\star \approx \frac{G}{H}.$$

This is exactly the Newton–Raphson step for maximizing $\mathcal{L}$ using the first and second derivatives evaluated at $\beta = 0$. For stability, the resulting step is *clipped* to remain within the bounds $\beta^\star \in [-\varepsilon, \varepsilon]$, meaning

$$\beta^\star \;\leftarrow\; \begin{cases} \varepsilon, & \text{if } \beta^\star > \varepsilon, \\ -\varepsilon, & \text{if } \beta^\star < -\varepsilon, \\ \beta^\star, & \text{otherwise.} \end{cases}$$

This prevents excessively large updates, which could violate the smoothness assumptions of the quadratic approximation and cause instability in the flow.

Apply and Compose

Update all particles

$$z_i \leftarrow z_i + \beta^\star f(r_i)\,(z_i - x_0),$$

store the triplet $(x_0, \alpha, \beta^\star)$, and repeat the procedure for $k = 1, \ldots, n_{\text{steps}}$ to build a composition of elementary maps. (Optionally, estimate the forward KL to a known $\log \rho$ for monitoring.)

Do Only Nearby Points Get Pushed by the Gradient Update?

- The gradient $\nabla F_\ell(z)$ is nonzero everywhere, but its strength depends on the distance from the center $\tilde{z}_\ell$.
- It grows rapidly near $\tilde{z}_\ell$, then flattens as $\|z - \tilde{z}_\ell\| \gg \alpha$.
- Faraway points receive nearly constant, directionless pushes — effectively contributing nothing to transport.
- Thus, although the field has global support, it acts locally: only points within distance $\sim \alpha$ are meaningfully updated.

Intuition Behind the Objective

The goal is to choose the flow parameters β so that the transformed points look as if they were drawn from a standard Gaussian distribution. This is achieved by **maximizing the log-likelihood** of the transformed points under the Gaussian model, which is equivalent to **minimizing the Kullback–Leibler divergence**.

Likelihood as "How Likely Are My Points Under This Model?"

Suppose we observe points $z_1, \ldots, z_m$ and assume they come from some model density $p_\beta(z)$, where β controls the shape of the model (in our case, the parameters of the flow).

The *likelihood* is:

$$L(\beta) = \prod_{i=1}^{m} p_\beta(z_i),$$

and the *log-likelihood* is:

$$\mathcal{L}(\beta) = \sum_{i=1}^{m} \log p_\beta(z_i).$$

Here:

- A large $\mathcal{L}(\beta)$ means the model assigns high probability to the observed data $\Rightarrow$ good fit.
- A small $\mathcal{L}(\beta)$ means the model assigns low probability $\Rightarrow$ bad fit.

Connection to the Flow Setting

For our flow, the change-of-variables formula gives:

$$p_\beta(z) = \mu\big(T_\beta(z)\big)\ \left|\det \nabla T_\beta(z)\right|,$$

where μ is the standard Gaussian density. Thus:

$$\log p_\beta(z) = \log \mu\big(T_\beta(z)\big) + \log \left|\det \nabla T_\beta(z)\right|.$$

Maximizing $\mathcal{L}(\beta)$ therefore means choosing β so that the transformed points $T_\beta(z_i)$ look Gaussian *and* the transformation has the correct volume change (Jacobian determinant).

Why Flowing Target $\to$ Gaussian Doesn't Blow Up KL

The forward KL can be written as:

$$\mathrm{KL}(\rho_{\text{exact}} \,\|\, \rho_{\text{flow}}) = \mathbb{E}_{x\sim\rho_{\text{exact}}} \left[\log \rho_{\text{exact}}(x) - \log \rho_{\text{flow}}(x)\right].$$

The first term $\log \rho_{\text{exact}}(x)$ is constant for our dataset. Thus, minimizing KL is equivalent to maximizing:

$$\mathbb{E}_{x\sim\rho_{\text{exact}}} \left[\log \rho_{\text{flow}}(x)\right],$$

which, using the change-of-variables expression, becomes:

$$\mathbb{E}_{x\sim\rho_{\text{exact}}} \left[\log \mu(T(x)) + \log \left|\det \nabla T(x)\right|\right].$$

If T is a good map, $T(x) \sim \mathcal{N}(0, I)$ and $\log \mu(T(x))$ is maximized, making the KL small. If T is a bad map, the transformed points do not look Gaussian, $\log \mu(T(x))$ is small, and KL is large.

```python
def _transform(self, x_new):
    """Applies the full sequence of learned maps to new data."""
    if x_new.ndim == 1:
        x_new = x_new.reshape(1, -1)
    m, n = x_new.shape

    # Apply preconditioning
    z = (x_new - self.preconditioning['mean']) / self.preconditioning['std']

    # Calculate initial log Jacobian from preconditioning
```

```
    log_J = np.full(m, -n * np.log(self.preconditioning['std']))

    # Apply sequence of maps
    for p in self.maps:
        x0, alpha, beta = p['x0'], p['alpha'], p['beta']

        r = np.linalg.norm(z - x0, axis=1)
        f = self._radial_f(r, alpha)
        f_prime = self._radial_f_prime(r, alpha)

        # Update log Jacobian determinant
        term1 = 1 + beta * f
        term2 = 1 + beta * (f + r * f_prime)

        valid_map = (term1 > 0) & (term2 > 0)

        log_J_update = np.zeros(m)
        if np.any(valid_map):
            log_J_update[valid_map] = (n - 1) * np.log(term1[valid_map]) + np.log(term2[valid_map])
        log_J += log_J_update

        # Apply the map
        z = z + beta * f[:, np.newaxis] * (z - x0)
```

Mathematical explanation of `_transform`

Goal

Given *new* points $x^{\text{new}} \in \mathbb{R}^n$ (not the training data x used in `fit`), the function applies the *full composition* of the learned radial maps from training, while also keeping track of the log–determinant of the Jacobian of the transformation. The parameters (x_0, α, β) for each map are *exactly those learned during training* and stored in `self.maps`; no new x_0 is sampled at this stage.

Why Preconditioning Happens Again

During training (`fit`), the dataset x was preconditioned (mean-centered and rescaled) before fitting the maps. The resulting mean μ_{train} and scale σ_{train} were stored. When transforming *new* data x^{new}, we must bring it into the same normalized coordinate system as the training data before applying the learned maps. Thus the preconditioning step here:

$$z^{(0)} = \frac{x^{\text{new}} - \mu_{\text{train}}}{\sigma_{\text{train}}},$$

is applied only once to the new points — it does *not* "double precondition" the training data.

Initial Log–Jacobian from Preconditioning

Since preconditioning is an affine rescaling, the initial log–determinant is

$$\log J^{(0)} = -n \log \sigma_{\text{train}}.$$

Applying Each Learned Map

Each stored map is parameterized by (x_0, α, β), where x_0 is the fixed center selected during training for that map. For the current $z \in \mathbb{R}^n$:

Compute radial distance:

$$r = \|z - x_0\|.$$

Evaluate radial profile and derivative:

$$f(r) = \frac{\operatorname{erf}(r/\alpha)}{r}, \quad f'(r) = \frac{\frac{2}{\alpha\sqrt{\pi}} e^{-(r/\alpha)^2} - \frac{\operatorname{erf}(r/\alpha)}{r}}{r}.$$

Jacobian eigenvalues of the radial map: From the Tabak–Turner formula (p. 11),

$$\lambda_{\tan}(r) = 1 + \beta f(r) \quad \text{(multiplicity } n-1\text{)},$$

$$\lambda_{\mathrm{rad}}(r) = 1 + \beta\big(f(r) + r f'(r)\big).$$

Update log–Jacobian: If both $\lambda_{\tan}$ and λ_{rad} are positive (ensuring local invertibility), the log–determinant increment is:

$$\Delta \log J = (n-1)\log\big(1 + \beta f(r)\big) + \log\big(1 + \beta\big(f(r) + r f'(r)\big)\big),$$

and we update:

$$\log J \leftarrow \log J + \Delta \log J.$$

Update z: Apply the radial map:

$$z \leftarrow z + \beta f(r)\,(z - x_0).$$

Result: After all maps are applied, z contains the transformed samples in the target space, and $\log J$ contains the accumulated log–determinant of the Jacobian for the overall transformation.

```python
def log_prob(self, x_new):
    """
    Calculates the log probability density log(rho(x)) for new data points.

    Args:
        x_new (np.ndarray): New data points, shape (m, n).

    Returns:
        np.ndarray: The log probability for each point.
    """
    m, n = x_new.shape

    # Transform data to the target distribution space (y) and get log Jacobian
    y, log_J = self._transform(x_new)

    # Calculate log probability in the target Gaussian space
    log_prob_gaussian = -0.5 * np.sum(y**2, axis=1) - 0.5 * n * np.log(2 * np.pi)

    # Final log probability is log(rho(x)) = log(mu(y(x))) + log(J(x))
    return log_prob_gaussian + log_J
```

Mathematical explanation of `log_prob`

Change of Variables

The model density is defined by

$$\rho_{\mathrm{flow}}(x) \;=\; \mu\big(T(x)\big)\;\big|\det \nabla T(x)\big|\,.$$

Taking logs,

$$\log \rho_{\mathrm{flow}}(x) \;=\; \log \mu\big(T(x)\big) \;+\; \log\big|\det \nabla T(x)\big|\,.$$

Quantities Computed by `_transform`

Calling `_transform`(x) produces

$$(y,\ \log J)\ =\ \texttt{\_transform}(x), \qquad y\ =\ T(x), \quad \log J\ =\ \log|\det \nabla T(x)|\,.$$

The term $\log J$ is accumulated in `_transform` and passed here unchanged.

Gaussian Base

For the standard Gaussian base

$$\mu(y) = (2\pi)^{-n/2} \exp\!\left(-\tfrac{1}{2}\|y\|^2\right),$$

we have

$$\log \mu(y)\ =\ -\tfrac{1}{2}\|y\|^2\ -\ \tfrac{n}{2}\log(2\pi).$$

Final Expression Returned by `log_prob`

For each sample x_i with $y_i = T(x_i)$ and $\log J_i$ from `_transform`,

$$\log \rho_{\text{flow}}(x_i)\ =\ -\tfrac{1}{2}\|y_i\|^2\ -\ \tfrac{n}{2}\log(2\pi)\ +\ \log J_i$$

and the function returns the vector

$$\big(\log \rho_{\text{flow}}(x_1), \ldots, \log \rho_{\text{flow}}(x_m)\big).$$

Note: This function is called inside `estimate_forward_KL` to provide the per-sample $\log \rho_{\text{flow}}(x)$ values used in the KL divergence calculation.

```python
def sample(self, n_samples):
    """
    Generates samples from the learned density model.

    Args:
        n_samples (int): Number of samples to generate.

    Returns:
        np.ndarray: Generated samples, shape (n_samples, n).
    """
    # Sample from the standard normal distribution
    y_samples = np.random.randn(n_samples, len(self.maps[0]['x0']))

    # Apply the inverse of the learned maps to get samples in the original space
    z_samples = y_samples.copy()

    for p in reversed(self.maps):
        x0, alpha, beta = p['x0'], p['alpha'], p['beta']
        r = np.linalg.norm(z_samples - x0, axis=1)
        f = self._radial_f(r, alpha)

        # Apply the inverse map
        z_samples = z_samples - beta * f[:, np.newaxis] * (z_samples - x0)

    # Apply preconditioning to get back to original space
    return z_samples * self.preconditioning['std'] + self.preconditioning['mean']
```

Mathematical Explanation of `sample`

Draw from the *target* μ

Sample i.i.d.

$$y^{(i)} \sim \mu = \mathcal{N}(0, I_n), \qquad i = 1, \ldots, N,$$

forming $Y_{\text{samples}} \in \mathbb{R}^{N \times n}$.

Apply the inverse learned map in the normalized (preconditioned) space

During training, data were preconditioned to

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}},$$

and then mapped to the *target* via the composition

$$T \;=\; T_L \circ \cdots \circ T_1, \qquad T_\ell(u) = u + \beta_\ell \, f_\ell(r) \, (u - x_{0,\ell}), \quad r = \|u - x_{0,\ell}\|.$$

To synthesize z from y, apply inverses in reverse order:

$$z \;=\; T^{-1}(y) \;=\; T_1^{-1} \circ \cdots \circ T_L^{-1}(y),$$

with each inverse used in the code as

$$T_\ell^{-1}(u) = u - \beta_\ell \, f_\ell(r) \, (u - x_{0,\ell}), \qquad r = \|u - x_{0,\ell}\|.$$

Undo Preconditioning to Return to x-space

$$x_{\text{gen}} \;=\; z \, \sigma_{\text{train}} \;+\; \mu_{\text{train}}.$$

Now, the samples x_{gen} lie in the *source* space and are distributed according to the model's approximation ρ_{flow} to ρ. Here the *target* μ supplies randomness; T^{-1} transports it back to the source coordinates.

```python
import numpy as np

def sample_exact(n_samples):
    theta = np.random.randn(n_samples)
    r     = 1.0 + 0.1 * np.random.randn(n_samples)
    x = r * np.cos(theta)
    y = r * np.sin(theta)
    return np.stack([x, y], axis=1)

def log_rho_exact(xy):
    x = xy[:,0]
    y = xy[:,1]
    r     = np.sqrt(x**2 + y**2)
    theta = np.arctan2(y, x)

    log_p_theta = -0.5*theta**2 - 0.5*np.log(2*np.pi)

    log_p_r = (
        -0.5*((r - 1.0)/0.1)**2
        - 0.5*np.log(2*np.pi*0.1**2)
    )

    # log Jacobian term
    log_jac = -np.log(r)

    return log_p_theta + log_p_r + log_jac
```

Mathematical Explanation of `sample_exact` and `log_rho_exact`

The function `sample_exact` generates n_{samples} i.i.d. points $(x, y) \in \mathbb{R}^2$ from a distribution defined in *polar coordinates* (r, θ).

Distribution in Polar Coordinates

We define the radius r and angle θ as

$$\theta \sim \mathcal{N}(0, 1), \quad r \sim \mathcal{N}(\mu_r, \sigma_r^2), \quad \mu_r = 1.0, \quad \sigma_r = 0.1.$$

The mean radius $\mu_r = 1.0$ ensures that, on average, samples lie one unit away from the origin. In Cartesian coordinates, this corresponds to points clustered around the circumference of a circle of radius 1. The small standard deviation $\sigma_r = 0.1$ means that roughly 68% of the points will fall within the radial band $[0.9, 1.1]$, producing a thin annulus rather than a thick disk. Geometrically, this creates a ring-shaped cloud with a controlled radial "thickness" determined by σ_r.

For the angular component, θ is drawn from a standard normal distribution $\mathcal{N}(0, 1)$ rather than from a uniform distribution on $[0, 2\pi)$. This choice means that the majority of points have angles close to $\theta = 0$, with the density decreasing as $|\theta|$ increases. In Cartesian space, this produces a ring in which points are most densely clustered along the positive x-axis direction (corresponding to $\theta \approx 0$) and become sparser as we move around the circle. The spread in θ controls how far this high-density region extends, effectively determining how much of the ring is populated. By introducing this directional bias, we can test whether the density estimation method is capable of recovering both the radial structure (the annulus) and the non-uniform angular distribution.

Sampling Procedure

Given n_{samples}, the algorithm:

- Draws $\theta_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$ and $r_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(1.0, 0.1^2)$ for $i = 1, \ldots, n_{\text{samples}}$.
- Converts (r_i, θ_i) to Cartesian coordinates via

$$x_i = r_i \cos\theta_i, \quad y_i = r_i \sin\theta_i.$$

- Returns the matrix

$$\mathbf{X} = \begin{bmatrix} x_1 & y_1 \\ \vdots & \vdots \\ x_{n_{\text{samples}}} & y_{n_{\text{samples}}} \end{bmatrix} \in \mathbb{R}^{n_{\text{samples}} \times 2}.$$

Exact Log-Density

The function `log_rho_exact` computes the exact log-probability density $\log \rho(x, y)$ for points drawn from this distribution.

First, given a point (x, y), we recover polar coordinates:

$$r = \sqrt{x^2 + y^2}, \quad \theta = \text{atan2}(y, x).$$

`atan2(y, x)`

The function $\text{atan2}(y, x)$ returns the polar angle θ of the point (x, y) in the plane. Unlike the standard arctangent $\arctan(y/x)$, which is restricted to $(-\pi/2, \pi/2)$ and cannot distinguish between points in different quadrants, atan2 uses the signs of both x and y to determine the correct quadrant. It returns values in

$(-\pi, \pi]$, covering the entire circle without discontinuities at the $\pm\pi$ boundary.

For example, the points $(1, 1)$ and $(-1, -1)$ both satisfy $y/x = 1$, so $\arctan(y/x) = \pi/4$ in both cases. However, $\text{atan2}(1, 1) = \pi/4$ (first quadrant) while $\text{atan2}(-1, -1) = -3\pi/4$ (third quadrant), correctly distinguishing their angular positions. This makes atan2 especially useful for converting Cartesian coordinates (x, y) to polar coordinates (r, θ) in a continuous and unambiguous way.

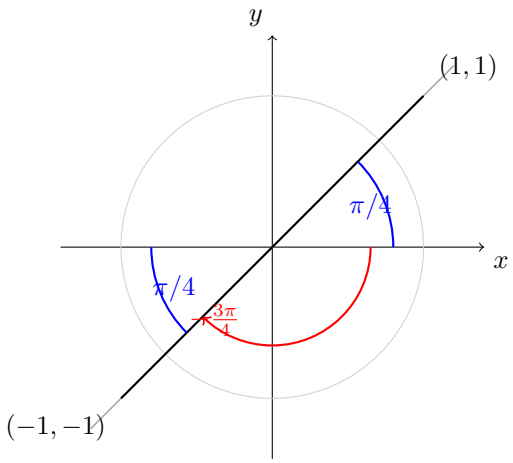


Figure 1: Arctan ambiguity: $\arctan(y/x) = \pi/4$ for both $(1, 1)$ and $(-1, -1)$ because they share slope 1. Quadrant-aware atan2 gives $\text{atan2}(1, 1) = \pi/4$ and $\text{atan2}(-1, -1) = -3\pi/4$, where $-3\pi/4 = 225°$ is the same direction in a different convention.

The joint density in polar coordinates factorizes as

$$p(r, \theta) = p_r(r)\, p_\theta(\theta),$$

where

$$p_\theta(\theta) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{\theta^2}{2}\right), \quad p_r(r) = \frac{1}{\sqrt{2\pi\sigma_r^2}} \exp\left(-\frac{(r - \mu_r)^2}{2\sigma_r^2}\right).$$

Jacobian Correction

Transforming from polar to Cartesian coordinates requires dividing by the Jacobian determinant

$$\left|\frac{\partial(r, \theta)}{\partial(x, y)}\right| = \frac{1}{r}.$$

Thus, the Cartesian density is

$$\rho(x, y) = \frac{p_r(r)\, p_\theta(\theta)}{r}.$$

Taking logs yields

$$\log \rho(x, y) = \log p_\theta(\theta) + \log p_r(r) - \log r.$$

This is exactly the expression computed in `log_rho_exact`.

```
x = sample_exact(1000)  # Sample from the true target density

flow = NormalizingFlow(n_p=500, epsilon=0.1)

n_steps = 600  # Number of steps to fit the model
flow.fit(x=x, log_rho_exact=log_rho_exact, n_steps=n_steps)
```

`sample_exact(1000)` calls the `sample_exact` function defined earlier, producing 1000 samples from the true target density. These samples are passed as input data to the training routine. `NormalizingFlow(n_p=500, epsilon=0.1)` creates a new instance of the `NormalizingFlow` class defined earlier in the code, with `n_p=500` setting the number of points each local radial map is designed to influence and `epsilon=0.1` setting the maximum allowed absolute step size for the parameter β in each map update. `n_steps = 600` specifies the number of gradient update iterations to run in the `fit` method. Finally, `flow.fit(...)` calls the `fit` method of the `NormalizingFlow` instance, where `x` is the array returned by `sample_exact`, `log_rho_exact` is the exact log-density function defined earlier in the program, and `n_steps` controls the number of update iterations. Inside `fit`, the method may call `estimate_forward_KL` to monitor the KL divergence between the learned flow density and the exact target density.

```
samples = flow.sample(1000)  # Generate samples from the learned density model

plt.scatter(samples[:, 0], samples[:, 1], s=1, alpha=0.5, label='Sampled Points')
plt.scatter(x[:, 0], x[:, 1], s=1, alpha=0.5, color='red', label='Original Points')
plt.title('Samples from the Learned Density Model')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
```

Sampling and Visualizing the Learned Distribution

This section of the code generates and visualizes new samples from the learned normalizing flow. First, `samples = flow.sample(1000)` draws 1000 points by sampling from the standard normal base distribution in the flow's latent space and applying the inverse of the learned transformation maps to return to the original data space. The `plt.scatter` command then plots these generated samples in blue, while the second `plt.scatter` plots the original target distribution points in red. By visually comparing the two sets of points, we can assess how effectively the learned flow transforms a standard normal distribution into the target distribution.

```
plt.plot(flow.kl_history)
plt.title('KL Divergence History')
plt.xlabel('Step')
plt.ylabel('KL')
plt.yticks([0, 0.5, 1.0, 1.5])
```

Plotting KL Divergence History

This code visualizes the evolution of the Kullback–Leibler (KL) divergence during training. It plots the stored history of KL values (`flow.kl_history`) over the optimization steps, providing a direct way to assess how effectively the flow reduces the divergence between the learned distribution and the target distribution. A decreasing curve indicates successful training progress.

```
# Test Case 1
# This creates a crescent-shaped distribution, similar to the paper
m = 1000
angle = np.random.uniform(-np.pi/2, np.pi/2, m)
radius = 1.0 + 0.1 * np.random.randn(m)
x = np.zeros((m, 2))
x[:, 0] = radius * np.cos(angle)
x[:, 1] = radius * np.sin(angle)
x += 0.02 * np.random.randn(m, 2)
```

```
# Train the model
# Using parameters similar to the paper.
# n_p = 50
# n_steps = 500.
model = NormalizingFlow(n_p=50, epsilon=0.5)
model.fit(x, n_steps=600)
```

Mathematical Explanation of the Crescent Distribution Generation

This test case creates $m = 1000$ two-dimensional points $(x, y) \in \mathbb{R}^2$ forming a crescent-shaped distribution. The shape is generated in *polar coordinates* (r, θ), then converted to Cartesian coordinates and perturbed with Gaussian noise.

Distribution in Polar Coordinates

The angle θ is sampled uniformly from a half-circle:

$$\theta \overset{\text{i.i.d.}}{\sim} \mathcal{U}\left(-\frac{\pi}{2}, \frac{\pi}{2}\right).$$

This restriction ensures that all points lie in the right-hand side of the plane, covering only 180° rather than the full circle. It is the key factor that creates the open, crescent-like geometry. The radius r is sampled from a normal distribution centered at 1.0 with small variance:

$$r \overset{\text{i.i.d.}}{\sim} \mathcal{N}(1.0, 0.1^2).$$

This means that most points will lie close to the unit circle, with a radial "thickness" controlled by $\sigma_r = 0.1$.

Conversion to Cartesian Coordinates

Given (r_i, θ_i), each point is mapped to Cartesian coordinates via

$$x_i = r_i \cos\theta_i, \quad y_i = r_i \sin\theta_i.$$

The result is a semi-circular arc of points centered around $(0, 0)$ with radius ≈ 1.0.

Adding Local Noise

To avoid a perfectly smooth curve and to better approximate real-world noisy data, we add independent Gaussian perturbations to both coordinates:

$$x_i \leftarrow x_i + \epsilon_{x,i}, \quad y_i \leftarrow y_i + \epsilon_{y,i},$$

where

$$\epsilon_{x,i}, \epsilon_{y,i} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 0.02^2).$$

This step introduces small variations in both radial and tangential directions, giving the crescent a more scattered appearance.

```
# Results
# Create a grid of points to evaluate the learned density
grid_res = 1000
x_range = np.linspace(-1.5, 1.5, grid_res)
y_range = np.linspace(-1.5, 1.5, grid_res)
xx, yy = np.meshgrid(x_range, y_range)
```

```
grid_points = np.c_[xx.ravel(), yy.ravel()]

# Calculate the probability on the grid
log_p = flow.log_prob(grid_points)
p = np.exp(log_p).reshape(grid_res, grid_res)

# Plotting
plt.style.use('seaborn-v0_8-whitegrid')
fig = plt.figure(figsize=(12, 5))

# Plot 1: Original Data
ax1 = fig.add_subplot(1, 2, 1)
ax1.scatter(x[:, 0], x[:, 1], s=10, alpha=0.6, c='blue')
ax1.set_title("Original Data Sample (m=1000)")
ax1.set_xlabel("x1")
ax1.set_ylabel("x2")
ax1.set_aspect('equal', adjustable='box')
ax1.set_xlim(-1.5, 1.5)
ax1.set_ylim(-1.5, 1.5)

# Plot 2: Learned Density
ax2 = fig.add_subplot(1, 2, 2)
# Use LogNorm for better visualization of low-density areas
contour = ax2.contourf(xx, yy, p, levels=15, cmap='viridis', norm=LogNorm())
ax2.contour(xx, yy, p, levels=15, colors='white', linewidths=0.5, alpha=0.7)
fig.colorbar(contour, ax=ax2, label="Probability Density")
ax2.set_title("Learned Density with Normalizing Flow")
ax2.set_xlabel("x1")
ax2.set_ylabel("x2")
ax2.set_aspect('equal', adjustable='box')
ax2.set_xlim(-1.5, 1.5)
ax2.set_ylim(-1.5, 1.5)

plt.tight_layout()
plt.show()
```

Evaluating and Visualizing the Learned Density

This section evaluates the Normalizing Flow by testing it on a set of evenly distributed points across the domain and comparing the resulting learned density to the original data.

- An evenly spaced grid of points is created across the region $[-1.5, 1.5] \times [-1.5, 1.5]$.
- The trained model is applied to each grid point to estimate the probability density at that location.
- Those likelihood values are then arranged back into the grid's shape so that they can be plotted as a smooth heatmap of the learned density.
- Two plots are generated: one showing the original data samples and another showing the learned density field.
- Using an evenly spaced grid ensures that the learned distribution is evaluated uniformly across the entire domain, allowing for a fair visual comparison to the true data distribution.

4.10 Xuan's LP Check Method

On the next page!

1 Checking Convexity Intepolation of points

Questions: Given a bunch of points $(x_i.y_i)_{i=1,...,n}$, with $x_i \in \mathbb{R}^d, y_i \in \mathbb{R}$, can this interpolate a convex function?

Idea: Points $(x_i.y_i)$ can interpolate a convex function means that there exist a convex function $f : \mathbb{R}^d \to \mathbb{R}$ such that $f(x_i) = y_i$. We have such convex function if and only if for all $i = 1, ..., n$, there exist subgradient vector $g_i \in \mathbb{R}^d$ such that for all $j = 1, ..., n$

$$y_j \geqslant y_i + g_i^T(x_j - x_i)$$

This means for every point (x_i, y_i), there is a supporting hyperplane that passes through (x_i, y_i) and keeps all other points on or above it. (Prove: the necesary condition already follows from definition of that convex function. For sufficient condition: we can build a convex function $f(x) = \max_{i=1,..,n}\{y_i + g_i^T(x - x_i)\}$, and $f(x_i) = y_i$ since we have for all $k \neq i$, $y_k + g_k^T(x_i - x_k) \leqslant y_i$.)

Now using this idea, we can construct a linear program: For $i, j = 1, ..., n$,

$$y_j \geqslant y_i + g_i^T(x_j - x_i)$$

where $x_i, x_j \in \mathbb{R}^d, y_i, y_j \in \mathbb{R}$ are known vectors and values. And only g_i's are unknown vectors. If this linear program is feasible, then we know there exist such subgradient vectors, so points can interpolate a convex function. If the linear program is not feasible, then points can not interpolate convex functions.

In the original problem, there are totally N points to check, and there are N inequlaities for each point. So the computational cost would be high if we have large amount of points. To deal with that, we can check the dual form of inequalities for each points. We can do that by using Farkas' lemma:

Either $Ax \leqslant b$ has a solution $x \in \mathbb{R}^n$, or $A^T y = 0$ has a solution $y \geqslant 0$ with $b^T y < 0$.

Apply Farkas Lemma to our original inequalities for each point i, we have $y_j \geqslant y_i + g_i^T(x_j - x_i)$ is not feasible if and only if $\sum_j \alpha_j(x_j - x_i) = 0$ has a solution $\alpha \in \mathbb{R}_+^n$ with $\sum_j \alpha_j(y_j - y_i) < 0$. Since CVXPY does not take strict inequality, we instead make a LP formation:

$$\min_{\alpha \geqslant 0} \sum_j \alpha_j(y_j - y_i)$$
$$\text{s.t.} \sum_j \alpha_j(x_j - x_i) = 0$$

We check the LP is unbounded or not. If it is unbounded, then it means that $\sum_j \alpha_j(x_j - x_i) = 0$ has a solution, and then our original inequality is infeasible. Vice versa.

We have implemented an LP blackbox for dealing this problem. There are several places to improve: 1. using different formulation of LP. 2. using different packgaes. 3. using parallelism to check the LP for all points at once.

2 Adaptable Step Size via Convexity Check Using LP

In the original implementation of Tabak and Turner, each update to the potential function was carried out with a fixed coefficient β multiplying the chosen basis function $F_{k+1}(z)$. Specifically, after k steps the

updated potential had the form

$$u_\beta^{k+1}(z) = u_\beta^k(z) + \beta_{k+1} F_{k+1}(z),$$

where both u_β^k and F_{k+1} are convex functions.

If $\beta_{k+1} \geqslant 0$, convexity of the updated potential is automatically preserved. Allowing $\beta_{k+1} < 0$ can improve flexibility and fitting power, but convexity of u_β^{k+1} is no longer guaranteed.

2.1 Motivation

To ensure the transport map remains the gradient of a globally convex potential on the data domain, we empirically test convexity on a fixed sample cloud $\{z_0^{(i)}\}_{i=1}^m$. We check the finite-family of affine minorant inequalities: for every point i, there exists subgradient vector $g_i \in \mathbb{R}^n$ such that

$$u_\beta^{k+1}\Big(z_0^{(j)}\Big) \;\geqslant\; u_\beta^{k+1}\Big(z_0^{(i)}\Big) \;+\; g_i^\top\big(z_0^{(j)} - z_0^{(i)}\big)$$

for all points j. This is equivalent to the convex interpolation problem on the sample set. As we fix a collection of sample points $\{z_0^{(i)}\}_{i=1}^m \subset \mathbb{R}^n$ drawn once at initialization. At each iteration we evaluate the updated potential at these points:

$$\big(z_0^{(i)},\, u_\beta^{k+1}(z_0^{(i)})\big)_{i=1}^m.$$

and we check if these points can interpolate a convex function using the LP blackbox.

2.2 Adaptable β Rule

- If the LP is feasible, we accept the candidate β_{k+1}.
- If the LP is infeasible, we reduce the absolute value of β_{k+1} (e.g. via multplying a constant factor) until feasibility is restored.

This ensures that each update preserves empirical convexity relative to the chosen sample points, preventing invalid potentials while maintaining flexibility.

Algorithm.

(1) Set backtracking limit $B = 20$, shrink factor $s = 1.25$, and an initial candidate $\beta^{\text{cand}} < 0$.

(2) **For** $t = 1, \ldots, B$:

  (1) Define $u_\beta(z) = u^k(z) + \beta^{\text{cand}} F_{k+1}(z)$.

  (2) Let $x_i \leftarrow z_0^{(i)}$ and $y_i \leftarrow u_\beta(x_i)$ for $i = 1, \ldots, m$.

  (3) Run the LP convex interpolation check on $\{(x_i, y_i)\}_{i=1}^m$.

  (4) **If** feasible, accept $\beta_{k+1} \leftarrow \beta^{\text{cand}}$ and **stop**.

  (5) **Else** set $\beta^{\text{cand}} \leftarrow \beta^{\text{cand}}/1.25$ and continue.

2.3 Some Expreriment Results

The implementation details, together with reproducible code and example notebooks, are available on GitHub:

https://github.com/echen347/polymath25-ML/blob/main/Normalizing%20Flows/Normalizing_flow_and_convexity_check%20(updated%20with%20Jacobian%20and%20inverse%20map%20computation%20and%20convexity%20check%20using%20LP).ipynb

Here we track the forward KL divergence value with a Monte Carlo estimate at each update. With the LP-based convexity check active (backtracking by 1/1.25 on infeasibility), the KL decreases steadily over 600 iterations (Fig. 1). From step 60 to 600, it drops from 0.8570 to 0.0438 (about a 95% reduction).

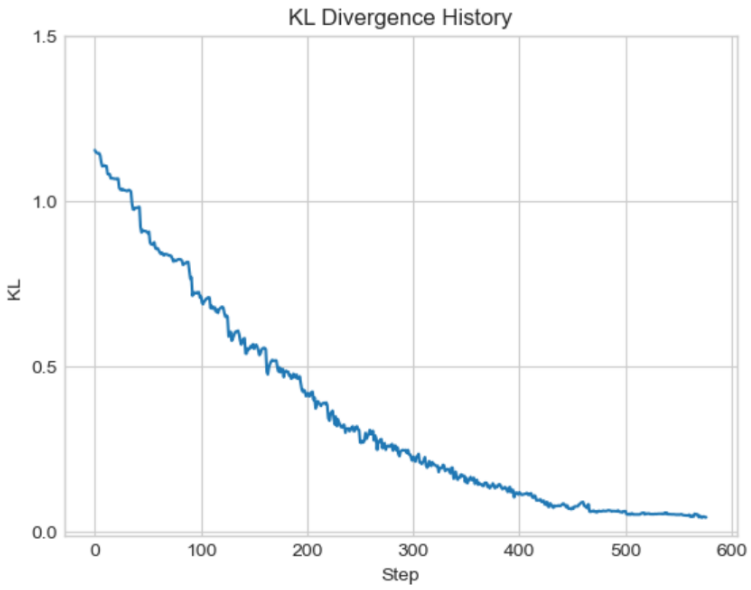


Figure 1: KL divergence per iteration.

2.4 Summary

This modification introduces an adaptive mechanism of β. The LP check is not computationally efficient right now. The main aim of this method is to provide a principled safeguard of global convexity.

4.11 Convexity Interpolation

On the next page!

Convexity Interpolation

Tony Deng

July 2025

1 Theory

Let $\varphi : \mathbb{R}^d \to \mathbb{R}$. We say φ is convex at x if there exists a neighborhood U of x such that for all $z \in U$,

$$\varphi(z) - \varphi(x) \geq \nabla\varphi(x) \cdot (z - x)$$

The problem we are trying to solve is the following: Let $F_1, ..., F_n : \mathbb{R} \to \mathbb{R}$ be convex functions and $r_1, ..., r_n : \mathbb{R}^d \to \mathbb{R}$ be defined by $r_i = \left\|x - x_0^i\right\|$. We want to determine the maximum β_n such that

$$\frac{1}{2}\|x\|^2 + \sum_1^{n-1} \beta_i F_i \circ r_i + \beta_n F_n \circ r_n$$

is convex. Therefore, for all x, we need to find a neighborhood U_x such that for all $z \in U_x$,

$$\begin{aligned}
\beta_n &\geq -\frac{(1/2)\|z\|^2 + \sum_1^{n-1} \beta_i F_i(r_i(z)) - (1/2)\|x\|^2 + \sum_1^{n-1} \beta_i F_i(r_i(x)) - \nabla((1/2)\|\cdot\|^2 + \sum_1^{n-1} \beta_i F_i \circ r_i) \mid_x \cdot(z - x)}{F_n(r_n(z)) - F_n(r_n(x)) - \nabla(F_n \circ r_n) \mid_x \cdot(z - x)} \\
&= -\frac{(1/2)\|z - x\|^2 + \sum_1^{n-1} \beta_i F_i(r_i(z)) - \beta_i F_i(r_i(x)) - \beta_i (F_i'(r_i)/r_i)(x - x_0^i) \cdot (z - x)}{F_n \circ r_n(z) - F_n \circ r_n(x) - (F_n'(r_n)/r_n)(x - x_0^n) \cdot (z - x)} \\
&= -\frac{(1/2)\|z - x\|^2 + \sum_1^{n-1} \beta_i F_i(r_i(z)) - \beta_i F_i(r_i(x)) - \beta_i f_i(r_i)(x - x_0^i) \cdot (z - x)}{F_n \circ r_n(z) - F_n \circ r_n(x) - f_n(r_n)(x - x_0^n) \cdot (z - x)}
\end{aligned}$$

If the demonenator is zero, we define the fraction to be $-\infty$. Since z is close to x and hence $r_i(z)$ is close to $r_i(x)$, we can use the Taylor approximation. For $i \in [n]$, can approximate

$$F_i(r_i(z)) - F_i(r_i(x)) \approx F_i'(r_i(x))(r_i(z) - r_i(x)) + \frac{1}{2}F_i''(r_i(x))(r_i(z) - r_i(x))^2$$

and also

$$r_i(z) - r_i(x) \approx \nabla r_i \mid_x \cdot(z - x) = \frac{(x - x_0^i) \cdot (z - x)}{r_i(x)}$$

The terms cancel and we have

$$F_i(r_i(z)) - F_i(r_i(x)) - \nabla(F_i \circ r_i) \mid_x \cdot(z - x) \approx \frac{1}{2}F_i''(r_i(x))(r_i(z) - r_i(x))^2$$

Therefore, the fraction is approximately

$$\beta_n \geq -\frac{\|z - x_0^n\|^2 + \sum_1^{n-1} \beta_i F_i''(r_i(x))(r_i(z) - r_i(x))^2}{F_n''(r_n(x))(r_n(z) - r_n(x))^2}$$

If $x \neq x_0^n$, we can choose z in the neighborhood such that $\|z - x_0^n\| = r_n(z) = r_n(x) = \|x - x_9\|$. Hence, the denominator is zero and the fraction is $-\infty$. Therefore, to compute the maximum, we only need to consider $x = x_0^n$. We have

$$\beta_n \geq -\frac{\|z - x_0^n\|^2 + \sum_1^{n-1} \beta_i F_i''(r_i(x_0^n))(r_i(z) - r_i(x_0^n))^2}{F_n''(r_n(z))r_n(z)^2}$$

We can use the Taylor approximation

$$r_i(z) - r_i(x_0^n) \approx \frac{(x_0^n - x_0^i) \cdot (z - x_0^n)}{r_i(x_0^n)}$$

and hence

$$\beta_n \geq -\frac{\|z - x_0^n\|^2 + \sum_1^{n-1} \beta_i F_i''(r_i(x_0^n))(\frac{(x_0^n - x_0^i) \cdot (z - x_0^n)}{r_i(x_0^n)})^2}{F_n''(r_n(z))r_n(z)^2}$$

We want to maximize this fraction. Since $r_n(z) = \|z - x_0\|$, we can write this fraction as

$$-\frac{1}{F''(r_n(z))} - \frac{1}{F''(r_n(z))}\left(\sum_1^{n-1} \beta_i F''(r_i(x_0^n))\left(\frac{(x_0^n - x_0^i) \cdot (z - x_0^n)}{\left\|x_0^n - x_0^i\right\| \|z - x_0^n\|}\right)^2\right)$$

Since $r_n(z) \to 0$, we can approximate $F''(r_n(z))$ by $\lim_{r\to 0} F''(r)$. We can treat this value as a fixed constant. If F is defined by $F(z) = z\,\mathrm{erf}(z/\alpha) + (\alpha/\sqrt{\pi})\exp(-z^2/\alpha^2)$ then $\lim_{r\to 0} F''(r) = 2/(\alpha\sqrt{\pi})$. Therefore, we only need to minimize the following quantity

$$\sum_1^{n-1} \beta_i F''(r_i(x_0^n)) \left(\frac{(x_0^n - x_0^i)\cdot(z - x_0^n)}{\|x_0^n - x_0^i\|\,\|z - x_0^n\|} \right)^2$$

We will make the following substitution for convenience. Let

$$c_i = \beta_i F''(r_i(x_0^n)) \qquad x_i = \frac{x_0^n - x_0^i}{\|x_0^n - x_0^i\|} \qquad u = \frac{z - x_0^n}{\|z - x_0^n\|}$$

The problem become

$$\min_u \sum_1^{n-1} c_i (x_i \cdot u)^2$$

subject to $\|u\| = 1$. We know $(x_i \cdot u)^2 = u^T x_i x_i^T u$, so we need to minimize

$$\min_{\|u\|=1} u^T \left(\sum_1^{n-1} c_i x_i x_i^T \right) u$$

This is a Rayleigh quotient, and the minimum is the smallest eigenvalue of

$$\sum_1^{n-1} c_i x_i x_i^T$$

(this is a symmetric real matrix so the smallest eigenvalue is guaranteed to exist).

2 Test

For simplicity, assume

$$\begin{aligned} F(z) &= z\,\mathrm{erf}(z) + (1/\sqrt{\pi})\exp(-z^2) \\ d &= 1 \\ n &= 2 \\ \beta_1 &= 1 \\ x_0^1 &= -1 \\ x_0^2 &= 1 \end{aligned}$$

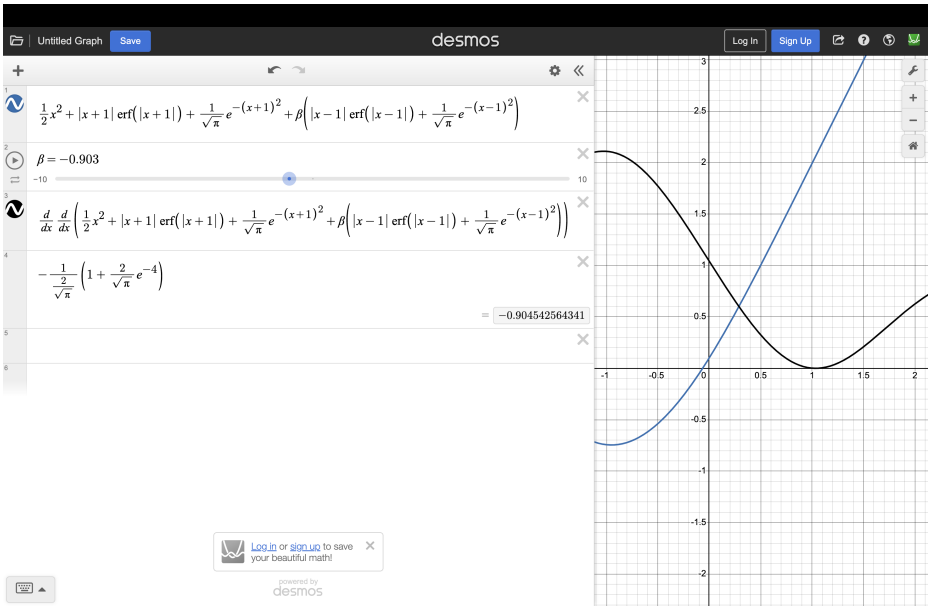
Therefore, we want to find the minimum β_2 such that

$$\frac{1}{2}x^2 + |x+1|\,\mathrm{erf}(|x+1|) + (1/\sqrt{\pi})\exp(-(x+1)^2) + \beta_2|x-1|\,\mathrm{erf}(|x-1|) + (1/\sqrt{\pi})\exp(-(x-1)^2)$$

is a convex function. By calculation, we know $x_i = 1$ and $c_i = (2/\sqrt{\pi})e^{-4}$, so the minimum eigenvalue of $\sum c_i x_i x_i^T$ is simply c_i. Moreover, we know $\lim_{r\to 0} F''(r) = 2/\sqrt{\pi}$, so the beta threshold we calculated is

$$-\frac{1}{2/\sqrt{\pi}}\left(1 + (2/\sqrt{\pi})e^{-4}\right) = -0.905$$

I also plotted the function on desmos and the beta threshold it calculated is between -0.903 and -0.904. The results are very close to each other. It also shows that F'' achieves its minimum at $x_0^2 = 1$, so this is the point closest to non-convexity.


Untitled Graph
Save
desmos
Log In
Sign Up
$\frac{1}{2}x^2+|x+1|\operatorname{erf}(|x+1|)+\frac{1}{\sqrt{\pi}}e^{-(x+1)^2}+\beta\left(|x-1|\operatorname{erf}(|x-1|)+\frac{1}{\sqrt{\pi}}e^{-(x-1)^2}\right)$
$\beta=-0.903$
−10
10
$\frac{d}{dx}\frac{d}{dx}\left(\frac{1}{2}x^2+|x+1|\operatorname{erf}(|x+1|)+\frac{1}{\sqrt{\pi}}e^{-(x+1)^2}+\beta\left(|x-1|\operatorname{erf}(|x-1|)+\frac{1}{\sqrt{\pi}}e^{-(x-1)^2}\right)\right)$
$-\frac{1}{\frac{2}{\sqrt{\pi}}}\left(1+\frac{2}{\sqrt{\pi}}e^{-4}\right)$
$=-0.904542564341$
Log in or sign up to save your beautiful math!
powered by desmos

4.12 Local Normalizing Flows

On the next page!

Local Normalizing Flows

Tony Deng

1 Introduction

In [1], Tabak and Turner proposed to write the transportation map as a composition of elementary maps, each is a perturbation of the identity. According to this map, the position of particle i at time step $n+1$ can be deduced from the following formula:

$$x_i^{n+1} = x_i^n + \beta f(\left\|x_i^n - \tilde{x}^{n+1}\right\|)(x_i^n - \tilde{x}^{n+1})$$

where x_i^n denote the particle's position at time step n. However, since we can not guarantee the composition of the gradient of a convex map is still the gradient of a convex map, we propose to write the transportation map as a linear combination of the elementary maps. According to this map, the particle at time step $n+1$ can be deduced from the following formula:

$$x_i^{n+1} = x_i^n + \beta f(\left\|x_i^0 - \tilde{x}^{n+1}\right\|)(x_i^0 - \tilde{x}^{n+1})$$

We can also expand the formula and write the whole expression as

$$x_i^{n+1} = x_i^0 + \sum_{k=1}^{n+1} \beta_k f(\left\|x_i^0 - \tilde{x}^k\right\|)(x_i^0 - \tilde{x}^j)$$

Let J_n denote the Jacobian matrix of the map T at time step n. If $\tilde{x}^{n+1} = x_i^0$, then $J_{n+1} = J_n$. If otherwise, we have the following recursive formula:

$$J_{n+1} = J_n + \beta f(r_{n+1})I + \beta \frac{f'(r_{n+1})}{r_{n+1}}(x_i^0 - \tilde{x}^{n+1})(x_i^0 - \tilde{x}^{n+1})^T$$

I failed to express the determinant of J_{n+1} from the determinant of J_n by a simple, analytic formula, but the determinant of any symmetric matrix can be computed in $O(d^3)$ time. Therefore, we can store J_n as we iterate through n and compute the determinant by python.

2 Log determinant and log likelihood

I find some methods to calculate

$$\frac{\partial}{\partial \beta} \log(\det J_{n+1}) \mid_{\beta=0}$$

We can write the expression as

$$\begin{aligned} \frac{\partial}{\partial \beta} \log(\det J_{n+1}) &= \frac{\partial/\partial\beta(\det J_{n+1}) \mid_{\beta=0}}{\det J_{n+1} \mid_{\beta=0}} \\ &= \frac{\operatorname{tr}(\operatorname{adj}(J_n)(\partial/\partial\beta)J_{n+1} \mid_{\beta=0})}{\det J_n} \qquad \text{by Jacobi's formula} \end{aligned}$$

Here, $\operatorname{adj}(J_n)$ is the adjugate matrix of J_n. If J_n is invertible, $\operatorname{adj}(J_n) = J_n^{-1} \det J_n$ and the expression can be written as

$$\frac{\partial}{\partial \beta} \log(\det J_{n+1}) = \operatorname{tr}\left(J_n^{-1} \frac{\partial}{\partial \beta} J_{n+1} \mid_{\beta=0}\right)$$

$\partial/\partial\beta J_{n+1}$ is the entry-wise differentiation with respect to β. In particular, if $x_i^0 = \tilde{x}^{n+1}$, we know $\partial/\partial\beta J_{n+1} \mid_{\beta=0} = 0$ is the all zero matrix. If otherwise,

$$\frac{\partial}{\partial \beta} J_{n+1} \mid_{\beta=0} = f(r_{n+1})I + \frac{f'(r_{n+1})}{r_{n+1}}(x_i^0 - \tilde{x}^{n+1})(x_i^0 - \tilde{x}^{n+1})^T$$

In this case, therefore,

$$\frac{\partial}{\partial\beta}\log(\det J_{n+1})\mid_{\beta=0}=\operatorname{tr}\left(J_n^{-1}\left(fI+\frac{f'}{r}(x_i^0-\tilde{x}^{n+1})(x_i^0-\tilde{x}^{n+1})^T\right)\right)$$

Let μ be the pdf of independent normal distribution. We know

$$\begin{aligned}\frac{\partial}{\partial\beta}\log\mu(T(x))\mid_{\beta=0}&=\frac{\partial}{\partial\beta}\left(-\frac{n}{2}\log(2\pi)-\frac{1}{2}\left\|T(x)\right\|^2\right)\mid_{\beta=0}\\&=-T(x)\mid_{\beta=0}\cdot\frac{\partial}{\partial\beta}T(x)\mid_{\beta=0}\\&=-x_i^n\cdot\frac{\partial}{\partial\beta}x_i^{n+1}\mid_{\beta=0}\\&=-x_i^n\cdot f(\left\|x_i^0-\tilde{x}^{n+1}\right\|)(x_i^0-\tilde{x}^{n+1})\end{aligned}$$

Therefore, if $x_i^0\neq\tilde{x}^{n+1}$, the gradient of the log likelihood is determined by

$$\begin{aligned}\frac{\partial}{\partial\beta}L&=\frac{\partial}{\partial\beta}\log(J_{n+1})+\log\mu(T(x))\mid_{\beta=0}\\&=\operatorname{tr}\left(J_n^{-1}\left(fI+\frac{f'}{r}(x_i^0-\tilde{x}^{n+1})(x_i^0-\tilde{x}^{n+1})^T\right)\right)-x_i^0\cdot f(\left\|x_i^n-\tilde{x}^{n+1}\right\|)(x_i^0-\tilde{x}^{n+1})\end{aligned}$$

If $x_i^0=\tilde{x}^{n+1}$, the gradient of the log likelihood is simply 0.

References

[1] A family of nonparametric density estimation algorithms by E.G. Tabak, C.V. Turner (2013): available at https://core.ac.uk/download/pdf/80364696.pdf